From One Seed to Seven Complete Worked Realizations

Complete Seed-to-Reconstruction Edition

Projector-Resolved Seed Calculus from the Primitive Datum to Final Reconstruction

Algebra, Signals, Quantum Information, Probability, Arithmetic, Physics, and Nonlinear Transport

A complete derivation of the calculus, followed by expanded step-by-step proofs, exact symbolic calculations, layout-audited figures, Python certificates, and a candidate integer sequence

A common finite calculus is useful only if its axioms can be instantiated, its domain objects can be solved, and its synthesis maps recover the original data without hidden loss. This volume performs that check seven times in mathematically different settings.

Komorebi P.

Abstract

This seed-to-reconstruction edition begins at the primitive datum rather than at the first projector. It develops, in dependency order, the typed operator grammar, generated hull, seed rank, relation quotient, address system, packet projectors, transfer algebra, path calculus, exact refinement, transform atlases, redundant multi-atlas synthesis, nonlinear interaction tensors, obstruction packets, dense-core completion, and the final reconstruction certificate. A single cyclic seed is then carried through every stage without changing the declared grammar.

Only after that complete derivation does the document retain and continue the original seven detailed domain realizations. No earlier worked example, formula, figure, proof, or source listing has been removed.

This document gives seven independent finite realizations of Projector-Resolved Seed Calculus. The same four operations are proved and then instantiated in every area:

$$\boxed{\text{declare complete packets} \longrightarrow \text{analyze} \longrightarrow \text{transfer or refine} \longrightarrow \text{synthesize.}}$$

The worked areas are finite linear algebra, polyphase/Fourier/wavelet signal analysis, a two-qubit Bell circuit, conditional expectation and Bayesian coarsening, a reciprocal root-filter sequence with an explicit closed form and integer recurrence, a coupled-oscillator problem, and a combined nonlinear-filter/holonomy obstruction example.

All displayed exact identities were recomputed independently in SymPy. The seed-to-reconstruction edition contains thirty-two exact tests: eleven foundational seed-to-finish tests plus the retained twenty-one domain tests–seven implementation certificates, seven independent re-derivations, and seven additional step-level audits. It contains sixty-one reproducible figure pairs: twenty-three foundational figures plus the retained thirty-eight domain figures. Every intended reconstruction, operator identity, Bayesian factorization, ordinary differential equation, and flat-holonomy check has exact residual zero. The sole nonzero residual is an intentionally perturbed holonomy packet whose squared Frobenius norm is exactly $8/5$.

The document proves that the finite calculus is internally consistent, non-vacuous, and operational across the seven declared realizations. It does not claim that every infinite-dimensional problem, every physical theory, or every mathematical object automatically satisfies the same hypotheses.

Exact claim and proof boundary

Result proved here

Let a finite realization supply a complete family of pairwise annihilating projectors, an admissible operator or nonlinear map, and a declared synthesis law. Then the object and operator reconstruction identities follow exactly. Each of the seven chapters verifies the hypotheses in a concrete domain and solves one complete worked problem.

Result complete seed-to-finish chain

The new first part proves the dependency chain from a primitive seed and fixed operator grammar through generated hulls, relation quotients, packet coordinates, transfer blocks, refinement, multi-atlas synthesis, nonlinear interactions, obstruction packets, completion, and final residual certificates. The seven original worked areas are retained in full after that construction.

Boundary not claimed

A finite collection of realizations does not prove universal applicability to every category or continuum limit. It also does not establish historical novelty merely because a formula is generated cleanly. The arithmetic chapter therefore labels its integer sequence as a candidate OEIS submission rather than asserting that no equivalent formulation exists anywhere in the literature.

Reading map

The complete logical dependency is

primitive seed and grammar $\longrightarrow$ generated hull and relations $\longrightarrow$ addresses and projectors
$\longrightarrow$ transfer, refinement, nonlinear extension, completion $\longrightarrow$ master end-to-end example
$\longrightarrow$ seven domain realizations
$\longrightarrow$ cross-domain certificate $\longrightarrow$ code and reproducibility appendices

Readers interested mainly in the mathematical proof may read Chapters 1–17. Readers interested in exact symbolic implementation may continue to the code appendices.

Contents

Abstract i

Exact claim and proof boundary ii

Reading map iii

I The Common Finite Calculus 1

1 The primitive seed and the typed operator grammar 2

1.1 Why the seed must come before the coordinates . . . 2

1.2 Typed operator words . . . 3

1.3 Seed rank . . . 4

1.4 Master seed: one cyclic shift . . . 5

2 Relations, quotients, and finite normal forms 8

2.1 The free presentation . . . 8

2.2 The shift relation . . . 8

2.3 Normal form and the first nontrivial object . . . 9

2.4 Symbolic certificate . . . 10

3 Addresses before projectors 11

3.1 Address maps . . . 11

3.2 Three nested address levels . . . 11

3.3 Root-of-unity addresses from the seed operator . . . 12

4 The finite realization theorem 14

4.1 Complete packet systems . . . 14

4.2 Transfer blocks . . . 14

4.3 Refinement . . . 15

4.4 Transform atlases . . . 15

4.5 The seven-realization protocol 16

5 The transfer algebra as a genuine calculus 18

5.1 The block representation is an algebra isomorphism 18

5.2 Support graphs and packet paths 19

5.3 The master transfer operator 19

5.4 Composition with a second operator 21

5.5 Adjoints, symmetries, and diagonalization 22

6 Refinement, coarsening, and multiple atlases 24

6.1 Coordinate refinement maps 24

6.2 The master refinement 25

6.3 Transfer blocks under refinement 25

6.4 Transform atlases and pulled-back projectors 25

6.5 A redundant exact multi-atlas frame 26

6.6 Completion direction 27

7 Nonlinear packets, obstruction packets, and completion 29

7.1 Exact nonlinear packet coordinates 29

7.2 Multilinear interaction tensors 29

7.3 Packet jets 30

7.4 The master nonlinear map 30

7.5 Obstruction packets 31

7.6 Completion from a dense core 32

8 One seed through the entire calculus 34

8.1 The complete step ledger 34

8.1.1 Step 0: choose the ambient type 34

8.1.2 Step 1: choose the seed 35

8.1.3 Step 2: choose the operator grammar 35

8.1.4 Step 3: compute the orbit 35

8.1.5 Step 4: impose the relation 35

8.1.6 Step 5: generate the object 35

8.1.7 Step 6: declare the packet addresses 35

8.1.8 Step 7: analyze and synthesize 36

8.1.9 Step 8: declare a transfer operator 36

8.1.10 Step 9: reconstruct the operator 36

8.1.11 Step 10: compose by paths 36

8.1.12 Step 11: refine . . . 36
8.1.13 Step 12: transport to Fourier space . . . 37
8.1.14 Step 13: retain several atlases simultaneously . . . 37
8.1.15 Step 14: apply a nonlinear operation . . . 37
8.1.16 Step 15: test an obstruction . . . 38
8.1.17 Step 16: close the reconstruction loop . . . 38
8.2 Complete symbolic implementation . . . 38

9 The start-to-finish realization theorem 40
9.1 Statement . . . 40
9.2 Why the construction deserves the name calculus . . . 41
9.3 Transition to the seven independent realizations . . . 42

II Seven Worked Areas 44

10 Area I: finite linear algebra and exact transfer blocks 45
10.1 Problem . . . 45
10.2 Step 1: declare the projectors . . . 45
10.3 Step 2: decompose and reconstruct the object . . . 45
10.4 Step 3: resolve the operator . . . 46
10.5 Step 4: multiply operators by packet paths . . . 48
10.6 Step 5: refine one packet . . . 49
10.7 Selected symbolic implementation . . . 50

11 Area II: polyphase, Fourier, and wavelet signal coordinates 51
11.1 Problem . . . 51
11.2 Step 1: the signal and residue projectors . . . 51
11.3 Step 2: exact polyphase analysis and synthesis . . . 51
11.4 Step 3: nested phase refinement . . . 53
11.5 Step 4: Haar analysis . . . 53
11.6 Step 5: Fourier analysis as a projector atlas . . . 54
11.7 Selected symbolic implementation . . . 57

12 Area III: a quantum circuit as packet paths 58
12.1 Problem . . . 58
12.2 Step 1: Hilbert space and parity packets . . . 58
12.3 Step 2: the circuit . . . 58

12.4 Step 3: packet path expansion . . . 59
12.5 Step 4: density matrix and partial trace . . . 60
12.6 Selected symbolic implementation . . . 61

13 Area IV: stochastic projectors and Bayesian refinement 62
13.1 Problem . . . 62
13.2 Step 1: a conditional-expectation projector . . . 62
13.3 Step 2: Bayesian conditioning . . . 64
13.4 Step 3: the sufficient case . . . 64
13.5 Step 4: the nonsufficient case . . . 65
13.6 Step 5: exact information-loss factorization . . . 66

14 Area V: a closed root-filter family and a candidate integer sequence 68
14.1 Problem . . . 68
14.2 Step 1: the signed order-five packet . . . 68
14.3 Step 2: reciprocal square and sequence definition . . . 70
14.4 Step 3: an exact integer recurrence . . . 70
14.5 Step 4: an independent finite coefficient formula . . . 72
14.6 Step 5: the reciprocal-order polynomial family . . . 72
14.7 OEIS-style candidate record . . . 73
14.8 Selected symbolic implementation . . . 74

15 Area VI: a coupled-oscillator physics problem 75
15.1 Problem . . . 75
15.2 Step 1: equations of motion . . . 75
15.3 Step 2: exact modal solution . . . 76
15.4 Step 3: verify the equation . . . 77
15.5 Step 4: energy packets . . . 78

16 Area VII: nonlinear packets and geometric holonomy 80
16.1 Part A: nonlinear signal packets . . . 80
16.2 Part B: a finite gauge-gluing problem . . . 81
16.3 Negative control . . . 83

III Cross-Domain Synthesis 85

17 What the seven examples prove 86
17.1 One theorem schema, seven domain meanings . . . 86

17.2 Seven-realization certificate . . . 86
17.3 Why this is stronger than a list of examples . . . 88
17.4 What remains outside this document . . . 88

IV Appendices **90**

A Exact result ledger **91**

B Candidate OEIS submission record **93**

C Reproduction commands **95**

D Full symbolic source listings **96**
D.1 Seed-to-reconstruction foundations . . . 96
D.2 Foundation figure generator . . . 99
D.3 Finite linear algebra . . . 108
D.4 Signals and transform atlases . . . 110
D.5 Quantum circuit . . . 112
D.6 Conditional expectation and Bayesian refinement . . . 113
D.7 Arithmetic root-filter sequence . . . 115
D.8 Coupled oscillators . . . 117
D.9 Nonlinear packets and holonomy . . . 118
D.10 Detailed intermediate certificates . . . 120
D.11 Seed-to-reconstruction foundation tests . . . 122
D.12 Implementation-certificate tests . . . 123
D.13 Independent re-derivation audit . . . 125
D.14 Detailed derivation audits . . . 127

E Layout and figure audit **129**

F Source and claim map **130**

G Conclusion **131**

Part I

The Common Finite Calculus

Chapter 1

The primitive seed and the typed operator grammar

1.1 Why the seed must come before the coordinates

The calculus begins before a projector is chosen. A projector tells us how to resolve an object that already exists; it does not explain how that object was generated, which operations are admissible, or which relations make two operator expressions represent the same value. The first layer therefore separates six pieces of data:

$$\boxed{\mathfrak{S} = (X, S, \mathcal{A}, \mathcal{R}, \mathcal{P}, \tau).}$$

Here X is the ambient type, S is the seed family, $\mathcal{A}$ is the fixed operator grammar, $\mathcal{R}$ is the relation system, $\mathcal{P}$ is a packet atlas, and τ is the declared completion structure. In a finite realization, τ is automatic; in an infinite realization it must be named because algebraic generation and topological completion are different operations.

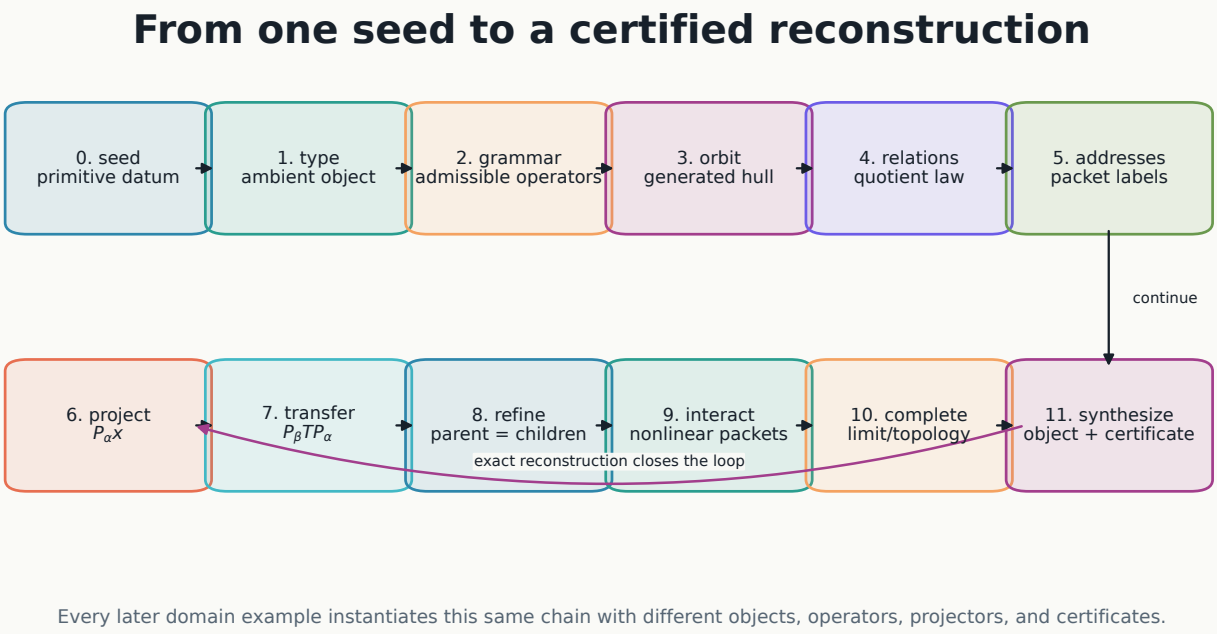


Figure 1.1: The complete dependency path. The later seven-area examples start at the left and do not skip directly to a convenient coordinate representation.

Boundary against hidden encoding

A datum is not a useful seed merely because a sufficiently complicated operator can decode the target from it. The grammar must be fixed independently of the target, the relations must be explicit, and the generated hull must be measurable. Otherwise the phrase "one seed generates the object" has no mathematical content.

1.2 Typed operator words

A typed grammar may be represented by a quiver. Vertices are object types and arrows are admissible primitive operations. A composable path

$$\gamma = a_m \cdots a_2 a_1$$

is an operator word. A realization assigns a vector space or module X_v to each vertex and a linear map to each arrow. Evaluation sends a path to the corresponding composition.

For the one-space finite core used below, all arrows act on one space X, so the grammar generates a unital algebra

$$\mathcal{A} \subseteq \mathrm{End}_K(X).$$

The point of the typed formulation is that the same definitions remain meaningful when different arrows have different source and target spaces.

Definition 1.1 (Generated hull)**.** For a seed family $S \subseteq X$, define

$$\mathrm{Hull}_{\mathcal{A}}(S) = \mathrm{span}_K\{As : A \in \mathcal{A},\ s \in S\}.$$

When $\mathcal{A}$ is given by primitive generators, A ranges over all finite admissible operator words and their finite linear combinations.

Theorem 1.2 (Smallest invariant subspace)**.** $\mathrm{Hull}_{\mathcal{A}}(S)$ *is the smallest* $\mathcal{A}$*-invariant linear subspace of* X *containing* S*.*

Proof. The hull contains S because the identity belongs to the unital algebra. If $B \in \mathcal{A}$ and $x = \sum_j c_j A_j s_j$ lies in the hull, then

$$Bx = \sum_j c_j (BA_j) s_j \in \mathrm{Hull}_{\mathcal{A}}(S),$$

so the hull is invariant. Conversely, any invariant subspace containing S contains every As and therefore their span. □

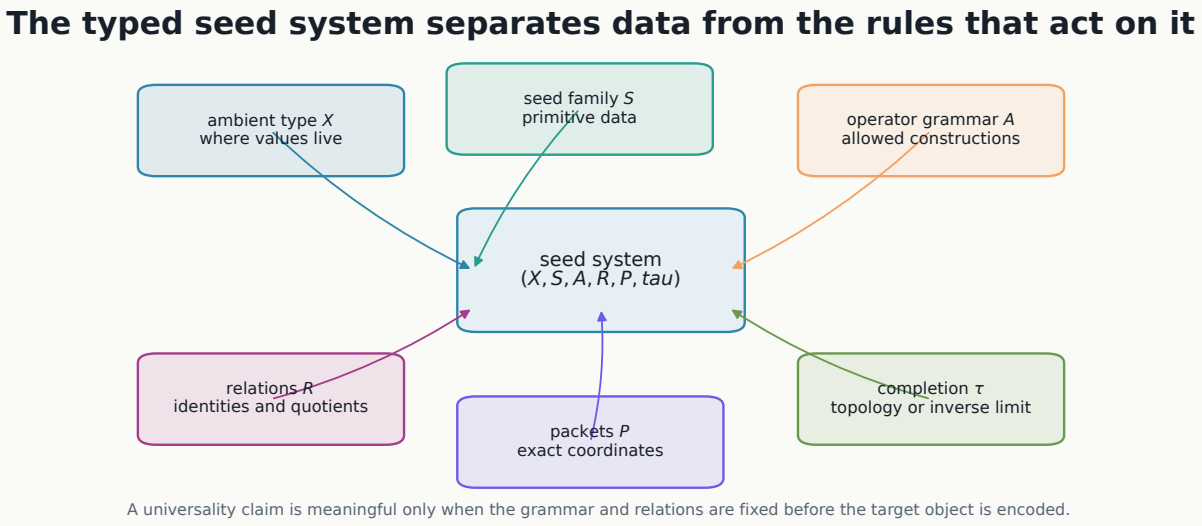


Figure 1.2: The seed system keeps ambient type, primitive data, grammar, relations, packet coordinates, and completion data separate.

1.3 Seed rank

Definition 1.3 (Seed rank)**.** The seed rank of a finite $\mathcal{A}$-module X is

$$\mathrm{srank}_{\mathcal{A}}(X) = \min\{|S| : \mathrm{Hull}_{\mathcal{A}}(S) = X\}.$$

A one-seed realization is called cyclic.

For a single operator $U \in \mathrm{End}(X)$ and one proposed seed s, define the orbit matrix

$$\mathcal{O}_U(s) = \begin{bmatrix} s & Us & U^2 s & \cdots & U^{n-1} s \end{bmatrix}, \qquad n = \dim X.$$

Theorem 1.4 (Finite cyclicity criterion)**.** *The vector* s *generates* X *under the polynomial algebra* $K[U]$ *if and only if*

$$\mathrm{rank}\, \mathcal{O}_U(s) = n.$$

Proof. The columns of the orbit matrix span the Krylov space generated by s. Cayley–Hamilton reduces every power U^m, $m \geq n$, to a linear combination of $I, U, \ldots, U^{n-1}$. Thus the full generated hull equals the column space of $\mathcal{O}_U(s)$, and this hull is X precisely when the matrix has rank n. □

1.4 Master seed: one cyclic shift

We now introduce the single example that will be carried through every layer of the calculus. Let

$$X = \mathbb{C}^8, \qquad s = e_0 = (1, 0, 0, 0, 0, 0, 0, 0)^{\mathsf{T}},$$

and let U be the forward cyclic shift

$$Ue_j = e_{j+1 \pmod 8}.$$

Its matrix has ones immediately below the diagonal and one in the upper-right corner.

Detailed calculation orbit

The first eight operator words give

$$s = e_0, \quad Us = e_1, \quad U^2 s = e_2, \quad \ldots, \quad U^7 s = e_7.$$

Therefore

$$\mathcal{O}_U(s) = I_8$$

up to the chosen ordering of the standard basis, and

$$\boxed{\operatorname{rank} \mathcal{O}_U(s) = 8.}$$

The seed is cyclic and $\operatorname{Hull}_{\mathbb{C}[U]}(s) = X$.

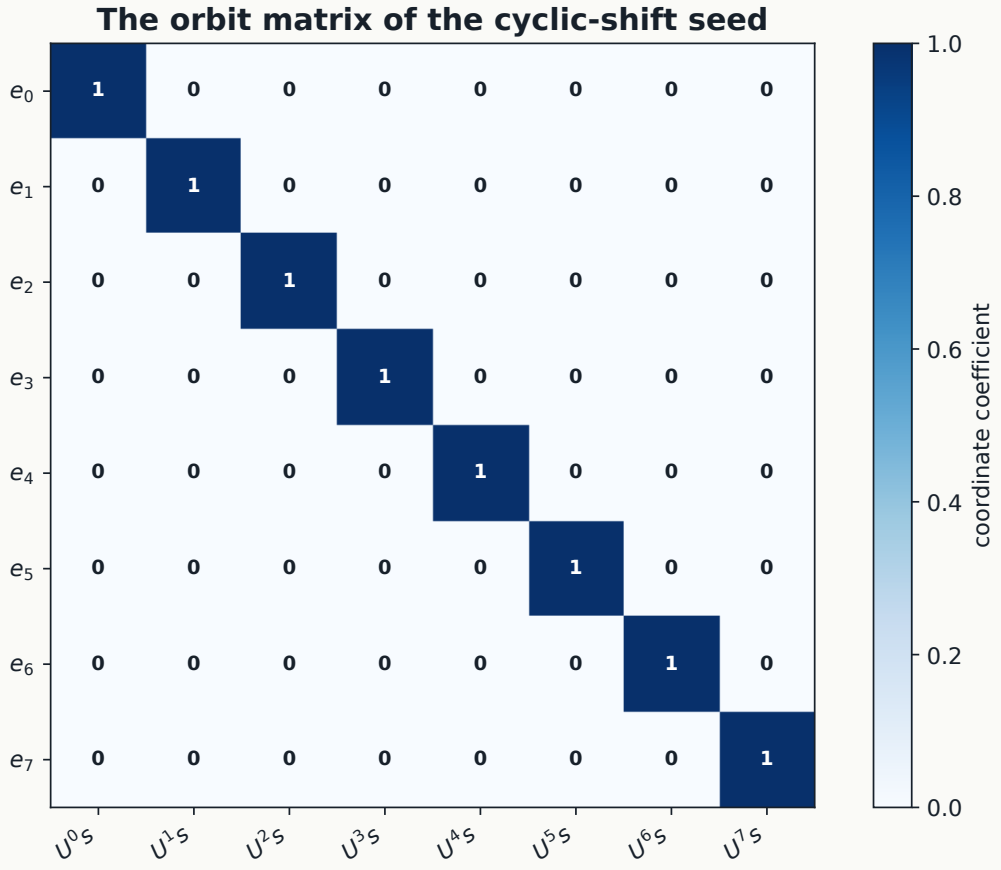


Figure 1.3: The orbit columns are the eight coordinate vectors. The rank calculation is exact and does not depend on a numerical tolerance.

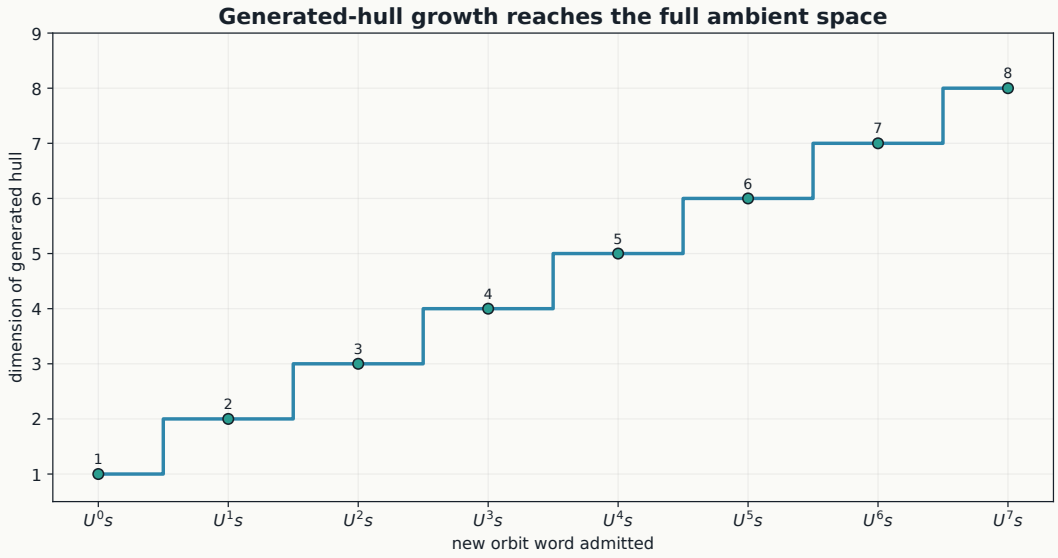


Figure 1.4: The dimension of the partial hull after admitting $s, Us, \ldots, U^k s$. At depth seven the full eight-dimensional space has been generated.

Seed-to-finish Step 1: primitive datum

Start with $s = e_0$. No signal coefficients, Fourier bins, wavelet coefficients, or packet boundaries have yet been inserted.

Seed-to-finish Step 2: fixed grammar

Declare one primitive operator U and permit finite polynomial expressions in U. The target vector introduced later must be generated from this already-declared grammar.

Seed-to-finish Step 3: generated hull

The orbit rank proves that the declared seed and grammar generate the entire ambient space. This is the first exact certificate in the chain.

Chapter 2

Relations, quotients, and finite normal forms

2.1 The free presentation

Generation alone produces many operator words that may evaluate to the same element. Relations are the kernel data that turn the free word space into the actual realized object.

Let $\mathcal{A}^{(S)}$ denote the free left $\mathcal{A}$-module on formal symbols $[s]$, $s \in S$. Define the evaluation map

$$\varepsilon : \mathcal{A}^{(S)} \longrightarrow X, \qquad \varepsilon\left(\sum_{s\in S} A_s[s]\right) = \sum_{s\in S} A_s s.$$

Theorem 2.1 (Seed presentation theorem)**.** *The evaluation map has image* $\mathrm{Hull}_{\mathcal{A}}(S)$ *and induces an isomorphism*

$$\boxed{\mathcal{A}^{(S)}/\operatorname{Ker}\varepsilon \cong \mathrm{Hull}_{\mathcal{A}}(S).}$$

Proof. The image is exactly the span of the values As, so it is the generated hull. The quotient isomorphism is the first isomorphism theorem. □

2.2 The shift relation

For the master example,

$$U^8 = I.$$

Consequently the polynomial evaluation map

$$\varepsilon_s : \mathbb{C}[z] \longrightarrow X, \qquad p(z) \longmapsto p(U)s$$

has $(z^8 - 1)$ in its kernel.

Proposition 2.2 (Exact cyclic quotient)**.** *For the cyclic shift seed,*

$$\boxed{X \cong \mathbb{C}[z]/(z^8 - 1)}$$

as a $\mathbb{C}[z]$*-module, where multiplication by* z *corresponds to* U*.*

Proof. The map is surjective by cyclicity. Both the quotient and X have dimension eight. Since $(z^8 - 1)$ lies in the kernel, the induced surjection from the eight-dimensional quotient to X is an isomorphism. □

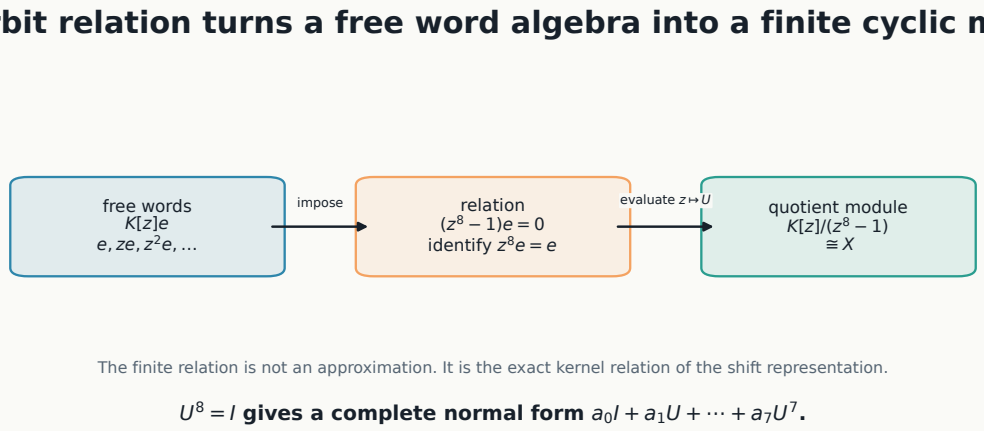


Figure 2.1: The exact relation $U^8 = I$ converts infinitely many formal words into eight normal-form coordinates.

2.3 Normal form and the first nontrivial object

Every generated object has a unique representative

$$p(U)s = \sum_{k=0}^{7} a_k U^k s.$$

Choose

$$p(z) = 2 - z + 3z^2 + z^4 + 4z^5 - 2z^6 + 5z^7.$$

Then

$$\boxed{x = p(U)s = (2, -1, 3, 0, 1, 4, -2, 5)^{\mathsf{T}}.}$$

The eight entries have not been hidden in an arbitrary decoder: they are the coefficients of the unique normal form in the fixed quotient algebra.

Detailed calculation coefficient extraction

Because $U^k s = e_k$,

$$\begin{aligned} p(U)s &= 2e_0 - e_1 + 3e_2 + 0e_3 + e_4 + 4e_5 - 2e_6 + 5e_7 \\ &= (2, -1, 3, 0, 1, 4, -2, 5)^{\mathsf{T}}. \end{aligned}$$

Seed-to-finish Step 4: relation system

Record $U^8 = I$. This is the precise rule that allows arbitrarily long operator words to be reduced to a finite normal form.

Seed-to-finish Step 5: generated object

Choose a polynomial p after the grammar and relation are fixed, and form $x = p(U)s$. The object now exists before any packet atlas is chosen.

2.4 Symbolic certificate

The exact implementation is short because the mathematical normal form has already been proved.

```
import sympy as sp
from proof_atlas.foundations import cyclic_shift

U = cyclic_shift(8)
s = sp.eye(8)[:, 0]
coeff = [2, -1, 3, 0, 1, 4, -2, 5]
x = sum((coeff[k] * U**k * s for k in range(8)), sp.zeros(8, 1))

assert U**8 == sp.eye(8)
assert sp.Matrix.hstack(*[U**k * s for k in range(8)]).rank() == 8
assert x == sp.Matrix(coeff)
```

Chapter 3

Addresses before projectors

3.1 Address maps

An address system is a declared rule assigning each elementary coordinate to a packet label. Let $\{e_j\}_{j\in J}$ be a basis and let

$$a : J \longrightarrow A$$

be an address map. Define

$$P_\alpha e_j = \begin{cases} e_j, & a(j) = \alpha, \\ 0, & a(j) \neq \alpha. \end{cases}$$

Theorem 3.1 (Address-projector theorem)**.** *The operators constructed from an address map satisfy*

$$P_\alpha P_\beta = \delta_{\alpha\beta} P_\alpha, \qquad \sum_{\alpha\in A} P_\alpha = I.$$

Conversely, every complete family of pairwise annihilating projectors gives the direct-sum decomposition

$$X = \bigoplus_{\alpha\in A} P_\alpha X.$$

Proof. Each basis vector belongs to exactly one address class, so applying two distinct selectors kills it and summing all selectors returns it. The identities follow on a basis and hence on all of X. Conversely, completeness gives $x = \sum_\alpha P_\alpha x$, and pairwise annihilation implies uniqueness of this decomposition. □

3.2 Three nested address levels

For the master vector, use three levels:

$$\begin{aligned} \text{level 1} &: j \pmod 2, \\ \text{level 2} &: j \pmod 4, \\ \text{level 3} &: j \pmod 8. \end{aligned}$$

Let E_j be the individual coordinate projector. Let

$$R_r = E_r + E_{r+4}, \qquad 0 \le r < 4,$$

and

$$P_{\mathrm{e}} = E_0 + E_2 + E_4 + E_6, \qquad P_{\mathrm{o}} = E_1 + E_3 + E_5 + E_7.$$

Then

$$\boxed{P_{\mathrm{e}} = R_0 + R_2, \qquad P_{\mathrm{o}} = R_1 + R_3, \qquad R_r = E_r + E_{r+4}.}$$

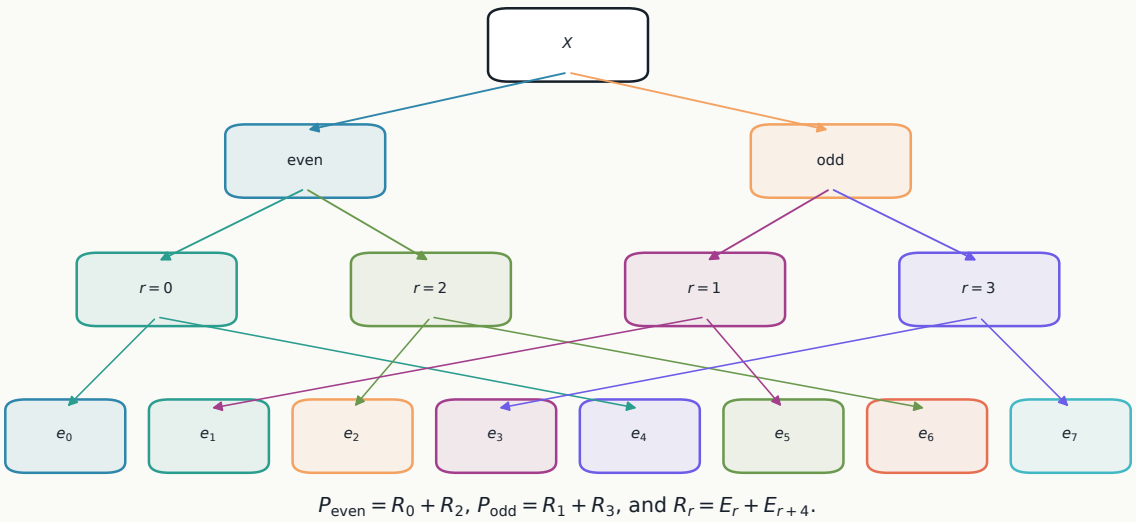


Figure 3.1: The address tree is declared before the numerical packets are formed. Every parent is exactly the sum of its descendants.

3.3 Root-of-unity addresses from the seed operator

The same cyclic operator that generated the object also generates its Fourier packet projectors. Let $\omega = e^{2\pi i/N}$ and suppose $U^N = I$. Define

$$Q_k = \frac{1}{N} \sum_{m=0}^{N-1} \omega^{-km} U^m, \qquad 0 \le k < N.$$

Theorem 3.2 (Cyclic character projectors)**.** *The operators Q_k satisfy*

$$Q_k Q_\ell = \delta_{k\ell} Q_k, \qquad \sum_{k=0}^{N-1} Q_k = I, \qquad U Q_k = \omega^k Q_k$$

with the sign convention determined by the chosen Fourier transform.

Proof. Multiply the two sums and collect the coefficient of U^r. The inner character sum is

$$\sum_{m=0}^{N-1} \omega^{(\ell-k)m} = N\delta_{k\ell}.$$

This proves idempotence and annihilation. Summing Q_k over k leaves only the $m = 0$ term, giving I. Multiplication by U shifts the summation index and produces the character factor. □

Remark 3.3. The spatial packets E_j and Fourier packets Q_k are not competing truths. They are two exact atlases of the same generated object. Their usefulness is determined by the operator or question being studied.

Seed-to-finish Step 6: addresses

Declare parity, residue-mod-four, coordinate, and Fourier-character addresses. Different atlases may coexist because each has its own exact synthesis law.

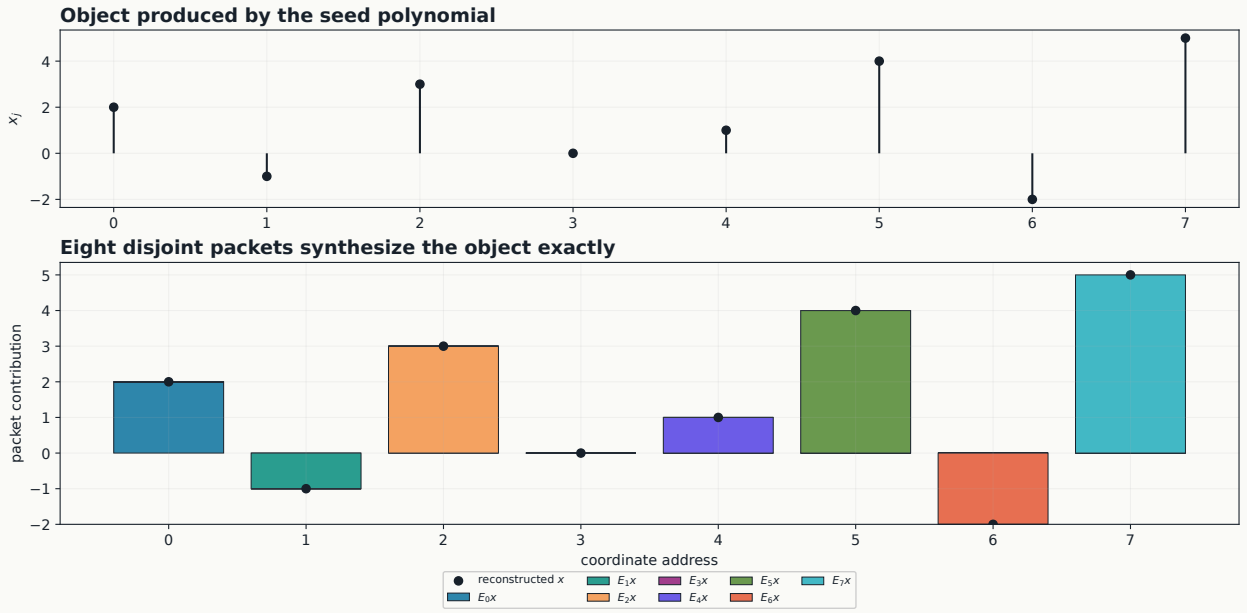


Figure 3.2: The master vector resolved into its eight coordinate packets and reassembled. Negative packet values remain exact coordinates; no positivity assumption is needed.

Chapter 4

The finite realization theorem

4.1 Complete packet systems

Definition 4.1 (Complete packet system)**.** Let X be a finite-dimensional vector space over a field K. A finite family $\{P_\alpha\}_{\alpha\in A} \subseteq \mathrm{End}(X)$ is a complete packet system when

$$P_\alpha P_\beta = \delta_{\alpha\beta} P_\alpha, \qquad \sum_{\alpha\in A} P_\alpha = I.$$

The packet space is $X_\alpha = P_\alpha X$.

The pairwise annihilation condition says that one coordinate is not counted twice. Completeness says that no coordinate is omitted.

Definition 4.2 (Analysis and synthesis)**.** Define

$$\mathcal{A}_\mathcal{P} x = (P_\alpha x)_\alpha, \qquad \mathcal{S}_\mathcal{P}(x_\alpha)_\alpha = \sum_\alpha x_\alpha.$$

Theorem 4.3 (Exact object reconstruction)**.** *For every complete packet system,*

$$\boxed{\mathcal{S}_\mathcal{P}\mathcal{A}_\mathcal{P} = I_X.}$$

Equivalently,

$$\boxed{x = \sum_\alpha P_\alpha x.}$$

Proof. By completeness,

$$\mathcal{S}_\mathcal{P}\mathcal{A}_\mathcal{P} x = \sum_\alpha P_\alpha x = \left(\sum_\alpha P_\alpha\right) x = x.$$

□

4.2 Transfer blocks

Definition 4.4 (Transfer block)**.** For $T \in \mathrm{End}(X)$, define

$$T_{\beta\alpha} = P_\beta T P_\alpha.$$

It is the exact part of T carrying input packet X_α into output packet X_β.

Theorem 4.5 (Exact operator reconstruction)**.** *Every operator satisfies*

$$\boxed{T = \sum_{\alpha,\beta} P_\beta T P_\alpha.}$$

Proof. Insert completeness on both sides:

$$T = ITI = \left(\sum_\beta P_\beta\right) T \left(\sum_\alpha P_\alpha\right) = \sum_{\alpha,\beta} P_\beta T P_\alpha.$$

□

Theorem 4.6 (Block multiplication law)**.** *For $S, T \in \mathrm{End}(X)$,*

$$\boxed{(ST)_{\gamma\alpha} = \sum_\beta S_{\gamma\beta} T_{\beta\alpha}.}$$

Proof. Insert the identity between S and T:

$$P_\gamma S T P_\alpha = P_\gamma S \left(\sum_\beta P_\beta\right) T P_\alpha = \sum_\beta (P_\gamma S P_\beta)(P_\beta T P_\alpha).$$

□

4.3 Refinement

Definition 4.7 (Exact refinement)**.** A fine packet family $\{Q_\rho\}_{\rho\in R}$ refines $\{P_\alpha\}_{\alpha\in A}$ when the fine packets are complete and every coarse packet is the sum of its declared descendants:

$$P_\alpha = \sum_{\rho \succ \alpha} Q_\rho.$$

Proposition 4.8 (Coarse blocks reconstruct from fine blocks)**.** *If Q refines P, then*

$$\boxed{P_\beta T P_\alpha = \sum_{\sigma\succ\beta} \sum_{\rho\succ\alpha} Q_\sigma T Q_\rho.}$$

Proof. Substitute the descendant sums on the left and right of T and distribute. □

4.4 Transform atlases

Theorem 4.9 (Pulled-back coordinate projectors)**.** *Let $U : X \to Y$ be invertible, and let $\{E_\alpha\}$ be ordinary coordinate projectors on Y. Define*

$$P_\alpha = U^{-1} E_\alpha U.$$

Then $\{P_\alpha\}$ is a complete packet system on X.

Proof. Conjugation preserves multiplication and finite sums:

$$P_\alpha P_\beta = U^{-1} E_\alpha E_\beta U = \delta_{\alpha\beta} P_\alpha,$$

and

$$\sum_\alpha P_\alpha = U^{-1} \left(\sum_\alpha E_\alpha \right) U = I.$$

□

Fourier coordinates, wavelet coordinates, normal modes, quantum basis changes, and diagonalizing transforms are all instances of this theorem.

4.5 The seven-realization protocol

Every worked chapter follows the same proof protocol.

Step 1. Declare the ambient object and the packet family.

Step 2. Verify idempotence, annihilation, and completeness.

Step 3. Write the domain operator or nonlinear map in packet coordinates.

Step 4. Solve the domain problem at packet level.

Step 5. Synthesize the full result and compute an exact residual.

Step 6. Recompute the identity in symbolic Python independently of the prose derivation.

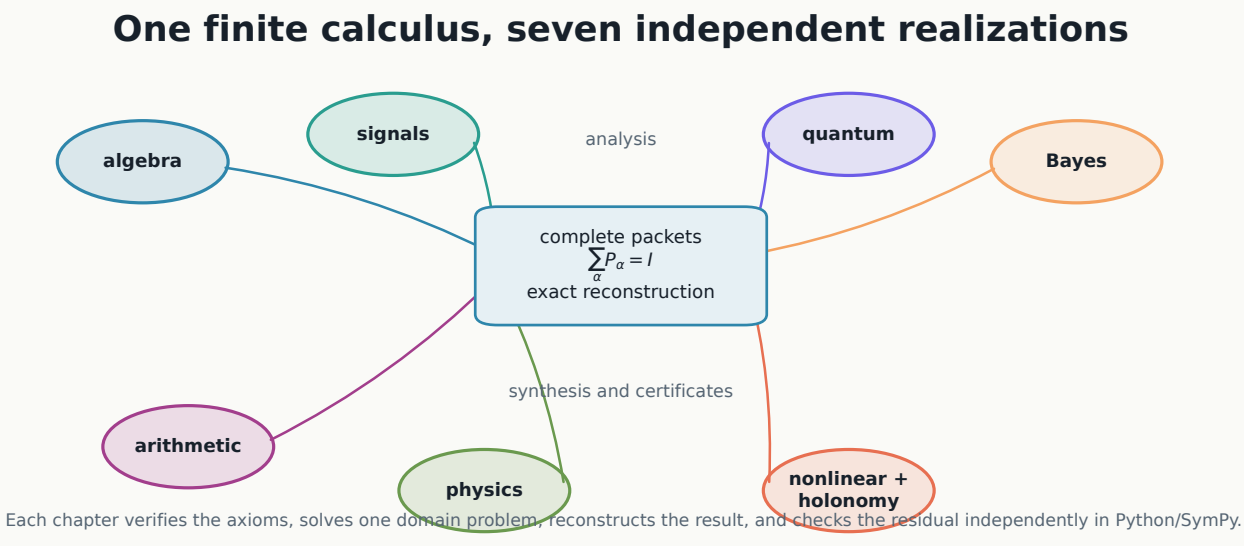


Figure 4.1: The common finite calculus and its seven realizations.

Table 4.1: The seven worked areas and their principal exact certificates.

Area	Domain problem	Exact certificate
Linear algebra	Reconstruct a vector, an operator, and an operator product from blocks	Four zero matrix residuals
Signals	Polyphase, Haar, and Fourier representations of one signal	Exact synthesis and projector completeness
Quantum	Bell-state preparation and parity path decomposition	Zero path residual and reduced state $I/2$
Probability	Conditional expectation and Bayesian coarsening	Idempotence, sufficiency, and exact defect factorization
Arithmetic	Reciprocal root-filter sequence and closed analytic formula	Integer recurrence, hypergeometric identity, composition formula
Physics	Two coupled oscillators	Exact ODE solution and conserved packet energies
Nonlinear geometry	Nonlinear filter and cycle holonomy	Zero nonlinear residual; designed obstruction norm $8/5$

Chapter 5

The transfer algebra as a genuine calculus

5.1 The block representation is an algebra isomorphism

The finite realization theorem reconstructs each operator from its transfer blocks. The stronger statement is that the complete family of blocks is an exact coordinate representation of the entire operator algebra.

Let

$$\mathfrak{B}_{\mathcal{P}} = \bigoplus_{\alpha,\beta} \mathrm{Hom}(X_\alpha, X_\beta).$$

Define

$$\Phi_{\mathcal{P}} : \mathrm{End}(X) \longrightarrow \mathfrak{B}_{\mathcal{P}}, \qquad \Phi_{\mathcal{P}}(T) = (P_\beta T P_\alpha)_{\beta,\alpha}.$$

Equip $\mathfrak{B}_{\mathcal{P}}$ with matrix-style multiplication

$$(B \star C)_{\gamma\alpha} = \sum_{\beta} B_{\gamma\beta} C_{\beta\alpha}.$$

Theorem 5.1 (Full block-algebra representation)**.** *$\Phi_{\mathcal{P}}$ is a unital algebra isomorphism. Its inverse is*

$$\boxed{\Phi_{\mathcal{P}}^{-1}((B_{\beta\alpha})) = \sum_{\alpha,\beta} B_{\beta\alpha}.}$$

Proof. Exact operator reconstruction proves that the displayed inverse recovers T. Conversely, if each $B_{\beta\alpha}$ maps X_α into X_β, then

$$P_\delta \left(\sum_{\alpha,\beta} B_{\beta\alpha} \right) P_\gamma = B_{\delta\gamma}$$

because all other corners are annihilated. The block multiplication theorem gives

$$\Phi_{\mathcal{P}}(ST) = \Phi_{\mathcal{P}}(S) \star \Phi_{\mathcal{P}}(T).$$

The identity operator corresponds to the diagonal block identity. □

5.2 Support graphs and packet paths

Define the support relation

$$\alpha \longrightarrow \beta \quad \Longleftrightarrow \quad P_\beta T P_\alpha \neq 0.$$

This produces a directed transfer graph. The graph records possible movement, while the blocks retain the exact linear maps and weights.

Theorem 5.2 (Path expansion for powers)**.** *For every $m \geq 1$,*

$$\boxed{P_{\alpha_m} T^m P_{\alpha_0} = \sum_{\alpha_1,\ldots,\alpha_{m-1}} (P_{\alpha_m} T P_{\alpha_{m-1}}) \cdots (P_{\alpha_1} T P_{\alpha_0}).}$$

Only directed packet paths contribute nonzero terms.

Proof. Insert $I = \sum_\alpha P_\alpha$ between every adjacent pair of copies of T and distribute. Pairwise annihilation removes every incompatible path. □

5.3 The master transfer operator

Return to the cyclic seed example and define

$$T = 2I - U + 3U^2.$$

In the coordinate basis,

$$T = \begin{pmatrix} 2 & 0 & 0 & 0 & 0 & 0 & 3 & -1 \\ -1 & 2 & 0 & 0 & 0 & 0 & 0 & 3 \\ 3 & -1 & 2 & 0 & 0 & 0 & 0 & 0 \\ 0 & 3 & -1 & 2 & 0 & 0 & 0 & 0 \\ 0 & 0 & 3 & -1 & 2 & 0 & 0 & 0 \\ 0 & 0 & 0 & 3 & -1 & 2 & 0 & 0 \\ 0 & 0 & 0 & 0 & 3 & -1 & 2 & 0 \\ 0 & 0 & 0 & 0 & 0 & 3 & -1 & 2 \end{pmatrix}.$$

Applying it to

$$x = (2, -1, 3, 0, 1, 4, -2, 5)^{\mathsf{T}}$$

gives

$$\boxed{Tx = (-7, 11, 13, -6, 11, 7, -5, 24)^{\mathsf{T}}.}$$

Detailed calculation coordinate rule

Because $(Ux)_j = x_{j-1}$ and $(U^2x)_j = x_{j-2}$ with indices modulo eight,

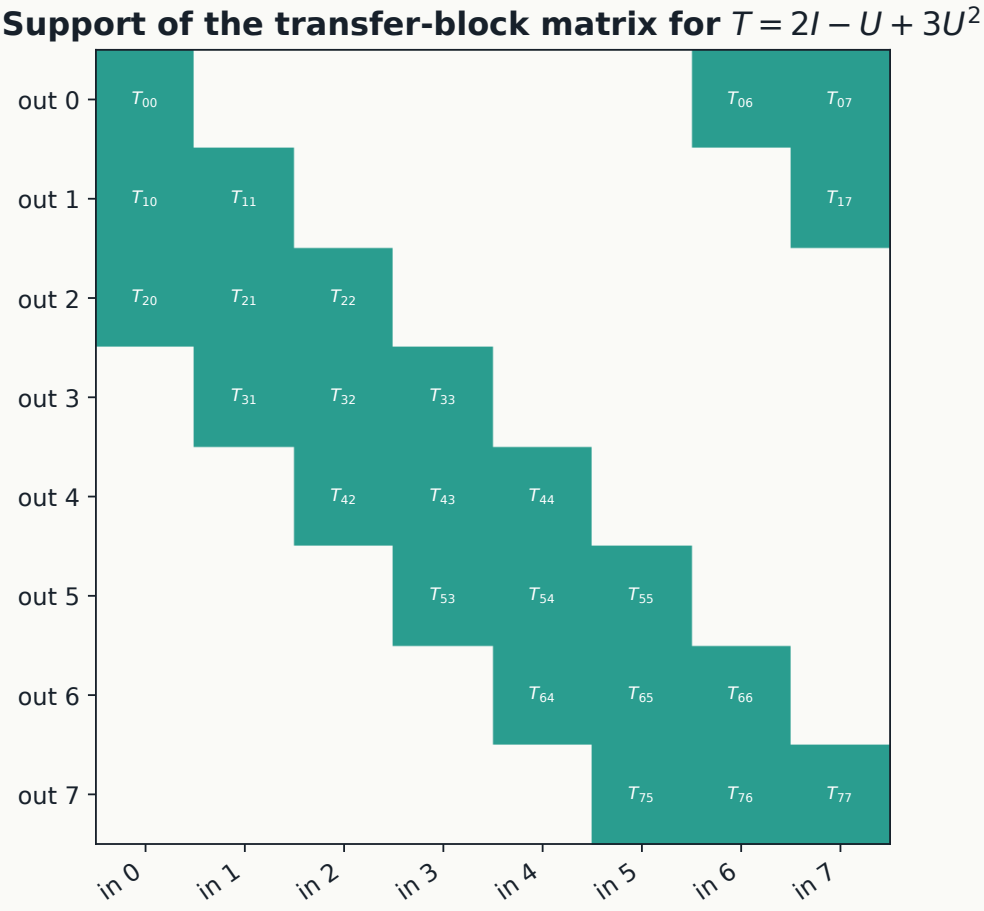
$$(Tx)_j = 2x_j - x_{j-1} + 3x_{j-2}.$$

For example,

$$(Tx)_0 = 2(2) - x_7 + 3x_6 = 4 - 5 - 6 = -7,$$

and

$$(Tx)_7 = 2(5) - x_6 + 3x_5 = 10 + 2 + 12 = 24.$$



Only the identity, one-step, and two-step cyclic transfer bands are nonzero.

Figure 5.1: The exact support pattern of the transfer blocks. A nonzero cell records a genuine packet-to-packet map, not merely a correlation.

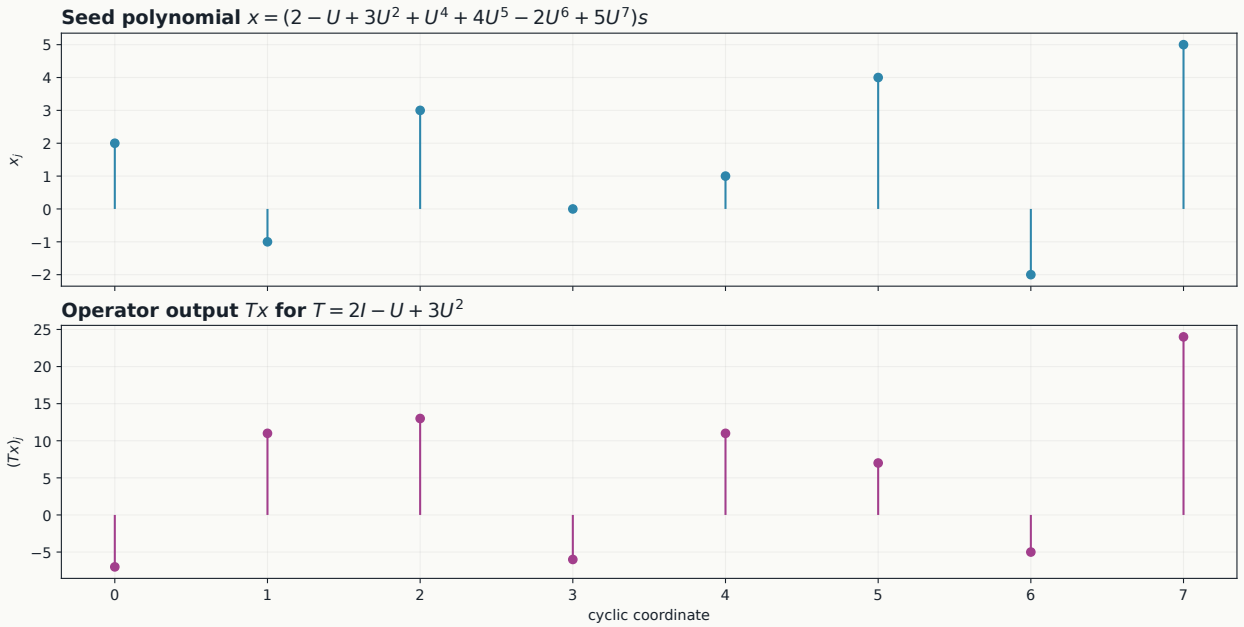


Figure 5.2: The generated object and the exact output of the declared transfer operator.

5.4 Composition with a second operator

Let

$$S = I + U^3.$$

The product ST may be calculated directly, but the calculus resolves it through every intermediate address:

$$P_b S T P_a = \sum_{\gamma=0}^{7} (P_b S P_\gamma)(P_\gamma T P_a).$$

Summing over a and b reconstructs ST exactly.

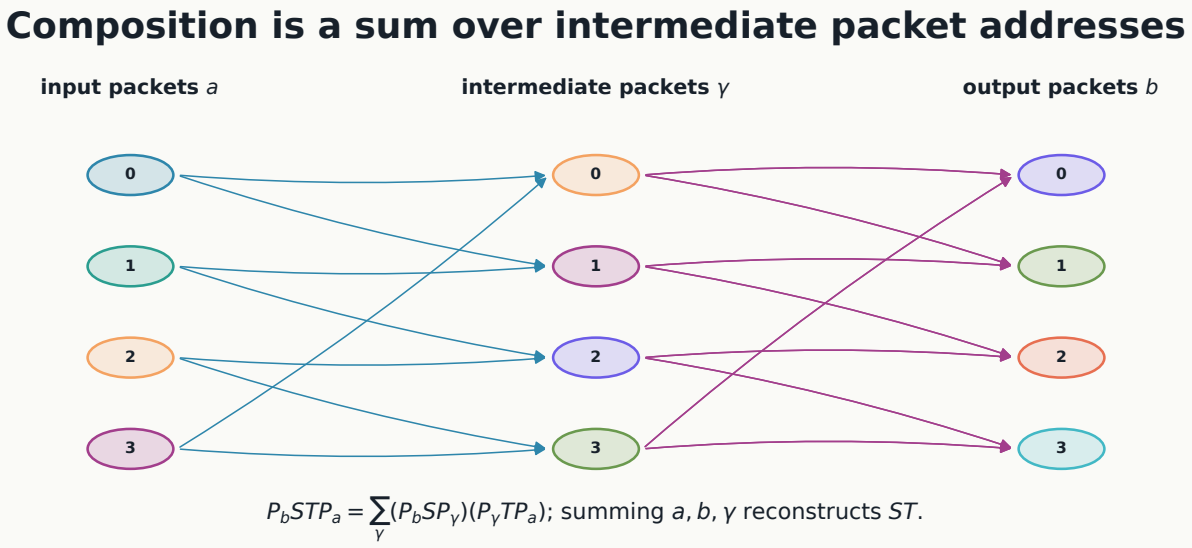


Figure 5.3: A reduced visualization of the packet-path law. The proof sums all intermediate addresses, including paths that vanish.

Exact certificate master product

The symbolic matrix

$$\sum_{a,b,\gamma} E_b S E_\gamma T E_a - ST$$

is exactly the zero 8×8 matrix.

5.5 Adjoints, symmetries, and diagonalization

When the packets are orthogonal,

$$(P_\beta T P_\alpha)^* = P_\alpha T^* P_\beta.$$

Thus adjunction reverses every transfer edge. If T commutes with every packet projector, all off-diagonal blocks vanish and the packets reduce T.

For the cyclic operator $T = 2I - U + 3U^2$, the Fourier transform diagonalizes U and hence diagonalizes T. If $FUF^* = \mathrm{diag}(\zeta_0, \ldots, \zeta_7)$, then

$$FTF^* = \mathrm{diag}(2 - \zeta_k + 3\zeta_k^2)_{k=0}^7.$$

The operator is banded in spatial coordinates and diagonal in Fourier coordinates.

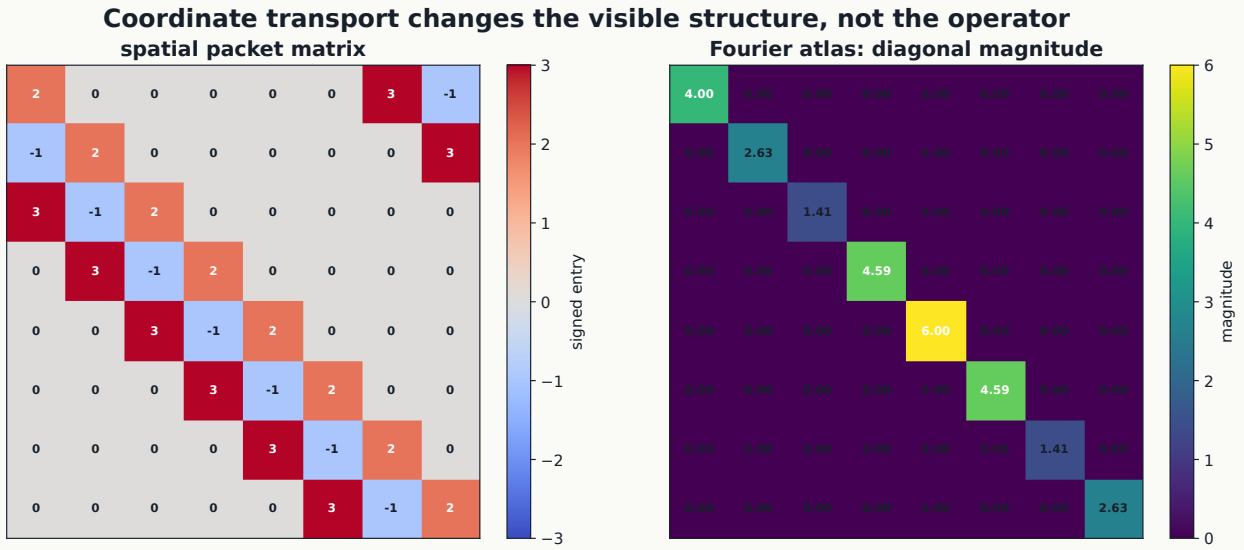


Figure 5.4: The same operator in the spatial and Fourier atlases. Coordinate transport changes the matrix pattern but not the underlying map.

Seed-to-finish Step 7: transfer blocks

Insert complete projectors on both sides of the operator. The resulting block field is an exact operator coordinate system.

Seed-to-finish Step 8: packet paths

Insert complete projectors between composed operators. Products, powers, circuits, and time steps become exact sums over packet paths.

Chapter 6

Refinement, coarsening, and multiple atlases

6.1 Coordinate refinement maps

Let $\mathcal{P} = \{P_\alpha\}$ be coarse and $\mathcal{Q} = \{Q_\rho\}$ be fine, with

$$P_\alpha = \sum_{\rho \succ \alpha} Q_\rho.$$

Both packet tuples are exact coordinate copies of X. Define

$$J = \mathcal{A}_\mathcal{Q}\mathcal{S}_\mathcal{P}, \qquad C = \mathcal{A}_\mathcal{P}\mathcal{S}_\mathcal{Q}.$$

Theorem 6.1 (Exact coordinate refinement)**.** *On the direct sums of the actual packet ranges,*

$$\boxed{CJ = I_{\oplus X_\alpha}, \qquad JC = I_{\oplus Y_\rho}.}$$

Thus refinement and coarsening are inverse coordinate changes when all children are retained.

Proof. Use exact analysis and synthesis at each level:

$$CJ = \mathcal{A}_\mathcal{P}\mathcal{S}_\mathcal{Q}\mathcal{A}_\mathcal{Q}\mathcal{S}_\mathcal{P} = \mathcal{A}_\mathcal{P} I_X \mathcal{S}_\mathcal{P} = I.$$

The proof of $JC = I$ is identical. □

Remark 6.2. Refinement is not truncation. Loss begins only when a child packet is discarded, merged through a noninvertible statistic, or replaced by a model.

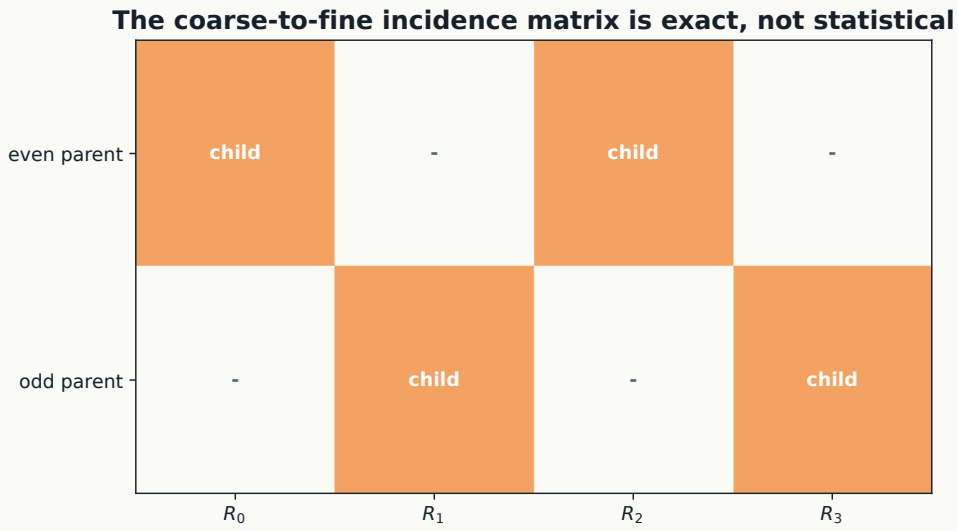


Figure 6.1: The exact parent-child incidence for the parity-to-mod-four refinement.

6.2 The master refinement

The parity packets refine into mod-four packets, which refine into coordinate packets:

$$P_{\mathrm{e}} = R_0 + R_2, \qquad P_{\mathrm{o}} = R_1 + R_3,$$
$$R_r = E_r + E_{r+4}.$$

Therefore

$$P_{\mathrm{e}}x = E_0x + E_2x + E_4x + E_6x,$$

and similarly for the odd packet. Every equality is exact over $\mathbb{C}^8$.

6.3 Transfer blocks under refinement

Refining both the source and target packets gives

$$P_\beta T P_\alpha = \sum_{\sigma \succ \beta} \sum_{\rho \succ \alpha} Q_\sigma T Q_\rho.$$

The coarse block is not replaced by a different operator. It is exactly reconstructed from the fine child blocks.

For a weighted filtration, assign a nonnegative weight $w(\beta, \alpha)$ to each transfer edge and define the weight of a path as the sum of its edge weights. Truncation by total weight then has an explicit omitted-path residual. The exact block and path formulas remain prior to any truncation.

6.4 Transform atlases and pulled-back projectors

Let $U : X \to Y$ be invertible and let E_α be the ordinary coordinate projectors on Y. The pulled-back projectors

$$P_\alpha = U^{-1} E_\alpha U$$

form a complete atlas on X. The signal chapter later takes U to be Fourier and Haar transforms; the physics chapter takes U to be the normal-mode transform; the quantum chapter takes U to be a basis-changing unitary.

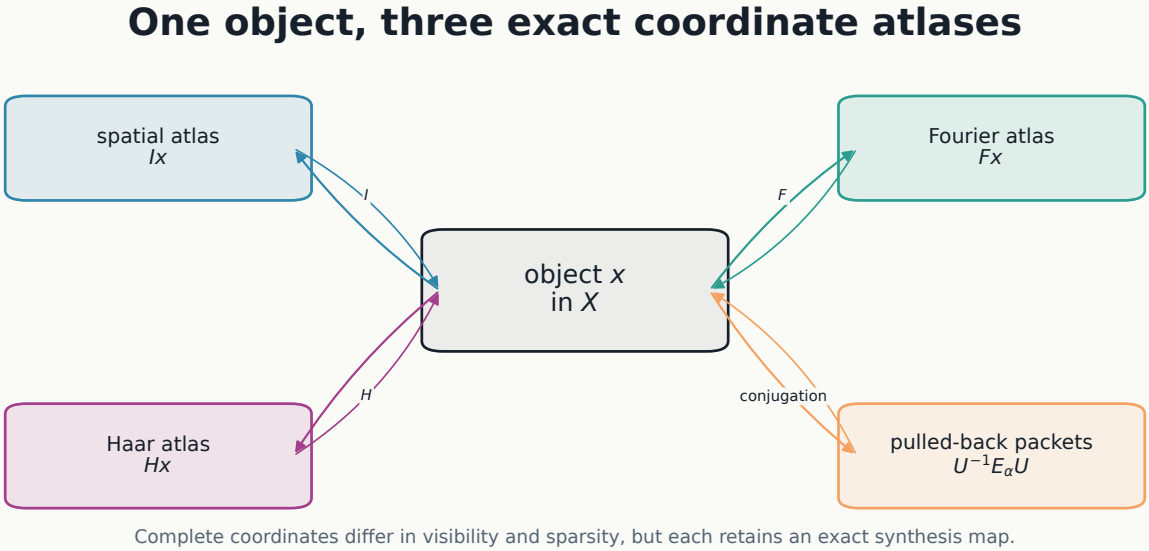


Figure 6.2: Spatial, Fourier, Haar, and general conjugated packet systems are exact atlases of one object.

6.5 A redundant exact multi-atlas frame

Several complete unitary atlases may be retained simultaneously. Let $U_1, \dots, U_m$ be unitary and let $w_a > 0$. Define

$$\mathcal{A}x = (w_a U_a x)_{a=1}^m.$$

Theorem 6.3 (Weighted multi-atlas reconstruction)**.** *The canonical synthesis*

$$\mathcal{S}((c_a)_a) = \frac{1}{W}\sum_{a=1}^m w_a U_a^* c_a, \qquad W = \sum_{a=1}^m w_a^2,$$

satisfies

$$\boxed{\mathcal{S}\mathcal{A} = I_X.}$$

Moreover,

$$\sum_{a=1}^m \|w_a U_a x\|^2 = W\|x\|^2.$$

Proof. Unitary inversion gives

$$\mathcal{S}\mathcal{A}x = \frac{1}{W}\sum_a w_a^2 U_a^* U_a x = \frac{1}{W}\sum_a w_a^2 x = x.$$

The energy identity follows from $\|U_a x\| = \|x\|$. □

For the master vector, choose

$$U_1 = I, \qquad U_2 = F, \qquad U_3 = H, \qquad (w_1, w_2, w_3) = (1, 2, 3).$$

Then $W = 14$ and

$$\widehat{x} = \frac{1}{14}\left(c_1 + 2F^{*}c_2 + 3H^{*}c_3\right) = x.$$

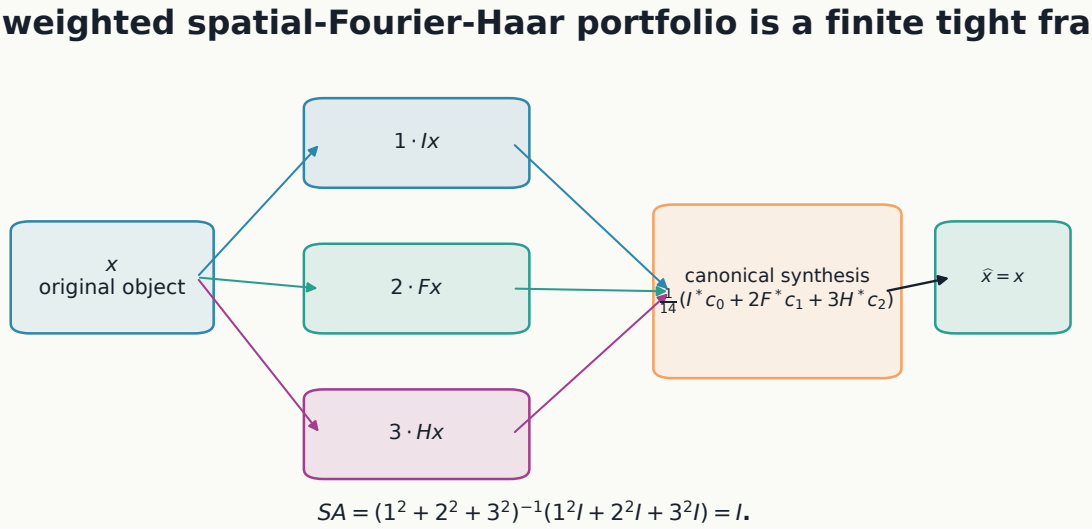


Figure 6.3: The master spatial-Fourier-Haar portfolio is redundant but exactly reconstructing.

6.6 Completion direction

A refinement tower

$$\mathcal{P}^{(0)} \prec \mathcal{P}^{(1)} \prec \cdots$$

is algebraically coherent when every parent is the sum of its children. In a finite object this tower terminates. In an infinite object one must additionally declare a topology or inverse-limit structure and prove convergence.

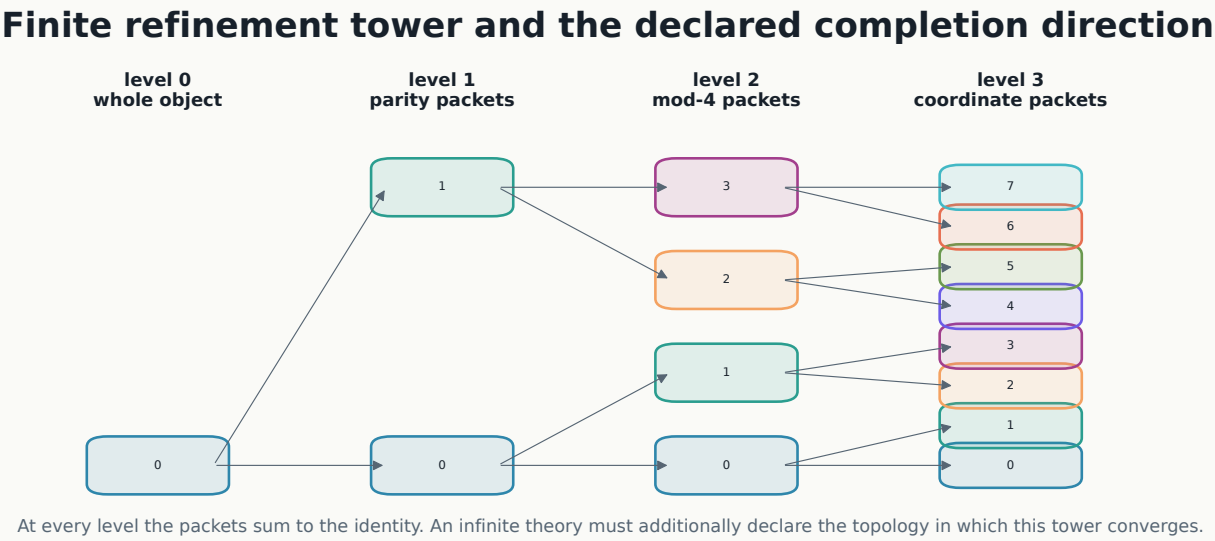


Figure 6.4: The finite master tower. The figure also marks the additional datum needed in an infinite theory: a declared convergence structure.

Seed-to-finish Step 9: refinement

Replace a packet by a complete child family. Because all children are retained, refinement is an invertible coordinate change.

Seed-to-finish Step 10: atlas portfolio

Transport ordinary coordinate projectors through Fourier, wavelet, modal, or other invertible transforms. Several unitary atlases may be retained together through the exact tight-frame synthesis theorem.

Chapter 7

Nonlinear packets, obstruction packets, and completion

7.1 Exact nonlinear packet coordinates

Let $N : X \to Y$ be an arbitrary map between finite spaces with complete packet systems $\mathcal{P}$ on X and $\mathcal{Q}$ on Y. Define the packet-coordinate map

$$\widetilde{N} = \mathcal{A}_{\mathcal{Q}} N \mathcal{S}_{\mathcal{P}}.$$

Theorem 7.1 (Exact nonlinear conjugacy)**.** *The nonlinear map is recovered exactly by*

$$\boxed{N = \mathcal{S}_{\mathcal{Q}} \widetilde{N} \mathcal{A}_{\mathcal{P}}.}$$

No linearity assumption on N is required.

Proof. Use $\mathcal{S}_{\mathcal{Q}}\mathcal{A}_{\mathcal{Q}} = I_Y$ and $\mathcal{S}_{\mathcal{P}}\mathcal{A}_{\mathcal{P}} = I_X$:

$$\mathcal{S}_{\mathcal{Q}} \widetilde{N} \mathcal{A}_{\mathcal{P}} = \mathcal{S}_{\mathcal{Q}} \mathcal{A}_{\mathcal{Q}} N \mathcal{S}_{\mathcal{P}} \mathcal{A}_{\mathcal{P}} = N.$$

□

The theorem says that nonlinear packet coordinates are exact, but it does not say that packets evolve independently. Nonlinearity creates interactions among input packets.

7.2 Multilinear interaction tensors

Let $B : X \times X \to Y$ be bilinear. Since $x = \sum_a P_a x$,

$$\boxed{B(x, x) = \sum_{a,c} B(P_a x, P_c x).}$$

Resolving the output gives

$$B_{b;ac}(x) = Q_b B(P_a x, P_c x),$$

which is a three-index interaction tensor rather than a two-index transfer matrix.

More generally, for a k-linear map M,

$$Q_b M(x_1, \dots, x_k) = \sum_{a_1,\dots,a_k} Q_b M(P_{a_1} x_1, \dots, P_{a_k} x_k).$$

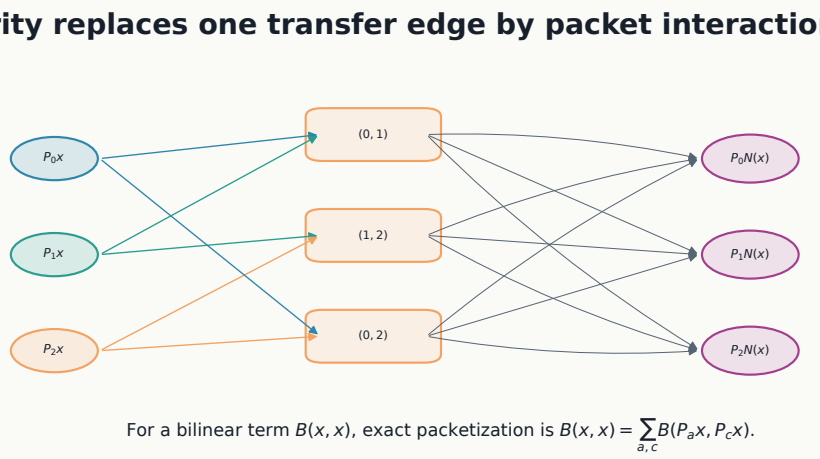


Figure 7.1: Linear transfer edges become nonlinear interaction hyperedges indexed by several input packets and one output packet.

7.3 Packet jets

If N is k times differentiable, its kth derivative is a k-linear map. Therefore

$$\boxed{Q_b D^k N(x)[h_1, \dots, h_k] = \sum_{a_1,\dots,a_k} Q_b D^k N(x)[P_{a_1} h_1, \dots, P_{a_k} h_k].}$$

The complete family of these tensors is the packet jet. Ordinary Taylor expansion may then be written packet by packet without changing its sum.

For a quadratic fixed-point equation

$$x = s + B(x, x),$$

repeated substitution produces rooted binary trees. Each leaf carries a seed packet, each internal vertex carries a bilinear interaction block, and synthesis sums all packet-labeled trees. Convergence is an analytic question; the packet labeling itself is exact at every finite order.

7.4 The master nonlinear map

Define

$$N(x) = x + (Ux) \odot (U^3 x),$$

where $\odot$ is coordinatewise multiplication. For the master vector,

$$\boxed{N(x) = (22, -5, -2, 6, 1, 7, -2, 3)^{\mathsf{T}}.}$$

Detailed calculation first coordinate

Because $(Ux)_0 = x_7 = 5$ and $(U^3x)_0 = x_5 = 4$,

$$N(x)_0 = x_0 + (Ux)_0(U^3x)_0 = 2 + 5 \cdot 4 = 22.$$

Every remaining coordinate follows the same cyclic rule.

The bilinear term resolves exactly as

$$(Ux) \odot (U^3x) = \sum_{a,c} (UE_ax) \odot (U^3E_cx).$$

The symbolic residual is the zero vector.

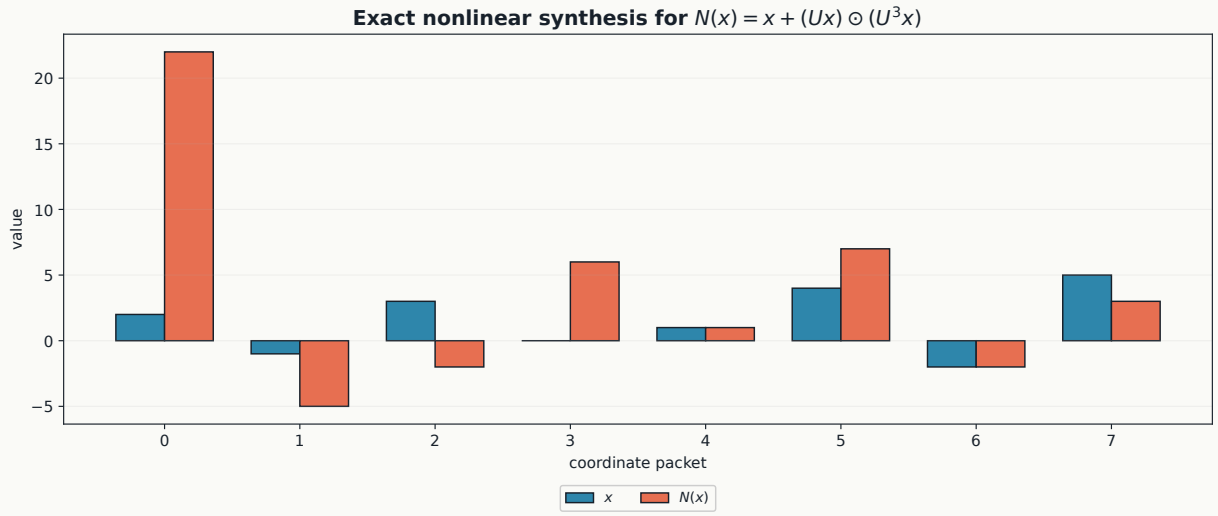


Figure 7.2: The master input and nonlinear output. The full output is synthesized from exact interaction packets.

7.5 Obstruction packets

Packet coordinates can expose a failure without guaranteeing that it vanishes. Let $L : H \to K$ be a linear map between finite-dimensional inner-product spaces. Let $L^\dagger$ be its Moore–Penrose inverse and define

$$\Pi_{\mathrm{Obs}} = I - LL^\dagger.$$

Theorem 7.2 (Finite obstruction criterion)**.** *The equation*

$$Lh = d$$

is solvable if and only if

$$\boxed{\Pi_{\mathrm{Obs}} d = 0.}$$

The obstruction packet is the orthogonal projection of d onto $\mathrm{Ran}(L)^\perp$.

Proof. $LL^\dagger$ is the orthogonal projector onto $\mathrm{Ran}(L)$. Thus $I - LL^\dagger$ projects onto the orthogonal complement. The vector d lies in the range exactly when its complementary projection vanishes. □

Take

$$L = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{pmatrix}.$$

Then

$$\Pi_{\mathrm{Obs}} = \frac{1}{3} \begin{pmatrix} 1 & 1 & -1 \\ 1 & 1 & -1 \\ -1 & -1 & 1 \end{pmatrix}.$$

For $d = (1, 2, 3)^{\mathsf{T}}$ the obstruction is zero. For

$$d' = (1, 2, 4)^{\mathsf{T}},$$

$$\boxed{\Pi_{\mathrm{Obs}} d' = \left(-\frac{1}{3}, -\frac{1}{3}, \frac{1}{3}\right)^{\mathsf{T}} \neq 0.}$$

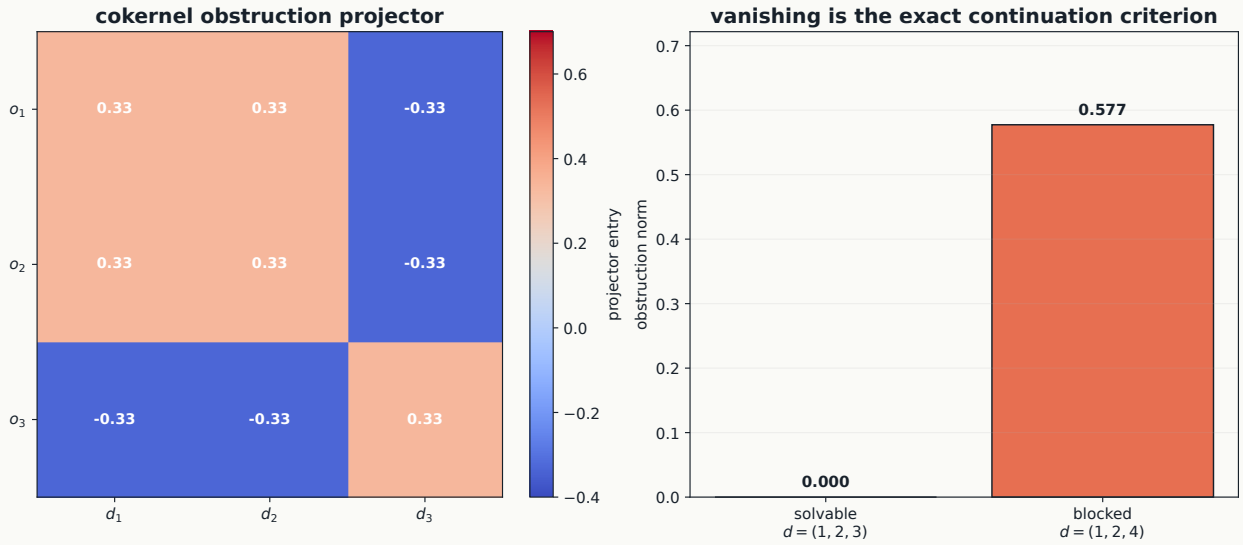


Figure 7.3: Positive and negative controls for the finite obstruction theorem. The nonzero packet localizes the exact failed compatibility condition.

7.6 Completion from a dense core

The finite theorems extend to completed spaces only under explicit continuity hypotheses.

Theorem 7.3 (Dense-core extension)**.** *Let X be a normed space, let $D \subseteq X$ be dense, and let $A, B : X \to Y$ be bounded linear maps into a normed space Y. If $Ax = Bx$ for every $x \in D$, then $A = B$ on X.*

Proof. For $x \in X$, choose $x_n \in D$ with $x_n \to x$. Boundedness gives

$$Ax = \lim_n Ax_n = \lim_n Bx_n = Bx.$$

□

Corollary 7.4 (Completion preserves a finite packet split)**.** *Suppose bounded operators $P_1, \dots, P_m$ satisfy*

$$P_i P_j = \delta_{ij} P_i, \qquad \sum_i P_i = I$$

on a dense invariant subspace D. Then the same identities hold on the completion of D.

Proof. Every difference between the two sides is a bounded operator vanishing on D, so apply theorem 7.3. □

Boundary infinite towers

An inverse system of exact finite reconstructions does not automatically produce a nonempty, complete, or physically meaningful limit. One must prove compatibility, completeness, domain preservation, and convergence in the declared topology.

Seed-to-finish Step 11: nonlinear interaction

Replace transfer matrices by interaction tensors or trees. Analysis and synthesis remain exact even though packet independence generally fails.

Seed-to-finish Step 12: obstruction

Project a continuation defect onto the complement of the correction range. Vanishing is the exact solvability criterion; nonvanishing is retained as a localized packet rather than hidden.

Seed-to-finish Step 13: completion

Pass from a dense finite or algebraic core only after the relevant projectors and operators are proved bounded or otherwise controlled on their domains.

Chapter 8

One seed through the entire calculus

8.1 The complete step ledger

This chapter repeats the master cyclic example without skipping any layer. The purpose is to show that the definitions form one continuous calculus rather than a collection of disconnected identities.

Master example: the length-eight cyclic signal through the complete calculus

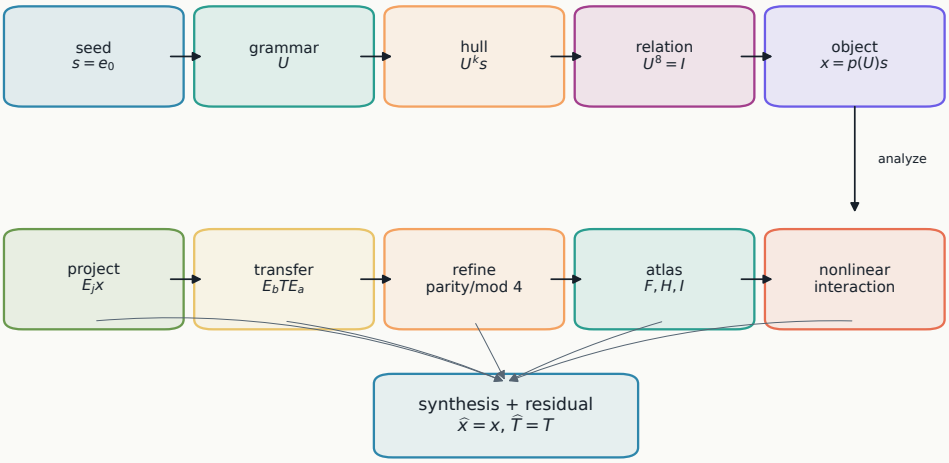


Figure 8.1: The master example from primitive seed through nonlinear interaction and final reconstruction.

8.1.1 Step 0: choose the ambient type

Declare

$$X = \mathbb{C}^8$$

with standard basis $e_0, \dots, e_7$ and the usual Hermitian inner product. The finite dimension makes all linear operators bounded and all sums finite.

8.1.2 Step 1: choose the seed

Set

$$s = e_0.$$

At this stage only one primitive vector has been supplied.

8.1.3 Step 2: choose the operator grammar

Let the grammar be generated by the forward cyclic shift U. The admissible linear operators are polynomials in U.

8.1.4 Step 3: compute the orbit

The orbit is

$$\{s, Us, \dots, U^7 s\} = \{e_0, e_1, \dots, e_7\}.$$

The orbit matrix has rank eight, so one seed generates all of X.

8.1.5 Step 4: impose the relation

The exact relation is

$$U^8 = I.$$

Hence every operator word has a unique normal form of degree at most seven.

8.1.6 Step 5: generate the object

Use

$$p(z) = 2 - z + 3z^2 + z^4 + 4z^5 - 2z^6 + 5z^7$$

and form

$$x = p(U)s = (2, -1, 3, 0, 1, 4, -2, 5)^{\mathsf{T}}.$$

8.1.7 Step 6: declare the packet addresses

Use four exact atlases:

(a) coordinate packets E_j;

(b) parity packets $P_{\mathrm{e}}, P_{\mathrm{o}}$;

(c) mod-four packets $R_0, \dots, R_3$;

(d) Fourier character packets $Q_0, \dots, Q_7$.

Each family is complete at its own level.

8.1.8 Step 7: analyze and synthesize

Coordinate analysis gives

$$\mathcal{A}_E x = (E_0 x, \dots, E_7 x),$$

and

$$\mathcal{S}_E \mathcal{A}_E x = \sum_{j=0}^{7} E_j x = x.$$

The exact residual is zero.

8.1.9 Step 8: declare a transfer operator

Set

$$T = 2I - U + 3U^2.$$

The output is

$$Tx = (-7, 11, 13, -6, 11, 7, -5, 24)^{\mathsf{T}}.$$

8.1.10 Step 9: reconstruct the operator

Resolve

$$T_{ba} = E_b T E_a.$$

Then

$$\sum_{a,b} T_{ba} = T.$$

The support graph has exactly the cyclic identity, one-step, and two-step bands dictated by the polynomial in U.

8.1.11 Step 10: compose by paths

Let $S = I + U^3$. The product is reconstructed from

$$E_b S T E_a = \sum_{\gamma} (E_b S E_\gamma)(E_\gamma T E_a).$$

The complete path-sum residual is zero.

8.1.12 Step 11: refine

Use

$$P_{\mathrm{e}} = R_0 + R_2, \qquad P_{\mathrm{o}} = R_1 + R_3,$$

and

$$R_r = E_r + E_{r+4}.$$

Every coarse object packet and every coarse operator block is recovered by summing its fine descendants.

8.1.13 Step 12: transport to Fourier space

Let F be the unitary DFT. The projectors

$$Q_k = F^* E_k F$$

are complete, mutually annihilating, and idempotent. In this atlas,

$$FTF^* = \mathrm{diag}(2 - \zeta_k + 3\zeta_k^2).$$

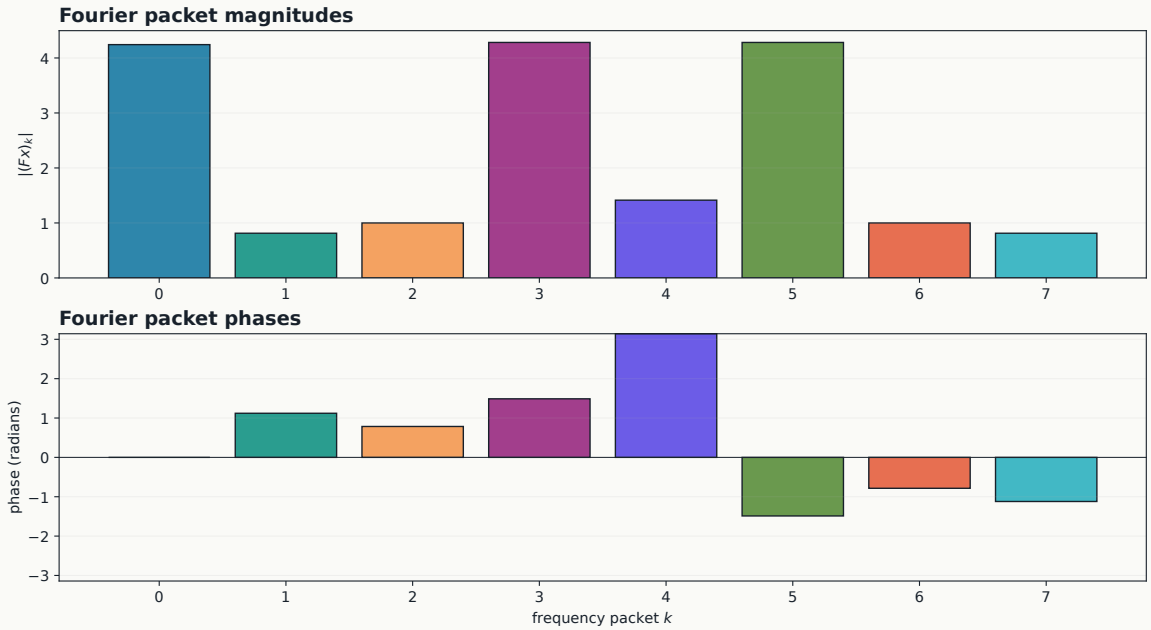


Figure 8.2: The exact Fourier magnitudes and phases of the master vector. Both are required for complete synthesis.

8.1.14 Step 13: retain several atlases simultaneously

Analyze x through spatial, Fourier, and Haar transforms with weights $(1, 2, 3)$. The canonical tight-frame synthesis divides by

$$1^2 + 2^2 + 3^2 = 14$$

and reconstructs x exactly.

8.1.15 Step 14: apply a nonlinear operation

Use

$$N(x) = x + (Ux) \odot (U^3 x).$$

The output interaction sum over all packet pairs equals the direct nonlinear term, and output synthesis returns

$$N(x) = (22, -5, -2, 6, 1, 7, -2, 3)^{\mathsf{T}}.$$

8.1.16 Step 15: test an obstruction

For the separate continuation map

$$L = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{pmatrix},$$

the datum $(1, 2, 3)^{\mathsf{T}}$ has zero obstruction and is solvable. The datum $(1, 2, 4)^{\mathsf{T}}$ has obstruction

$$\left(-\frac{1}{3}, -\frac{1}{3}, \frac{1}{3}\right)^{\mathsf{T}},$$

so no exact correction exists in the declared range.

8.1.17 Step 16: close the reconstruction loop

The exact symbolic certificates are

$$\begin{aligned} &\operatorname{rank} \mathcal{O}_U(s) = 8, && U^8 - I = 0, \\ &\textstyle\sum_j E_j x - x = 0, && \textstyle\sum_{a,b} E_b T E_a - T = 0, \\ &\textstyle\sum_{a,b,\gamma} E_b S E_\gamma T E_a - ST = 0, && \textstyle\sum_k Q_k - I = 0, \\ &\mathcal{S}\mathcal{A}x - x = 0, && \textstyle\sum_b E_b N(x) - N(x) = 0. \end{aligned}$$

The intended negative control is the nonzero blocked obstruction.

Master seed-to-reconstruction certificate ledger

orbit rank	8/8
relation residual	0.000e+00
spatial synthesis	0.000e+00
transfer reconstruction	0.000e+00
path-product residual	0.000e+00
Fourier completeness	0.000e+00
multi-atlas synthesis	0.000e+00
nonlinear synthesis	0.000e+00
solvable obstruction	0.000e+00
blocked obstruction	5.774e-01

The nonzero blocked-obstruction row is the intended negative control; all reconstruction identities vanish exactly in symbolic arithmetic.

Figure 8.3: The master certificate ledger. The nonzero obstruction row is intentionally retained because a calculus must detect failures as well as exact reconstructions.

8.2 Complete symbolic implementation

```python
from proof_atlas.foundations import build_certificate, matrix_zero

c = build_certificate()
assert c.orbit_rank == 8
assert matrix_zero(c.relation_residual)
assert matrix_zero(c.spatial_reconstruction_residual)
assert matrix_zero(c.transfer_reconstruction_residual)
assert matrix_zero(c.product_path_residual)
assert matrix_zero(c.fourier_completeness_residual)
assert matrix_zero(c.multi_atlas_reconstruction_residual)
assert matrix_zero(c.nonlinear_packet_residual)
assert matrix_zero(c.bilinear_interaction_residual)
assert matrix_zero(c.solvable_obstruction)
assert not matrix_zero(c.blocked_obstruction)
```

Chapter 9

The start-to-finish realization theorem

9.1 Statement

The preceding chapters can now be assembled into one dependency-ordered theorem.

Theorem 9.1 (Finite seed-to-reconstruction theorem)**.** *Let X be finite-dimensional. Suppose the following data are declared:*

(i) a finite seed family S;

(ii) a fixed unital operator algebra $\mathcal{A} \subseteq \mathrm{End}(X)$;

(iii) a relation system presenting the generated hull;

(iv) one or more complete packet atlases;

(v) exact refinement maps among selected atlases;

(vi) an admissible linear, multilinear, polynomial, or arbitrary finite map;

(vii) a synthesis law for every retained coordinate family.

Then:

(a) the generated object lies in the quotient of the free seed module by the relation kernel;

(b) every complete atlas reconstructs the object exactly;

(c) every linear operator reconstructs from its transfer blocks;

(d) operator products reconstruct from packet paths;

(e) complete refinement preserves both object and operator reconstruction;

(f) arbitrary nonlinear maps possess exact packet-coordinate conjugates;

(g) multilinear terms decompose into complete interaction tensors;

(h) finite correction problems admit exact obstruction packets;

(i) every bounded identity established on a dense invariant core extends to its completion;

(j) every final equality has an explicit object-level or operator-level residual certificate.

Proof. Part (a) is theorem 2.1. Parts (b) and (c) are the object and operator reconstruction theorems. Part (d) follows from repeated insertion of packet completeness. Part (e) is theorem 6.1 and the refined-block identity. Part (f) is theorem 7.1; part (g) follows by multilinearity. Part (h) is theorem 7.2. Part (i) is theorems 7.3 and 7.4. The residual in part (j) is obtained by subtracting the reconstructed quantity from the direct quantity at the appropriate type. □

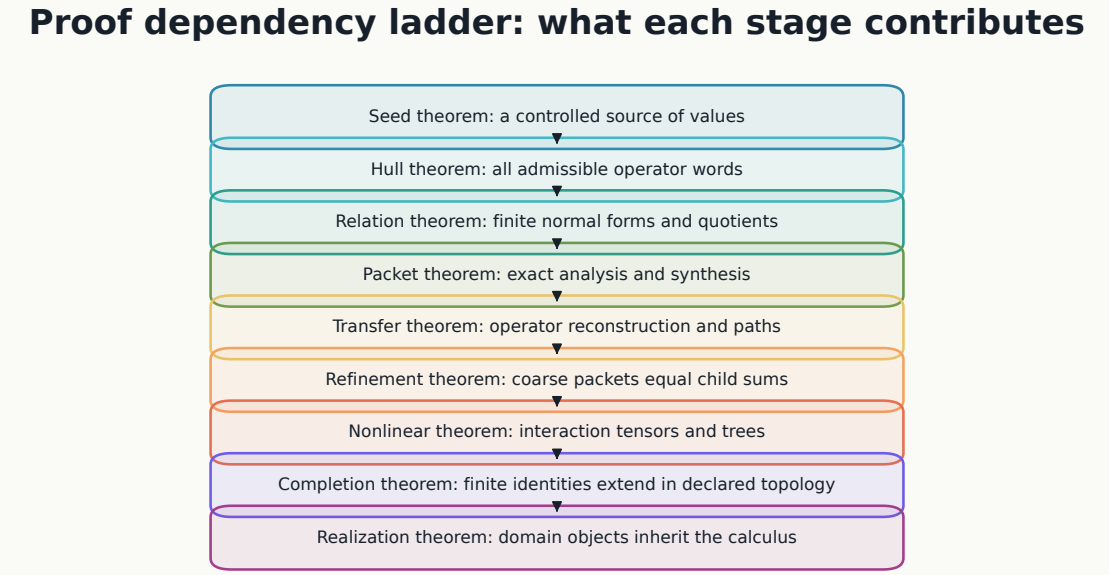


Figure 9.1: Each theorem depends on the preceding layers. The cross-domain examples come after, not before, this proof chain.

9.2 Why the construction deserves the name calculus

The term calculus is justified by closure under the recurring operations:

- seeds generate controlled hulls;
- relations reduce operator words to normal forms;
- projectors provide exact coordinate differentials in the broad sense of resolving variation by packet;
- transfer blocks compose by a matrix law;
- refinement has exact chain maps and inverse synthesis;
- nonlinear maps possess packet jets and interaction trees;
- obstructions localize failed continuation;
- completion transports finite identities under declared continuity hypotheses;
- synthesis returns every retained calculation to the original object space.

These operations are not one isolated theorem. They form a closed grammar for constructing, resolving, transforming, refining, and reconstructing mathematical objects.



Figure 9.2: The operational cycle of the calculus. Reconstruction is not the end: the residual may become the seed for the next refinement.

9.3 Transition to the seven independent realizations

The master example proves the entire chain in one finite cyclic module. The next seven chapters retain all of their original content and repeat the verification in different domains. Their role is to show that the common theorem is not tied to the cyclic shift example:

1. a general rational matrix decomposition;
2. signal polyphase, Fourier, and wavelet coordinates;
3. quantum circuit paths and reduced states;
4. conditional expectation and Bayesian information loss;
5. closed analytic and arithmetic coefficient families;
6. physical normal modes and conserved energy;

7. nonlinear filters and geometric holonomy.

Part II

Seven Worked Areas

Chapter 10

Area I: finite linear algebra and exact transfer blocks

10.1 Problem

Resolve a concrete vector and a concrete operator in $\mathbb{Q}^6$ into three two-dimensional packets, prove exact reconstruction, prove the block product law on a second operator, and refine one packet into one-dimensional children.

10.2 Step 1: declare the projectors

Partition the coordinate positions as

$$X_0 = \operatorname{span}(e_1, e_2), \qquad X_1 = \operatorname{span}(e_3, e_4), \qquad X_2 = \operatorname{span}(e_5, e_6).$$

The projectors are

$$P_0 = \operatorname{diag}(1,1,0,0,0,0), \quad P_1 = \operatorname{diag}(0,0,1,1,0,0), \quad P_2 = \operatorname{diag}(0,0,0,0,1,1).$$

They visibly satisfy

$$P_i P_j = \delta_{ij} P_i, \qquad P_0 + P_1 + P_2 = I_6.$$

10.3 Step 2: decompose and reconstruct the object

Take

$$x = (2, -1, 3, 4, -2, 5)^{\mathsf{T}}.$$

Then

$$\begin{aligned} P_0 x &= (2, -1, 0, 0, 0, 0)^{\mathsf{T}}, \\ P_1 x &= (0, 0, 3, 4, 0, 0)^{\mathsf{T}}, \\ P_2 x &= (0, 0, 0, 0, -2, 5)^{\mathsf{T}}. \end{aligned}$$

Adding gives x exactly.

Detailed calculation for object synthesis

The coordinate sum can be checked entry by entry:

	1	2	3	4	5	6
P_0x	2	−1	0	0	0	0
P_1x	0	0	3	4	0	0
P_2x	0	0	0	0	−2	5
$\sum_i P_ix$	2	−1	3	4	−2	5

The important point is that the zero pattern is forced by the projectors. No coordinate is counted twice and no coordinate is omitted.

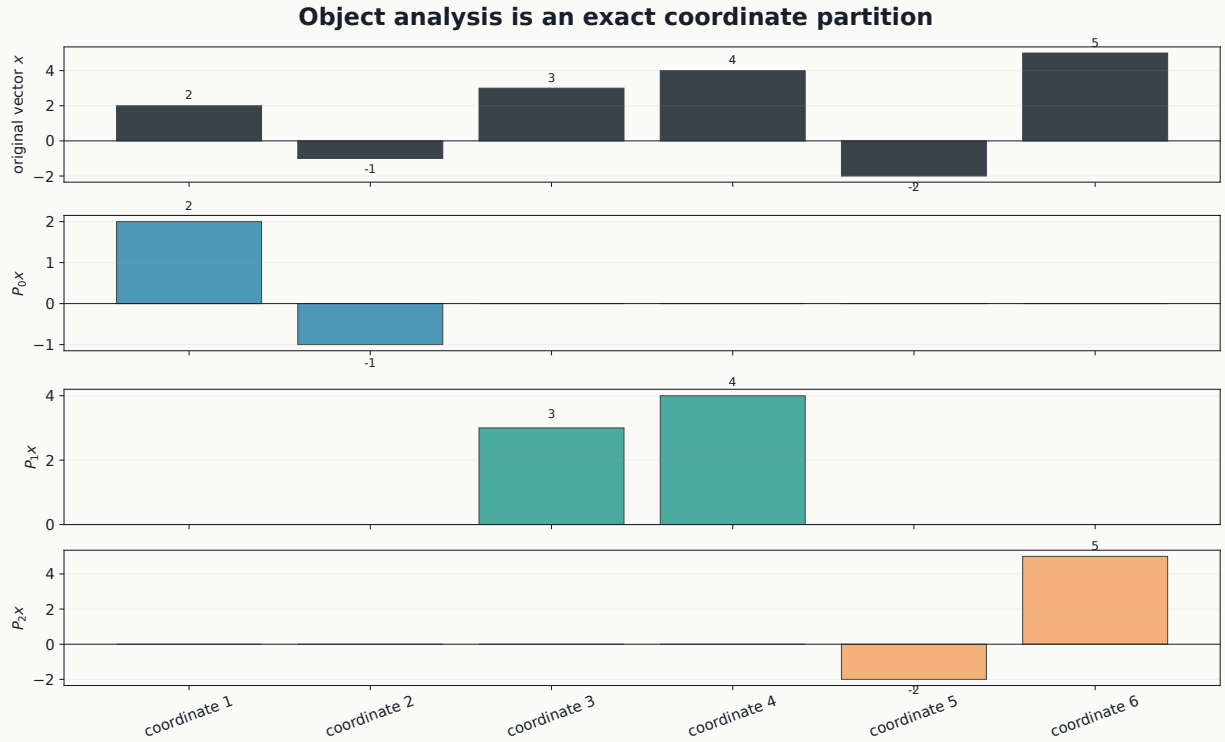


Figure 10.1: The vector before and after packet analysis. The three packet vectors have disjoint supports and sum coordinatewise to the original vector.

10.4 Step 3: resolve the operator

Let

$$T = \begin{pmatrix} 1 & 2 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 & 1 \\ 2 & 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 2 & 1 & 0 \\ 1 & 0 & 0 & 1 & 2 & 1 \\ 0 & 0 & 1 & 0 & 0 & 3 \end{pmatrix}.$$

In 2×2 packet blocks,

$$[T_{\beta\alpha}]^2_{\beta,\alpha=0} = \begin{pmatrix} \left(\begin{smallmatrix} 1 & 2 \\ 0 & 1 \end{smallmatrix}\right) & \left(\begin{smallmatrix} 0 & 0 \\ 1 & 0 \end{smallmatrix}\right) & \left(\begin{smallmatrix} 1 & 0 \\ 0 & 1 \end{smallmatrix}\right) \\ \left(\begin{smallmatrix} 2 & 0 \\ 0 & 1 \end{smallmatrix}\right) & \left(\begin{smallmatrix} 1 & 1 \\ 0 & 2 \end{smallmatrix}\right) & \left(\begin{smallmatrix} 0 & 0 \\ 1 & 0 \end{smallmatrix}\right) \\ \left(\begin{smallmatrix} 1 & 0 \\ 0 & 0 \end{smallmatrix}\right) & \left(\begin{smallmatrix} 0 & 1 \\ 1 & 0 \end{smallmatrix}\right) & \left(\begin{smallmatrix} 2 & 1 \\ 0 & 3 \end{smallmatrix}\right) \end{pmatrix}.$$

Summing the nine embedded blocks reproduces T entry by entry.

Applying the operator gives

$$Tx = (-2, 7, 11, 5, 7, 18)^{\mathsf{T}}.$$

The three output packets are

$$(-2, 7, 0, 0, 0, 0)^{\mathsf{T}}, \quad (0, 0, 11, 5, 0, 0)^{\mathsf{T}}, \quad (0, 0, 0, 0, 7, 18)^{\mathsf{T}}.$$

The block contributions can be displayed before they are added. For output packet X_0,

$$\begin{aligned} T_{00}P_0x &= (0, -1, 0, 0, 0, 0)^{\mathsf{T}}, \\ T_{01}P_1x &= (0, 3, 0, 0, 0, 0)^{\mathsf{T}}, \\ T_{02}P_2x &= (-2, 5, 0, 0, 0, 0)^{\mathsf{T}}, \end{aligned}$$

so their sum is $P_0Tx = (-2, 7, 0, 0, 0, 0)^{\mathsf{T}}$. The other two output packets obey the same rule:

output	from X_0	from X_1	from X_2	sum
X_0	$(0, -1)$	$(0, 3)$	$(-2, 5)$	$(-2, 7)$
X_1	$(4, -1)$	$(7, 8)$	$(0, -2)$	$(11, 5)$
X_2	$(2, 0)$	$(4, 3)$	$(1, 15)$	$(7, 18)$

where only the two coordinates belonging to the displayed output packet are shown.

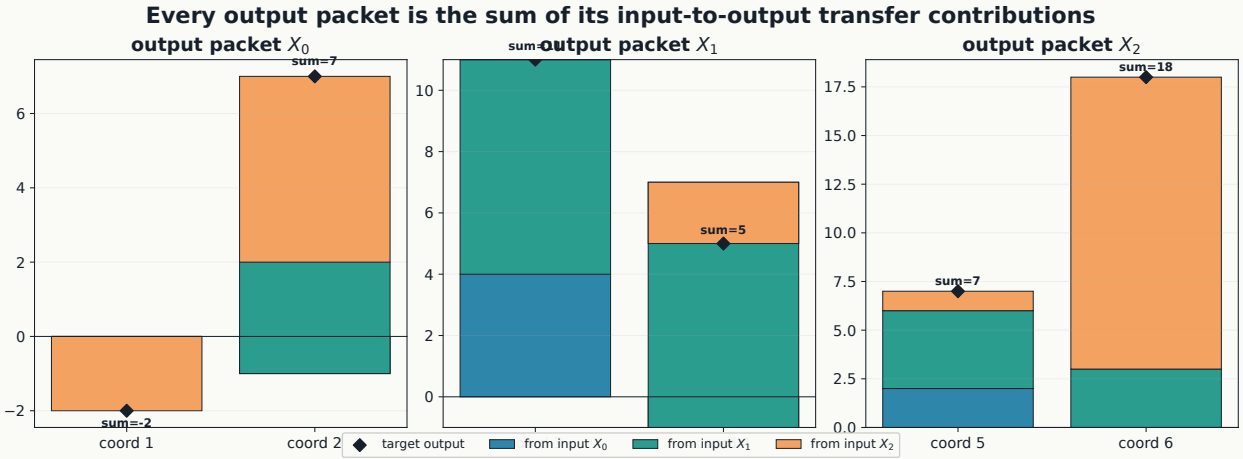


Figure 10.2: Each pair of bars is one output packet. Colored stacks are the three input-to-output contributions, and the diamond marks their exact sum.

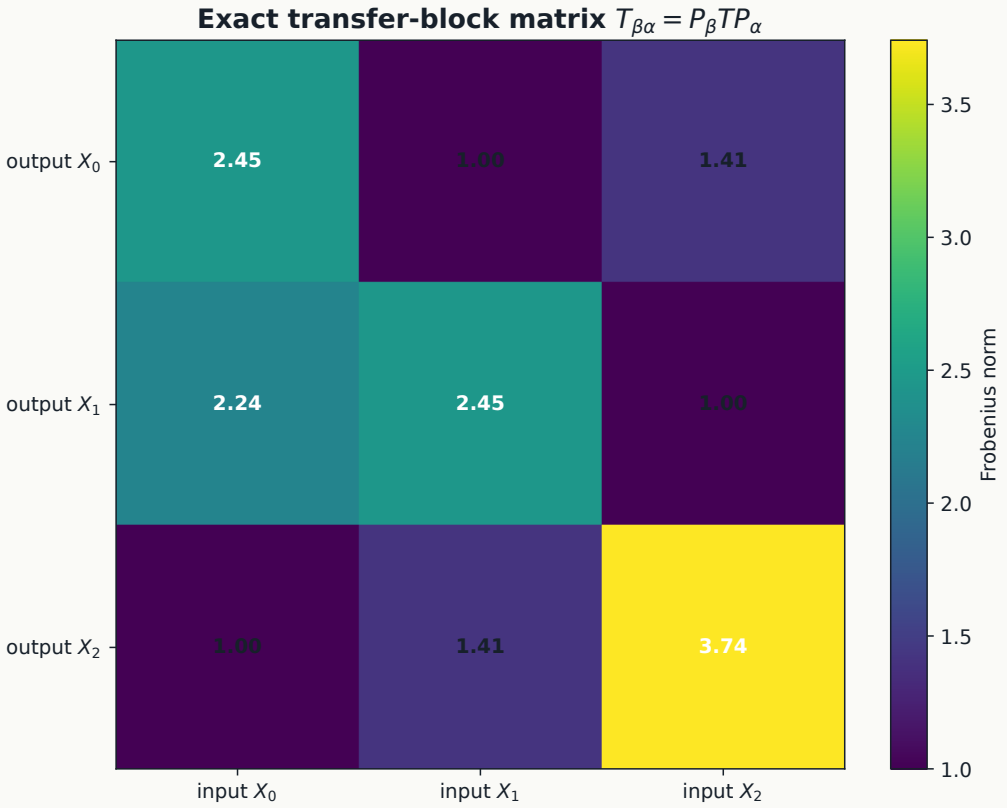


Figure 10.3: Norms of the nine exact transfer blocks. The heatmap is a visualization of an exact matrix decomposition, not an approximation.

10.5 Step 4: multiply operators by packet paths

Use the explicit second operator

$$S = \begin{pmatrix} 2 & 0 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 & 0 & 1 \end{pmatrix}.$$

For each input packet α and output packet γ, the code computes

$$P_\gamma S T P_\alpha - \sum_{\beta=0}^{2} (P_\gamma S P_\beta)(P_\beta T P_\alpha).$$

Every entry is exactly zero. This is a complete finite verification of theorem 4.6 for a nontrivial pair of dense-crossing operators.

One block can be inspected without code. For input X_2 and output X_0,

$$(P_0 S T P_2)_{X_0 \leftarrow X_2} = \begin{pmatrix} 2 & 0 \\ 1 & 4 \end{pmatrix}.$$

The three intermediate packet paths give

$$(P_0SP_0)(P_0TP_2) = \begin{pmatrix} 2 & 0 \\ 1 & 1 \end{pmatrix},$$
$$(P_0SP_1)(P_1TP_2) = \begin{pmatrix} 0 & 0 \\ 0 & 0 \end{pmatrix},$$
$$(P_0SP_2)(P_2TP_2) = \begin{pmatrix} 0 & 0 \\ 0 & 3 \end{pmatrix}.$$

Their sum is exactly the direct block. This calculation shows what the path index β means: it records the packet occupied between T and S.

10.6 Step 5: refine one packet

Define

$$Q_{00} = \mathrm{diag}(1,0,0,0,0,0), \qquad Q_{01} = \mathrm{diag}(0,1,0,0,0,0).$$

Then

$$\boxed{P_0 = Q_{00} + Q_{01}.}$$

Every coarse block touching P_0 is consequently the sum of its child blocks. For example,

$$P_2TP_0 = P_2TQ_{00} + P_2TQ_{01},$$

and refining both sides gives

$$P_0TP_0 = Q_{00}TQ_{00} + Q_{00}TQ_{01} + Q_{01}TQ_{00} + Q_{01}TQ_{01}.$$

Thus refinement changes the address resolution of the block matrix, not the represented operator.



Figure 10.4: A two-dimensional coordinate packet refined into two one-dimensional descendants.

Exact certificate

The exact symbolic residuals are

identity	residual
$\sum_i P_i x - x$	$0_{6\times 1}$
$\sum_{i,j} P_i T P_j - T$	$0_{6\times 6}$
$P_\gamma S T P_\alpha - \sum_\beta S_{\gamma\beta} T_{\beta\alpha}$	0 for all γ, α
$P_0 - Q_{00} - Q_{01}$	$0_{6\times 6}$

10.7 Selected symbolic implementation

```
P = coordinate_projectors((2, 2, 2))
packets = tuple(Pa * x for Pa in P)
object_residual = sum(packets, sp.zeros(6, 1)) - x

blocks_T = transfer_blocks(T, P)
operator_residual = reconstruct_operator(blocks_T) - T

blocks_ST = transfer_blocks(S * T, P)
product_rebuilt[g][a] = sum(
    blocks_S[g][b] * blocks_T[b][a] for b in range(3)
)
```

Chapter 11

Area II: polyphase, Fourier, and wavelet signal coordinates

11.1 Problem

Represent one finite signal in residue, polyphase, Haar, and Fourier coordinates; refine the phase system; reconstruct the signal exactly from every complete atlas; and show that Fourier bands are genuine pulled-back projectors.

11.2 Step 1: the signal and residue projectors

Use

$$x = (3, 1, 4, 1, 5, 9, 2, 6)^{\mathsf{T}}.$$

For order N, define

$$(P_{N,r}x)[n] = \mathbf{1}_{n \equiv r \pmod N} x[n].$$

The projectors satisfy

$$P_{N,r}P_{N,s} = \delta_{rs}P_{N,r}, \qquad \sum_{r=0}^{N-1} P_{N,r} = I.$$

11.3 Step 2: exact polyphase analysis and synthesis

The order-four channels are

$$x_0 = (3, 5)^{\mathsf{T}}, \quad x_1 = (1, 9)^{\mathsf{T}},$$
$$x_2 = (4, 2)^{\mathsf{T}}, \quad x_3 = (1, 6)^{\mathsf{T}}.$$

Interleaving them returns

$$(3, 1, 4, 1, 5, 9, 2, 6)^{\mathsf{T}} = x.$$

The address rule can be written as a table:

n	0	1	2	3	4	5	6	7
$r = n \bmod 4$	0	1	2	3	0	1	2	3
$q = \lfloor n/4 \rfloor$	0	0	0	0	1	1	1	1
$x[n]$	3	1	4	1	5	9	2	6

Analysis sends the entry at address n to channel r and decimated address q. Synthesis reverses this map by the exact formula

$$(\mathcal{S}_4(x_0, x_1, x_2, x_3))[4q + r] = x_r[q].$$

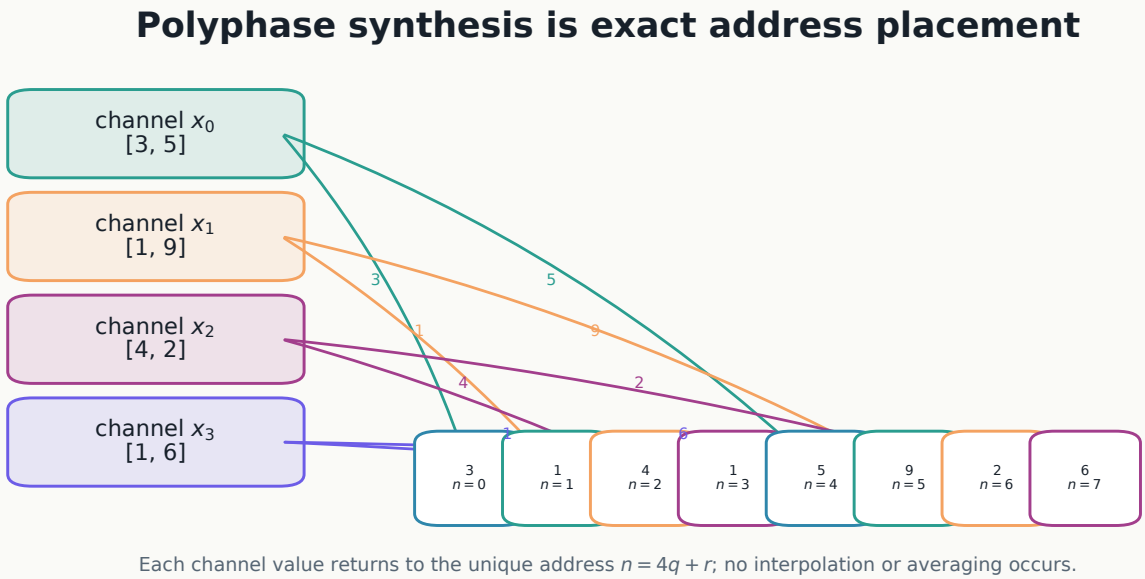


Figure 11.1: Every polyphase coefficient is returned to the unique address $n = 4q + r$. The arrows show the complete synthesis map explicitly.

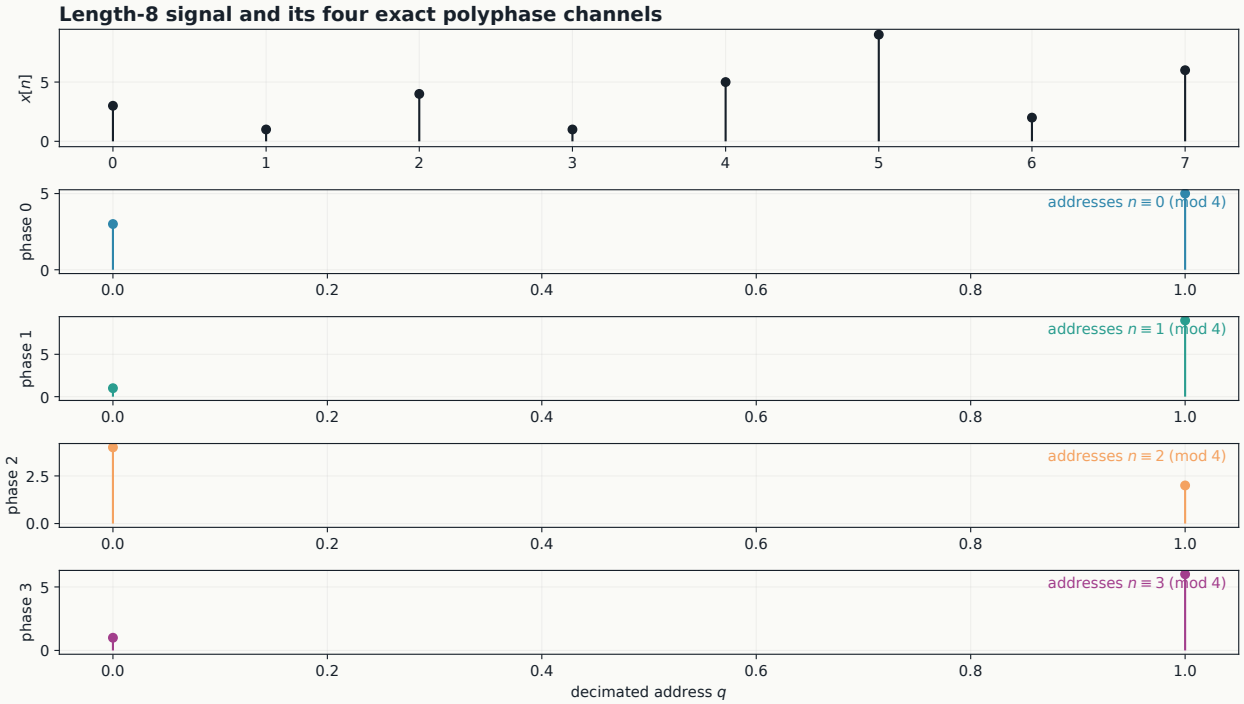


Figure 11.2: The signal and its four exact phase channels. Each sample appears in one and only one packet.

11.4 Step 3: nested phase refinement

The order-two packets split as

$$\boxed{P_{2,0} = P_{4,0} + P_{4,2}}, \qquad \boxed{P_{2,1} = P_{4,1} + P_{4,3}}.$$

At the channel level,

$$(3, 4, 5, 2) \longleftrightarrow (3, 5) \oplus (4, 2),$$

and

$$(1, 1, 9, 6) \longleftrightarrow (1, 9) \oplus (1, 6).$$

No interpolation is involved: refinement is address separation.

11.5 Step 4: Haar analysis

On every adjacent pair, use

$$H_2 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}.$$

The block-diagonal one-level transform H satisfies $H^{\mathsf{T}} H = I_8$. Its coefficients are

$$Hx = \left(2\sqrt{2}, \sqrt{2}, \frac{5\sqrt{2}}{2}, \frac{3\sqrt{2}}{2}, 7\sqrt{2}, -2\sqrt{2}, 4\sqrt{2}, -2\sqrt{2} \right)^{\mathsf{T}}.$$

Since $H^{-1} = H^{\mathsf{T}}$,

$$\boxed{x = H^{\mathsf{T}} H x.}$$

This is the simplest orthonormal wavelet filter-bank realization of the transform-atlas theorem.

For each adjacent pair (a, b), analysis produces

$$\ell = \frac{a + b}{\sqrt{2}}, \qquad h = \frac{a - b}{\sqrt{2}},$$

and synthesis returns

$$a = \frac{\ell + h}{\sqrt{2}}, \qquad b = \frac{\ell - h}{\sqrt{2}}.$$

For the first pair $(3, 1)$ this gives $(\ell, h) = (2\sqrt{2}, \sqrt{2})$ and then

$$\frac{2\sqrt{2} + \sqrt{2}}{\sqrt{2}} = 3, \qquad \frac{2\sqrt{2} - \sqrt{2}}{\sqrt{2}} = 1.$$

The same two-line inverse calculation reconstructs the other three pairs.

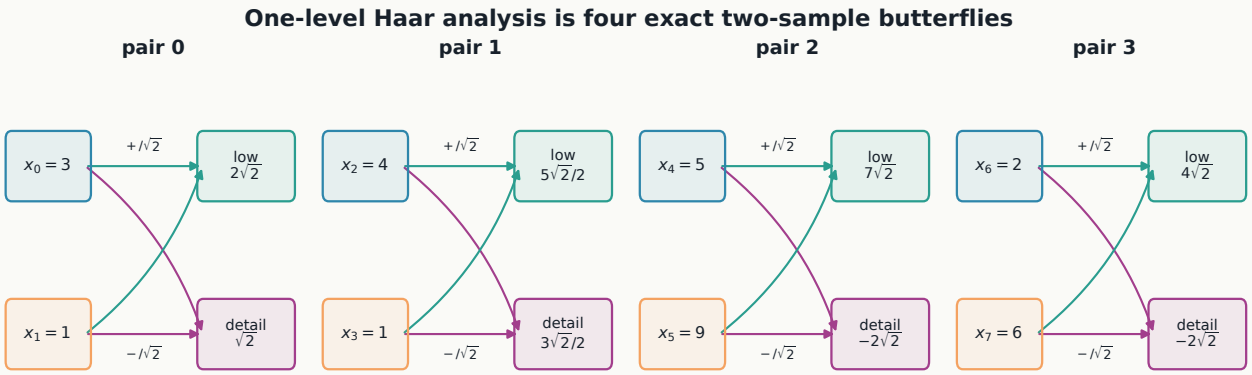


Figure 11.3: The four Haar butterflies. Each box displays the exact low and detail coefficient generated from one adjacent sample pair.

11.6 Step 5: Fourier analysis as a projector atlas

Let F_8 be the unitary discrete Fourier matrix,

$$(F_8)_{jk} = \frac{1}{\sqrt{8}} e^{-2\pi ijk/8}.$$

The coefficient vector is $\widehat{x} = F_8 x$, and

$$\boxed{x = F_8^* \widehat{x}.}$$

Now group frequency coordinates into

$$\{0\}, \quad \{1,7\}, \quad \{2,6\}, \quad \{3,5\}, \quad \{4\}.$$

If E_B selects one group, then

$$Q_B = F_8^* E_B F_8$$

is an exact projector in signal space. The five Q_B are pairwise annihilating and sum to I_8.

The first and Nyquist coefficients are especially simple:

$$\widehat{x}_0 = \frac{31}{\sqrt{8}} = \frac{31\sqrt{2}}{4}, \qquad \widehat{x}_4 = -\frac{3}{\sqrt{8}} = -\frac{3\sqrt{2}}{4}.$$

The conjugate symmetry $\widehat{x}_{8-k} = \overline{\widehat{x}_k}$ follows because x is real. The exact band energies are

B	$\|E_B \widehat{x}\|_2^2$
$\{0\}$	$961/8$
$\{1,7\}$	$97/4 - 5\sqrt{2}$
$\{2,6\}$	$13/4$
$\{3,5\}$	$97/4 + 5\sqrt{2}$
$\{4\}$	$9/8$

and their sum is $173 = \|x\|_2^2$.

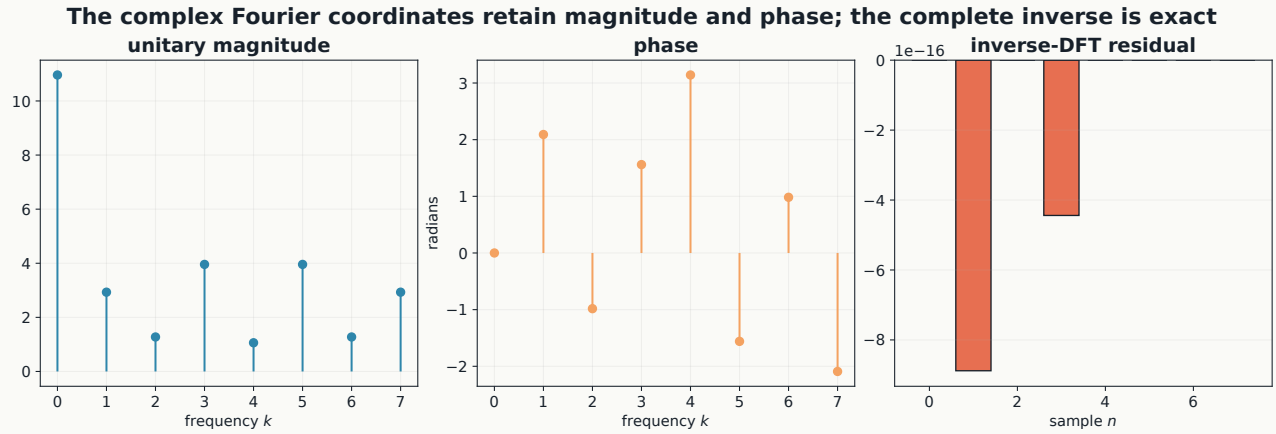


Figure 11.4: Magnitude and phase of all eight Fourier coefficients, together with the numerical inverse-transform residual. Both magnitude and phase are required for reconstruction.

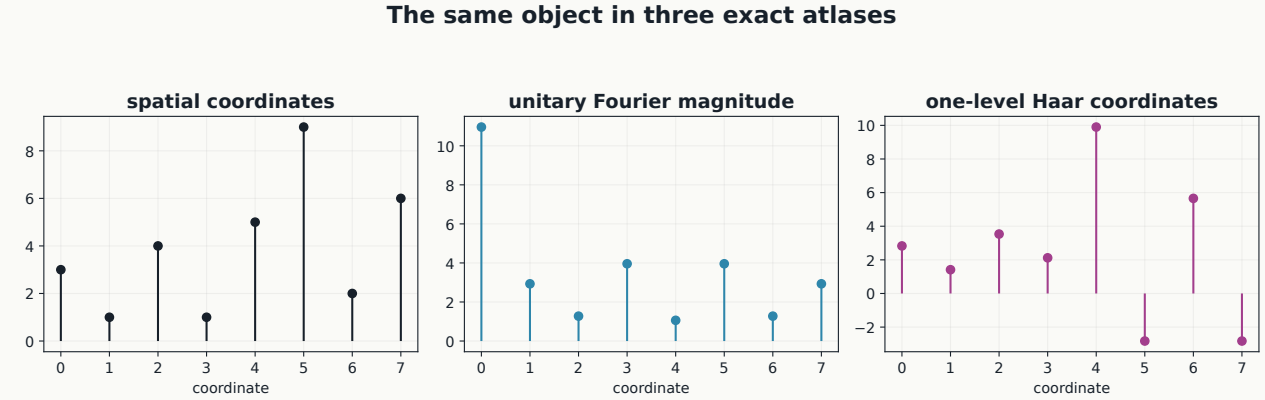


Figure 11.5: The same signal in spatial, Fourier, and Haar coordinates. Complete coordinates in each atlas reconstruct the same object.

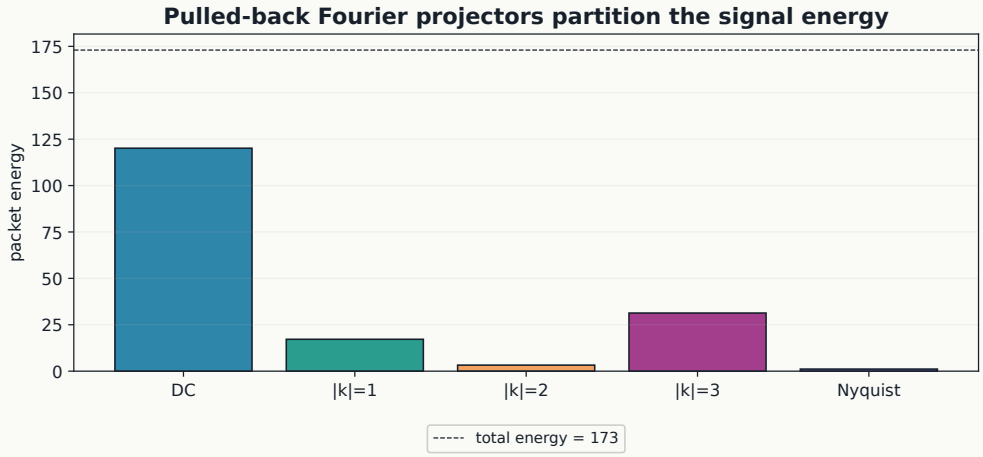


Figure 11.6: The pulled-back Fourier packets partition the squared norm of the signal. The legend is placed below the plot so no data are obscured.

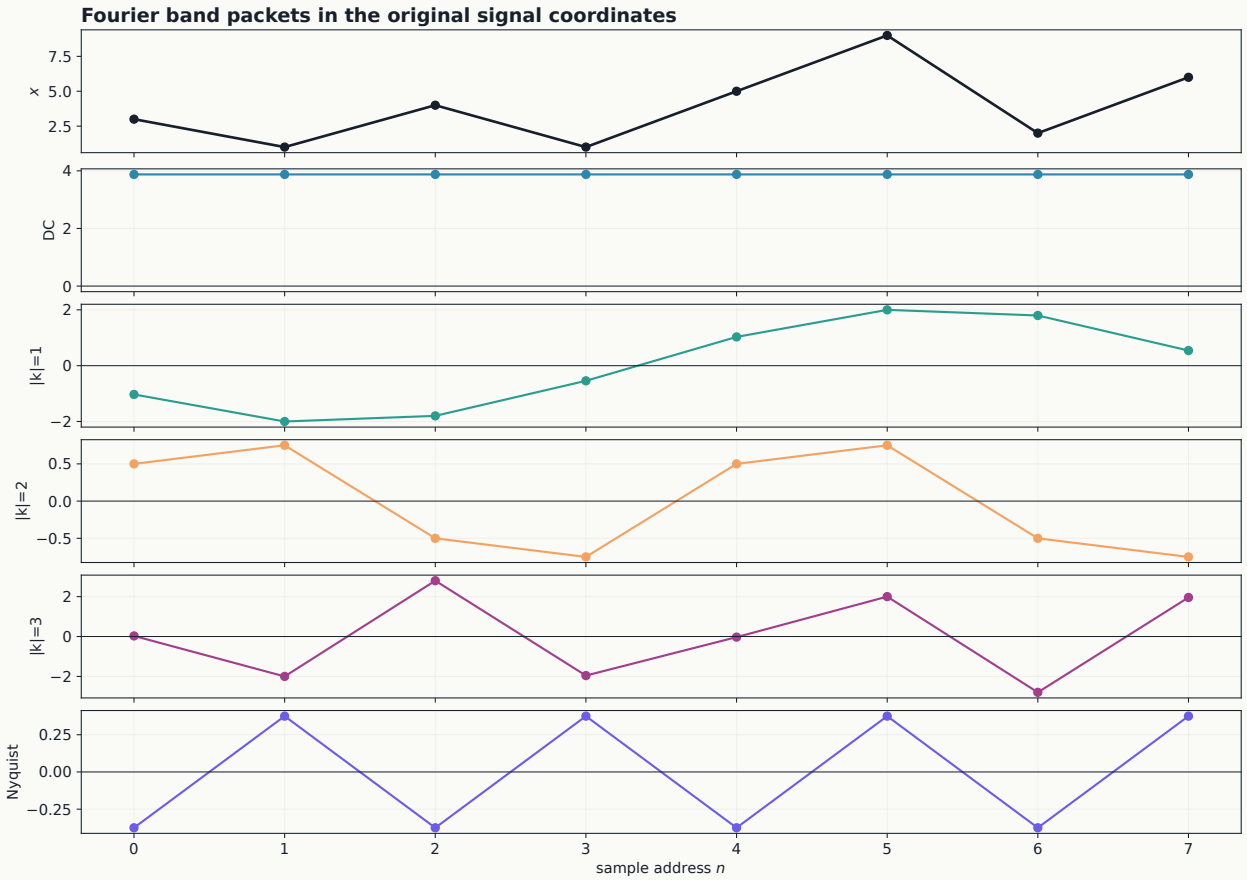


Figure 11.7: Each frequency selector is pulled back to the original sample coordinates. The five displayed packet signals add pointwise to the original signal.

Independent audit for the transform portfolio

The polyphase, Haar, and Fourier constructions are checked by different inverses:

- polyphase synthesis uses address interleaving;
- Haar synthesis uses the transpose of an orthogonal block matrix;
- Fourier synthesis uses the conjugate transpose of a unitary complex matrix.

Agreement of all three reconstructions is therefore not one repeated implementation check. It is three algebraically distinct inverse calculations on the same object.

Exact certificate

SymPy verifies exactly

$$\mathcal{S}_4\mathcal{A}_4x - x = 0,$$
$$P_{2,0} - P_{4,0} - P_{4,2} = 0, \qquad P_{2,1} - P_{4,1} - P_{4,3} = 0,$$
$$H^{\mathsf{T}}H - I_8 = 0, \qquad H^{\mathsf{T}}Hx - x = 0,$$
$$F_8^*F_8 - I_8 = 0, \qquad F_8^*F_8x - x = 0,$$
$$\sum_B Q_B - I_8 = 0, \qquad Q_B^2 - Q_B = 0 \text{ for every band } B.$$

The signal energy is

$$\|x\|_2^2 = 3^2 + 1^2 + 4^2 + 1^2 + 5^2 + 9^2 + 2^2 + 6^2 = 173.$$

11.7 Selected symbolic implementation

```
phase4 = polyphase_analyze(x, 4)
polyphase_residual = polyphase_synthesize(phase4, 8) - x

H = haar_one_level_matrix(8)
haar_residual = simplify(H.T * H * x - x)

F = dft8_matrix()
fourier_residual = simplify(F.conjugate().T * F * x - x)
Q = transform_projectors(F, ((0,), (1,7), (2,6), (3,5), (4,)))
```

Chapter 12

Area III: a quantum circuit as packet paths

12.1 Problem

Prepare a Bell state from one seed, decompose the circuit into parity packet paths, reconstruct the state, and certify entanglement by an exact reduced density matrix.

12.2 Step 1: Hilbert space and parity packets

Use the computational basis

$$|00\rangle, |01\rangle, |10\rangle, |11\rangle$$

of $\mathbb{C}^2 \otimes \mathbb{C}^2$. Define

$$P_{\mathrm{e}} = |00\rangle\langle 00| + |11\rangle\langle 11|,$$
$$P_{\mathrm{o}} = |01\rangle\langle 01| + |10\rangle\langle 10|.$$

Then

$$P_{\mathrm{e}} + P_{\mathrm{o}} = I_4, \qquad P_{\mathrm{e}} P_{\mathrm{o}} = 0.$$

12.3 Step 2: the circuit

Begin with

$$\psi_0 = |00\rangle.$$

Apply a Hadamard gate to the first qubit:

$$\psi_1 = (H \otimes I)|00\rangle = \frac{|00\rangle + |10\rangle}{\sqrt{2}}.$$

Apply CNOT:

$$\psi_2 = \mathrm{CNOT}\ \psi_1 = \frac{|00\rangle + |11\rangle}{\sqrt{2}} =: |\Phi^+\rangle.$$

In the ordered computational basis, the two circuit matrices are

$$H \otimes I = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & -1 & 0 \\ 0 & 1 & 0 & -1 \end{pmatrix}, \qquad \mathrm{CNOT} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}.$$

Multiplying by the seed vector gives

$$\begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \end{pmatrix} \xmapsto{H \otimes I} \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 0 \\ 1 \\ 0 \end{pmatrix} \xmapsto{\mathrm{CNOT}} \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 0 \\ 0 \\ 1 \end{pmatrix}.$$

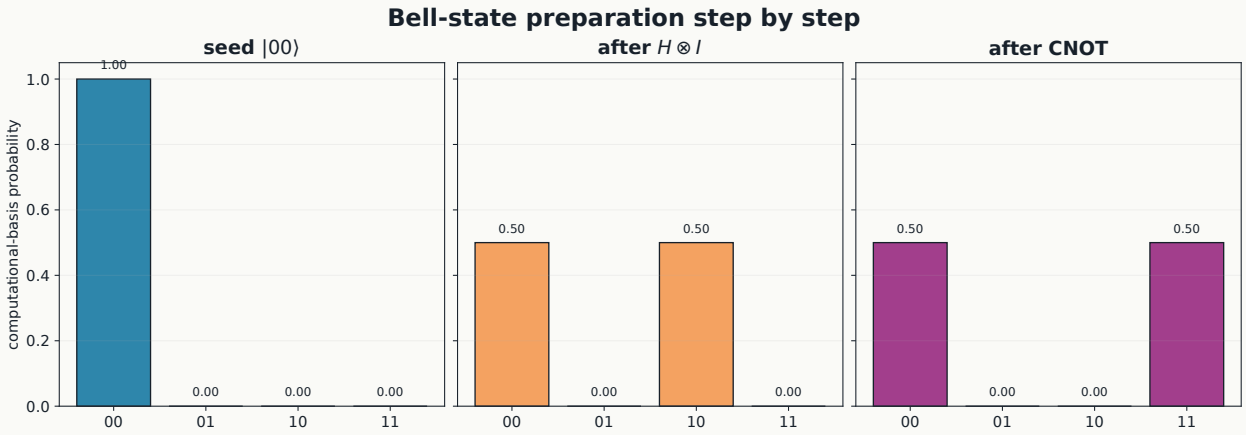


Figure 12.1: Computational-basis probabilities at the seed, intermediate, and final circuit stages. The Hadamard creates two branches; CNOT moves the odd intermediate branch into the even output packet.

12.4 Step 3: packet path expansion

The intermediate state has one even and one odd contribution. The only nonzero paths are

$$P_{\mathrm{e}}\,\mathrm{CNOT}\,P_{\mathrm{e}}(H \otimes I)P_{\mathrm{e}}|00\rangle = \frac{|00\rangle}{\sqrt{2}},$$

and

$$P_{\mathrm{e}}\,\mathrm{CNOT}\,P_{\mathrm{o}}(H \otimes I)P_{\mathrm{e}}|00\rangle = \frac{|11\rangle}{\sqrt{2}}.$$

Their sum is $|\Phi^+\rangle$. Every output-odd path vanishes.

All four parity routes can be listed explicitly:

intermediate parity	output parity	path vector
e	e	$(1,0,0,0)^{\mathsf{T}}/\sqrt{2}$
o	e	$(0,0,0,1)^{\mathsf{T}}/\sqrt{2}$
e	o	0
o	o	0

The path sum is coherent: vectors are added before probabilities are formed. In this example the two surviving outputs are orthogonal basis states, so the final computational-basis probabilities are 1/2 at 00 and 1/2 at 11.

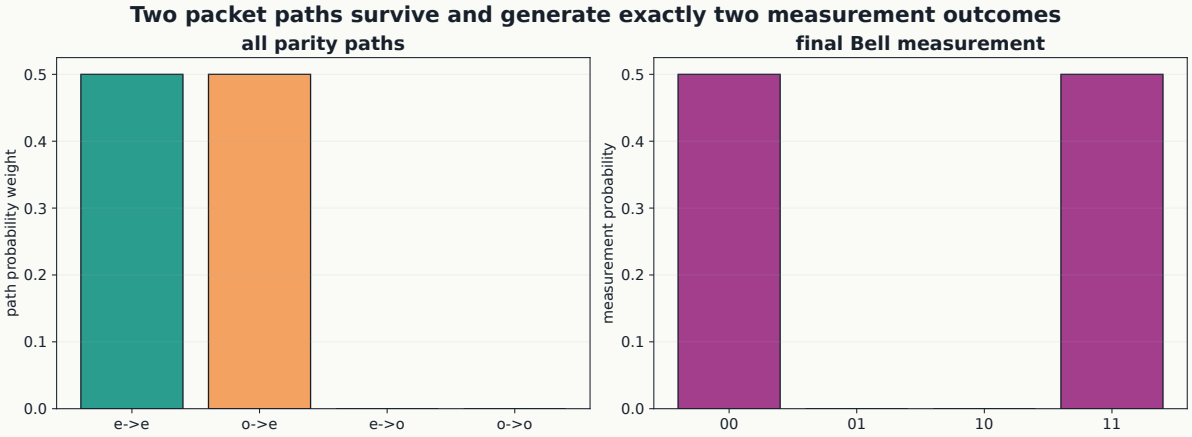


Figure 12.2: Left: squared norms of all four parity paths. Right: the final measurement distribution. The two zero paths are shown rather than omitted.

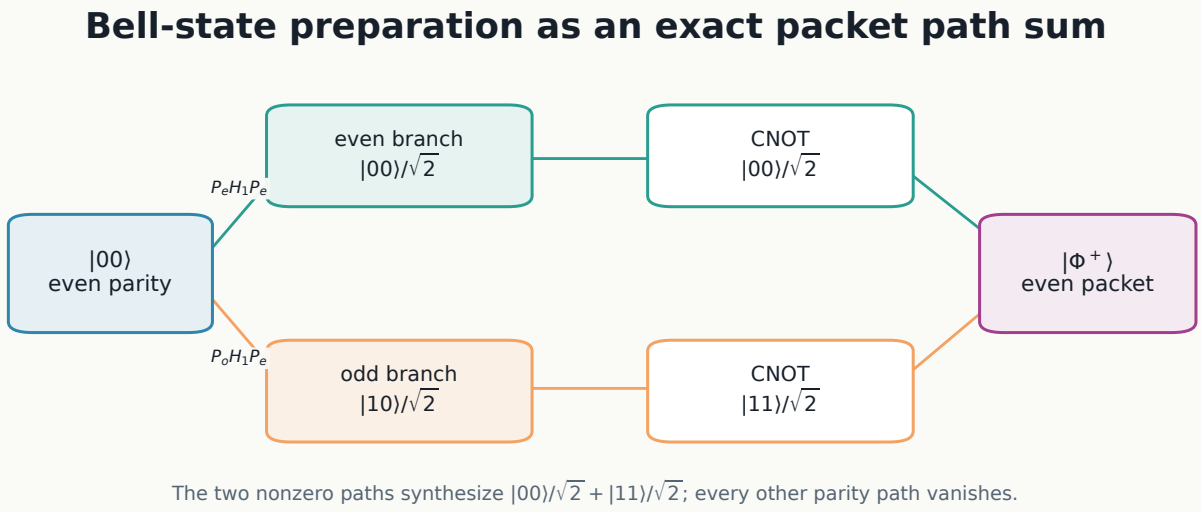


Figure 12.3: The Bell circuit resolved into the two nonzero parity paths.

12.5 Step 4: density matrix and partial trace

The pure-state density matrix is

$$\rho = |\Phi^+\rangle\langle\Phi^+| = \frac{1}{2}\begin{pmatrix} 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 \end{pmatrix}.$$

Tracing out the second qubit gives

$$\rho_A = \mathrm{Tr}_B(\rho) = \frac{1}{2} I_2.$$

Hence the first qubit is maximally mixed even though the two-qubit state is pure. This is an exact entanglement certificate.

The eigenvalues of ρ_A are $(1/2, 1/2)$, so its von Neumann entropy is

$$S(\rho_A) = -\operatorname{Tr}(\rho_A \log_2 \rho_A) = -2\left(\frac{1}{2}\log_2 \frac{1}{2}\right) = 1.$$

A pure bipartite state with a mixed reduced state cannot factor as $|u\rangle \otimes |v\rangle$. Thus the partial-trace calculation certifies one full ebit of entanglement.

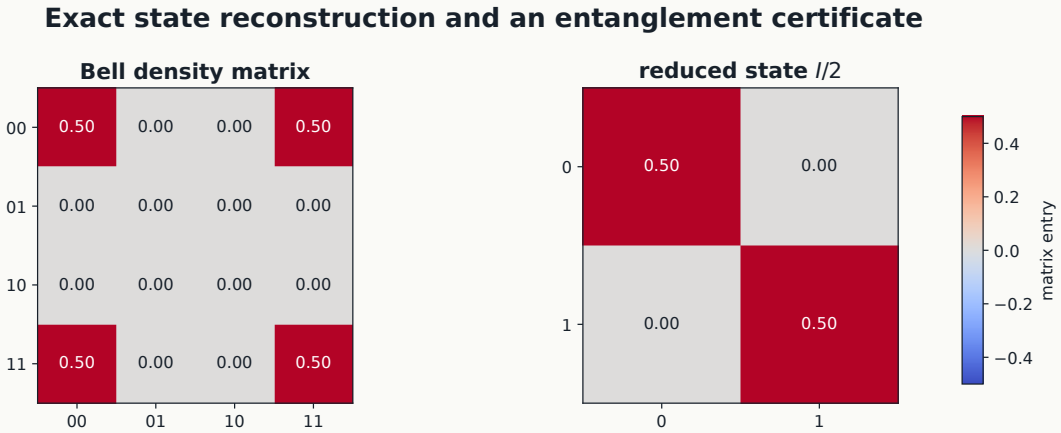


Figure 12.4: The Bell density matrix and its reduced one-qubit state.

Exact certificate

The symbolic computation returns

$$\begin{aligned}
&\|\psi_2\|^2 - 1 = 0,\\
&P_{\mathrm{o}}\psi_2 = 0,\\
&(P_{\mathrm{e}}\,\mathrm{CNOT}\,P_{\mathrm{e}}H_1P_{\mathrm{e}} + P_{\mathrm{e}}\,\mathrm{CNOT}\,P_{\mathrm{o}}H_1P_{\mathrm{e}})\,|00\rangle - \psi_2 = 0,\\
&\operatorname{Tr}_B(\rho) - \frac{1}{2}I_2 = 0.
\end{aligned}$$

12.6 Selected symbolic implementation

```
psi1 = (H \otimes I2) * psi0
bell = CNOT * psi1

path_even = Pe * CNOT * Pe * H1 * Pe * psi0
path_odd  = Pe * CNOT * Po * H1 * Pe * psi0
path_residual = path_even + path_odd - bell

rho = bell * bell.conjugate().T
rho_A = partial_trace_second_qubit(rho)
```

Chapter 13

Area IV: stochastic projectors and Bayesian refinement

13.1 Problem

Show that conditional expectation is an exact projector, prove when Bayesian conditioning commutes with packet coarsening, and factor the failure exactly when the likelihood varies inside a packet.

13.2 Step 1: a conditional-expectation projector

Use the finite probability measure

$$\mu = \left(\frac{1}{10}, \frac{2}{10}, \frac{3}{10}, \frac{4}{10}\right)$$

and the partition

$$A = \{0, 1\}, \qquad B = \{2, 3\}.$$

Conditional expectation onto packet-constant observables is represented by

$$E_\pi = \begin{pmatrix} \frac{1}{3} & \frac{2}{3} & 0 & 0 \\ \frac{1}{3} & \frac{2}{3} & 0 & 0 \\ 0 & 0 & \frac{3}{7} & \frac{4}{7} \\ 0 & 0 & \frac{3}{7} & \frac{4}{7} \end{pmatrix}.$$

For

$$f = (1, 4, -2, 3)^{\mathsf{T}},$$

we obtain

$$E_\pi f = \left(3, 3, \frac{6}{7}, \frac{6}{7}\right)^{\mathsf{T}}.$$

Direct multiplication proves

$$E_\pi^2 = E_\pi.$$

If $M = \mathrm{diag}(\mu)$, then

$$M E_\pi = E_\pi^{\mathsf{T}} M,$$

so E_π is self-adjoint for the weighted inner product $\langle u, v \rangle_\mu = u^{\mathsf{T}} M v$.

The residual is

$$r = f - E_\pi f = \left(-2, 1, -\frac{20}{7}, \frac{15}{7}\right)^\mathsf{T}.$$

Let $\mathbf{1}_A = (1,1,0,0)^\mathsf{T}$ and $\mathbf{1}_B = (0,0,1,1)^\mathsf{T}$. Direct calculation gives

$$\langle r, \mathbf{1}_A \rangle_\mu = \frac{1}{10}(-2) + \frac{2}{10}(1) = 0,$$

$$\langle r, \mathbf{1}_B \rangle_\mu = \frac{3}{10}\left(-\frac{20}{7}\right) + \frac{4}{10}\left(\frac{15}{7}\right) = 0.$$

Since packet-constant observables are spanned by $\mathbf{1}_A$ and $\mathbf{1}_B$, the residual is orthogonal to the entire target subspace.

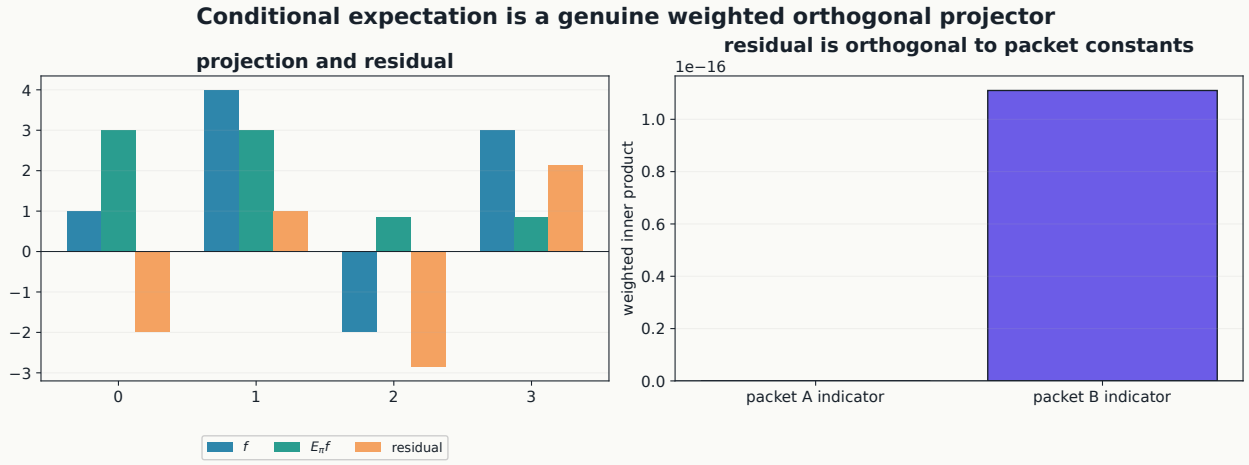


Figure 13.1: The observable, its conditional expectation, and the residual. The right panel displays the two weighted inner products that vanish exactly.

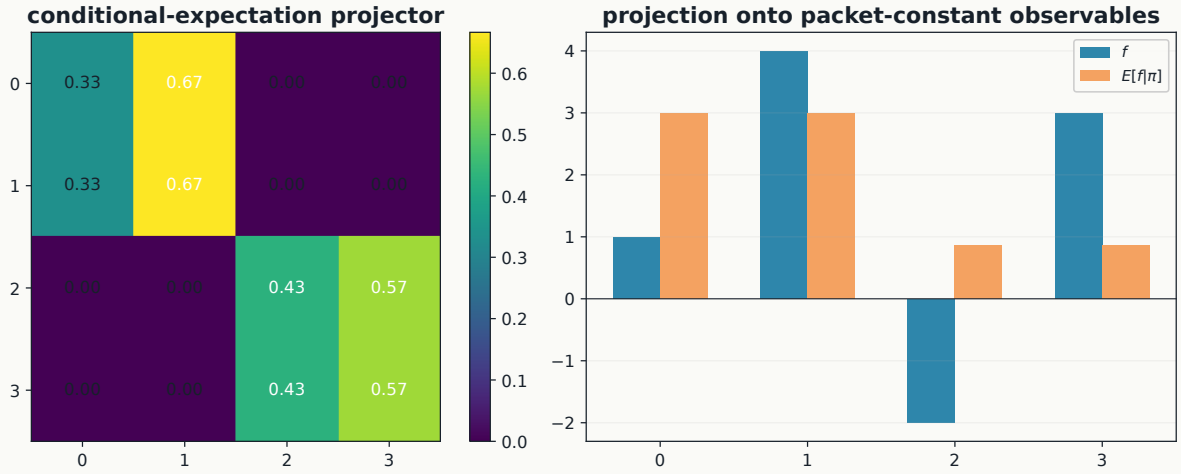


Figure 13.2: The finite conditional-expectation projector and its action on one observable.

13.3 Step 2: Bayesian conditioning

For prior p and positive likelihood λ, define

$$\mathcal{B}_\lambda(p)_i = \frac{\lambda_i p_i}{\sum_j \lambda_j p_j}.$$

Let $C_\pi p = (P_A, P_B)$ denote coarse packet masses.

Theorem 13.1 (Exact effective coarse likelihood)**.** *For every prior with positive packet masses, define*

$$r_a(p) = \frac{\sum_{i \in I_a} \lambda_i p_i}{P_a}.$$

Then

$$\boxed{C_\pi \mathcal{B}_\lambda(p) = \mathcal{B}_{r(p)}(C_\pi p).}$$

Proof. The ath coarse posterior mass is

$$\sum_{i \in I_a} \frac{\lambda_i p_i}{\sum_j \lambda_j p_j} = \frac{P_a r_a(p)}{\sum_b P_b r_b(p)},$$

which is exactly the coarse Bayesian update with likelihood $r(p)$. □

Theorem 13.2 (Finite sufficiency criterion)**.** *Assume at least two coarse packets can carry positive prior mass. A fixed coarse likelihood $\bar{\lambda}$ satisfies*

$$C_\pi \mathcal{B}_\lambda(p) = \mathcal{B}_{\bar{\lambda}}(C_\pi p)$$

for every strictly positive fine prior p if and only if there is one scalar $c > 0$ such that

$$\lambda_i = c\,\bar{\lambda}_a \qquad (i \in I_a).$$

Equivalently, after removal of one globally invisible scale, the likelihood must be constant inside every packet.

Proof. The stated condition is sufficient because then $r_a(p) = c\bar{\lambda}_a$ for every prior, and the common scale cancels in Bayesian normalization. Conversely, fix the coarse packet masses and vary the conditional distribution inside one packet. If two fine likelihood values in that packet differ, its weighted average $r_a(p)$ changes while the coarse prior stays fixed. Placing positive mass on another packet makes this change visible after normalization, contradicting prior-independent coarse closure. □

13.4 Step 3: the sufficient case

Take

$$\lambda = (2, 2, 1, 1)^\mathsf{T}.$$

The likelihood is constant inside each packet, so $r = (2, 1)$ independently of the hidden fine prior. The fine posterior is

$$\mathcal{B}_\lambda(\mu) = \left(\frac{2}{13}, \frac{4}{13}, \frac{3}{13}, \frac{4}{13}\right)^\mathsf{T}.$$

Coarsening gives

$$\left(\frac{6}{13}, \frac{7}{13}\right)^\mathsf{T},$$

which agrees exactly with the coarse update of $(3/10, 7/10)$ by likelihood $(2, 1)$.

13.5 Step 4: the nonsufficient case

Now take

$$\lambda = (1, 3, 2, 4)^{\mathsf{T}}.$$

The effective packet likelihoods are

$$r_A = \frac{7}{3}, \qquad r_B = \frac{22}{7}.$$

Therefore the exact coarse posterior is

$$C_\pi \mathcal{B}_\lambda(\mu) = \left(\frac{7}{29}, \frac{22}{29}\right)^{\mathsf{T}}.$$

If one incorrectly replaces the hidden likelihood structure by the nominal coarse likelihood

$$\bar{\lambda} = (2, 3)^{\mathsf{T}},$$

then the nominal posterior is

$$\mathcal{B}_{\bar{\lambda}}(C_\pi \mu) = \left(\frac{2}{9}, \frac{7}{9}\right)^{\mathsf{T}}.$$

The defect is

$$\boxed{\left(\frac{7}{29}, \frac{22}{29}\right) - \left(\frac{2}{9}, \frac{7}{9}\right) = \left(\frac{5}{261}, -\frac{5}{261}\right).}$$

The necessity of the sufficiency criterion can also be seen with two priors having exactly the same coarse packet masses:

$$p^{(1)} = \left(\frac{1}{10}, \frac{2}{10}, \frac{3}{10}, \frac{4}{10}\right), \qquad p^{(2)} = \left(\frac{2}{10}, \frac{1}{10}, \frac{4}{10}, \frac{3}{10}\right).$$

Both coarsen to $(3/10, 7/10)$, but the same fine likelihood $(1, 3, 2, 4)$ produces

$$C_\pi \mathcal{B}_\lambda(p^{(1)}) = \left(\frac{7}{29}, \frac{22}{29}\right), \qquad C_\pi \mathcal{B}_\lambda(p^{(2)}) = \left(\frac{1}{5}, \frac{4}{5}\right).$$

Their difference is

$$\left(\frac{6}{145}, -\frac{6}{145}\right) \neq 0.$$

Therefore the coarse prior alone cannot determine the coarse posterior when within-packet likelihoods differ.

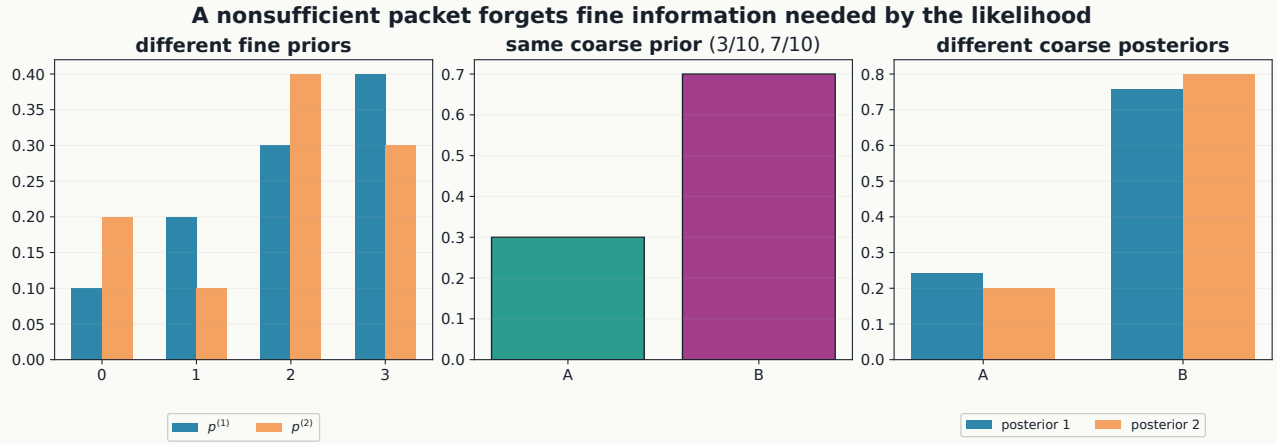


Figure 13.3: Two different fine priors have the same coarse masses but produce different coarse posteriors. This is a direct finite witness of information loss under nonsufficient coarsening.

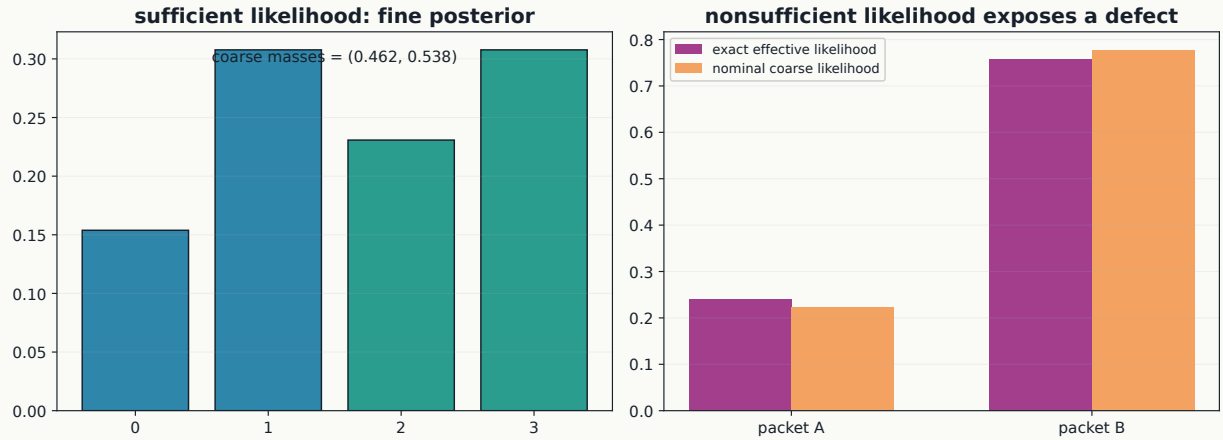


Figure 13.4: Sufficient packet conditioning and a nonsufficient negative control.

13.6 Step 5: exact information-loss factorization

Let

$$P = C_\pi p, \qquad \delta = r(p) - \bar{\lambda}, \qquad \bar{b} = \mathcal{B}_{\bar{\lambda}}(P), \qquad Z_r = \langle r(p), P \rangle .$$

Then direct algebra gives

$$\boxed{C_\pi \mathcal{B}_\lambda(p) - \mathcal{B}_{\bar{\lambda}}(P) = \frac{1}{Z_r}\left(\operatorname{diag}(P) - \bar{b}P^{\mathsf{T}}\right)\delta.}$$

The defect is therefore carried entirely by the within-packet likelihood variation δ.

For the numerical example,

$$P = \begin{pmatrix} 3/10 \\ 7/10 \end{pmatrix}, \quad r = \begin{pmatrix} 7/3 \\ 22/7 \end{pmatrix}, \quad \bar{\lambda} = \begin{pmatrix} 2 \\ 3 \end{pmatrix}, \quad \delta = \begin{pmatrix} 1/3 \\ 1/7 \end{pmatrix},$$

$$Z_r = P^{\mathsf{T}} r = \frac{29}{10}, \qquad \bar{b} = \begin{pmatrix} 2/9 \\ 7/9 \end{pmatrix},$$

and

$$\operatorname{diag}(P) - \bar{b}P^{\mathsf{T}} = \begin{pmatrix} 7/30 & -7/45 \\ -7/30 & 7/45 \end{pmatrix}.$$

Multiplication gives

$$\frac{1}{Z_r}\left(\operatorname{diag}(P) - \bar{b}P^{\mathsf{T}}\right)\delta = \begin{pmatrix} 5/261 \\ -5/261 \end{pmatrix},$$

exactly matching the directly computed defect.

Proof. Write $r = \bar{\lambda} + \delta$. Since

$$\mathcal{B}_r(P) = \frac{\operatorname{diag}(P) r}{Z_r},$$

and

$$\bar{b} = \frac{\operatorname{diag}(P)\bar{\lambda}}{Z_{\bar{\lambda}}},$$

subtract the two fractions, replace $Z_r - Z_{\bar{\lambda}}$ by $P^{\mathsf{T}}\delta$, and collect the δ terms. The displayed factorization follows. □

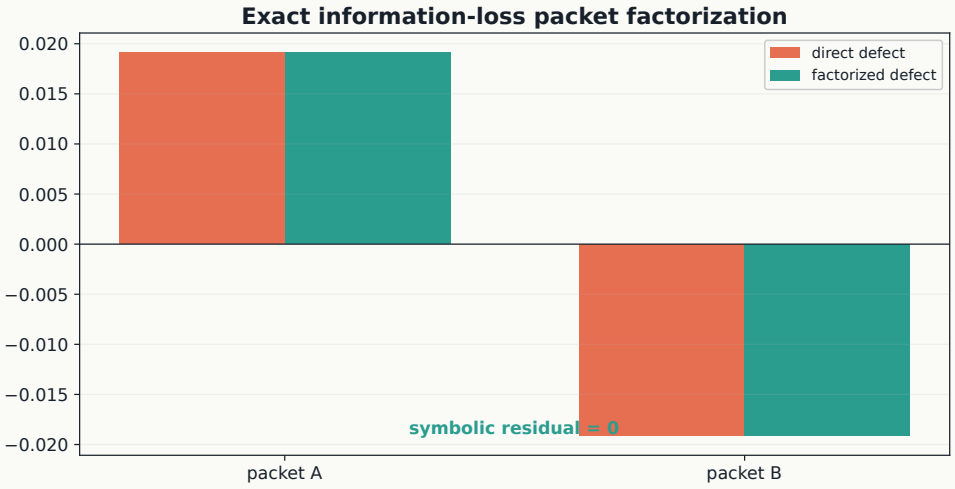


Figure 13.5: The direct Bayesian defect and the factorized information-loss packet agree exactly.

Exact certificate

The exact residuals are

$$E_\pi^2 - E_\pi = 0, \qquad M E_\pi - E_\pi^{\mathsf{T}} M = 0,$$

$$C_\pi \mathcal{B}_{(2,2,1,1)}(\mu) - \mathcal{B}_{(2,1)}(C_\pi \mu) = 0,$$

and

$$[C_\pi \mathcal{B}_\lambda(\mu) - \mathcal{B}_{\bar{\lambda}}(C_\pi \mu)] - \frac{\operatorname{diag}(P) - \bar{b} P^{\mathsf{T}}}{Z_r} \delta = 0.$$

Chapter 14

Area V: a closed root-filter family and a candidate integer sequence

14.1 Problem

Generate an exact integer sequence from a projector-resolved analytic seed, derive its root-sum and hypergeometric closed forms, prove an integer recurrence, prove a second finite composition formula, and produce a candidate OEIS record.

14.2 Step 1: the signed order-five packet

Define

$$D_5(z) = \sum_{n\geq 0} (-1)^n \frac{z^{5n}}{(5n)!}.$$

This is the residue-zero packet of an exponential system whose fifth derivative returns the negative of the original function:

$$D_5^{(5)} = -D_5.$$

Let

$$\eta_k = \exp\left(\frac{(2k+1)\pi i}{5}\right), \qquad 0 \leq k < 5,$$

so that $\eta_k^5 = -1$. The root filter gives

$$\boxed{D_5(z) = \frac{1}{5}\sum_{k=0}^{4} e^{\eta_k z}.}$$

Indeed,

$$\frac{1}{5}\sum_{k=0}^{4} e^{\eta_k z} = \sum_{m\geq 0}\left(\frac{1}{5}\sum_{k=0}^{4}\eta_k^m\right)\frac{z^m}{m!}.$$

The inner sum vanishes unless $5 \mid m$. For $m = 5n$ it equals

$$\frac{1}{5}\sum_{k=0}^{4}(\eta_k^5)^n = (-1)^n,$$

which selects exactly the coefficients of D_5.

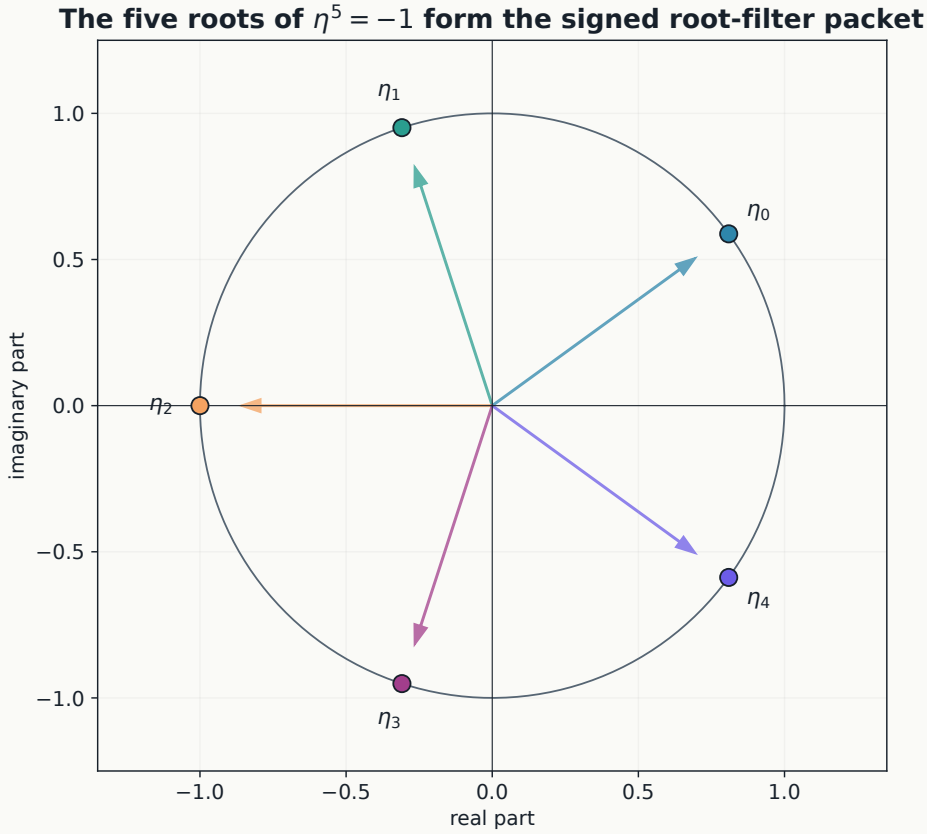


Figure 14.1: The five roots of $\eta^5 = -1$. Their character average annihilates every exponential coefficient outside degrees divisible by five.

Pairing conjugate roots yields the real closed form

$$\boxed{D_5(z) = \frac{1}{5}\left[e^{-z} + 2e^{az}\cos(bz) + 2e^{cz}\cos(dz)\right],}$$

where

$$a = \frac{1+\sqrt{5}}{4}, \quad b = \frac{\sqrt{10-2\sqrt{5}}}{4}, \quad c = \frac{1-\sqrt{5}}{4}, \quad d = \frac{\sqrt{10+2\sqrt{5}}}{4}.$$

The factorial multiplication formula also gives

$$\boxed{D_5(z) = {}_0F_4\left(\begin{matrix} - \\ \frac{1}{5}, \frac{2}{5}, \frac{3}{5}, \frac{4}{5} \end{matrix}; -\frac{z^5}{5^5}\right).}$$

To verify this term by term, use

$$(5n)! = 5^{5n} n! \left(\frac{1}{5}\right)_n \left(\frac{2}{5}\right)_n \left(\frac{3}{5}\right)_n \left(\frac{4}{5}\right)_n.$$

The nth hypergeometric term is therefore exactly $(-1)^n z^{5n}/(5n)!$.

14.3 Step 2: reciprocal square and sequence definition

Define

$$S(z) = D_5(z)^{-2} = \sum_{n\geq 0} a_n \frac{z^{5n}}{(5n)!}.$$

Using the closed form for D_5,

$$\boxed{S(z) = \frac{25}{\left[e^{-z} + 2e^{az}\cos(bz) + 2e^{cz}\cos(dz)\right]^2}.}$$

This is a genuine closed analytic formula for the whole sequence-generating function.

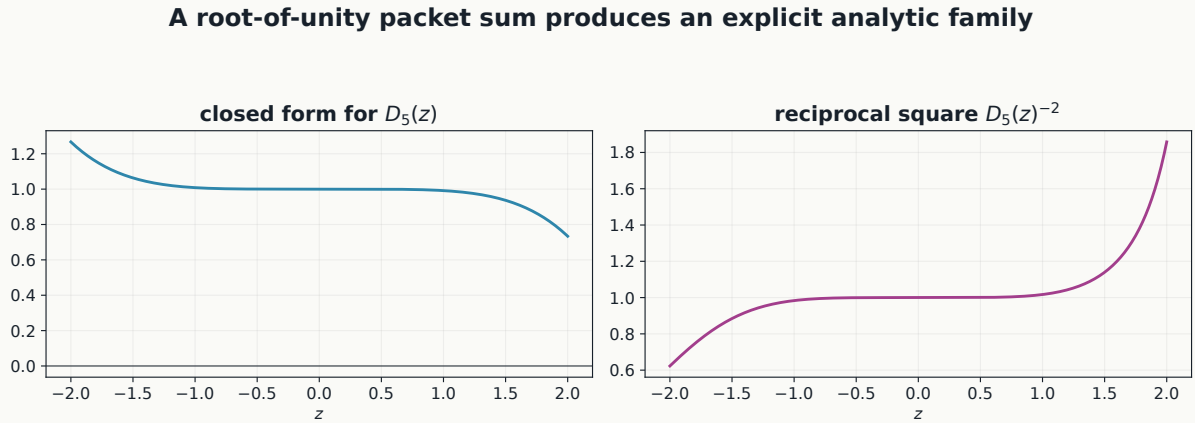


Figure 14.2: The signed root-filter denominator and its reciprocal square.

14.4 Step 3: an exact integer recurrence

Logarithmic differentiation of $S = D_5^{-2}$ gives

$$D_5 S' + 2D_5' S = 0.$$

Extracting the coefficient of $z^{5n-1}/(5n-1)!$ gives, for $n \geq 1$,

$$\boxed{\begin{aligned} a_n = &-\sum_{k=1}^{n-1} (-1)^k \binom{5n-1}{5k} a_{n-k} \\ &- 2\sum_{k=1}^{n} (-1)^k \binom{5n-1}{5k-1} a_{n-k}, \qquad a_0 = 1. \end{aligned}}$$

Every coefficient on the right is an integer. Induction proves

$$\boxed{a_n \in \mathbb{Z} \quad (n \geq 0).}$$

The first four nontrivial rows show how the recurrence works:

n	first convolution contribution	derivative contribution	a_n
1	0	2	2
2	252	502	754
3	1,507,506	1,501,502	3,009,008
4	34,919,099,764	23,186,570,502	58,105,670,266

Each row is an exact integer identity, not a rounded numerical evaluation.

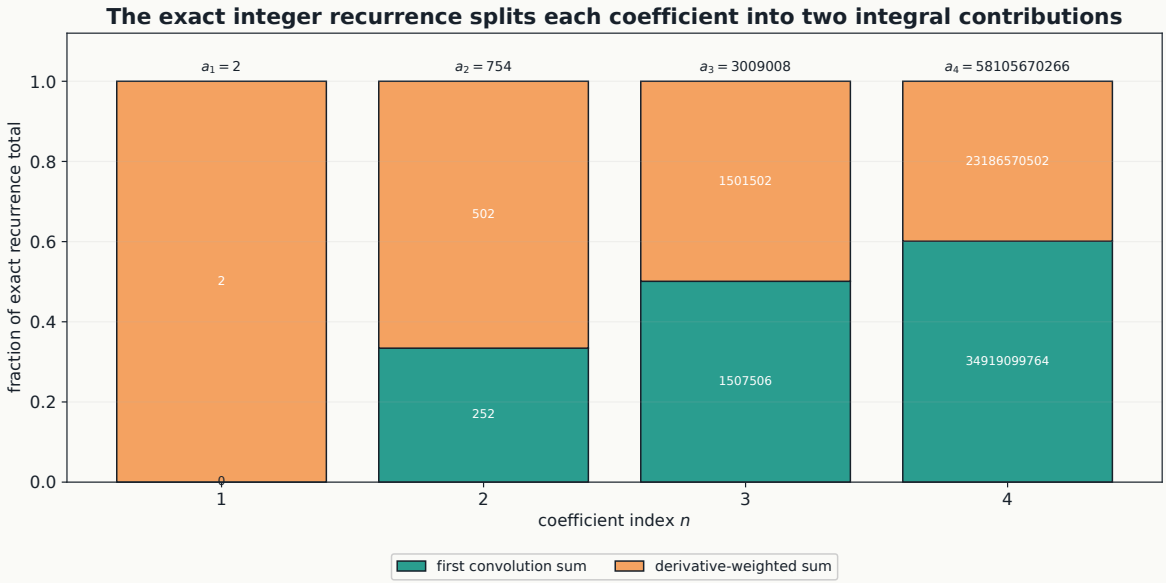


Figure 14.3: The two integral contributions to the first four nontrivial recurrence rows. Heights are normalized by the exact total so that both contributions remain visible; exact integers are printed inside the bars.

The first terms are

$$1,\ 2,\ 754,\ 3009008,\ 58105670266,\ 3690819041733782,$$
$$611297487195595912564,$$
$$225925381294863591587930378,$$
$$16659578502013406448739481791 4506, \ldots$$

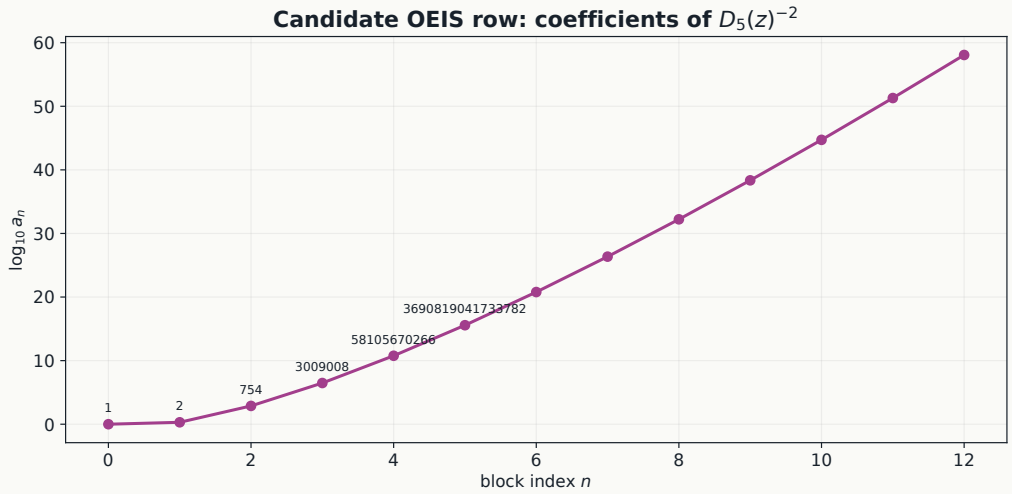


Figure 14.4: Growth of the candidate sequence on a base-ten logarithmic scale. Only the first few terms are annotated to prevent label collisions.

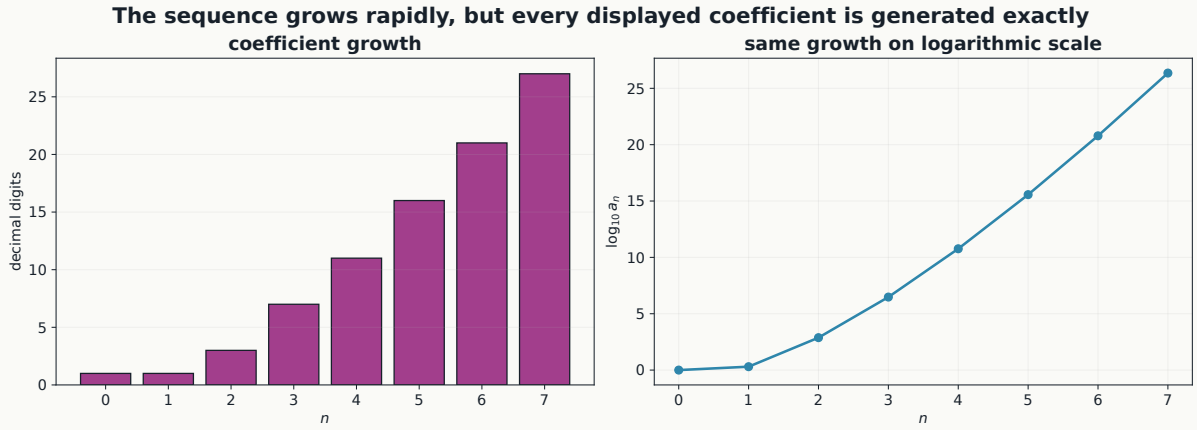


Figure 14.5: Two readable summaries of the same rapid growth: decimal digit count and base-ten logarithm.

14.5 Step 4: an independent finite coefficient formula

Write

$$D_5(z) = 1 + u(z), \qquad u(z) = \sum_{k\geq 1} (-1)^k \frac{z^{5k}}{(5k)!}.$$

Since

$$(1+u)^{-2} = \sum_{m\geq 0} (-1)^m (m+1) u^m,$$

the coefficient is

$$\boxed{a_n = (5n)! \sum_{m=1}^{n} (-1)^{n+m} (m+1) \sum_{\substack{k_1+\cdots+k_m=n \\ k_i\geq 1}} \prod_{i=1}^{m} \frac{1}{(5k_i)!}}$$

for $n \geq 1$, with $a_0 = 1$.

This formula and the recurrence arise from different algebraic operations. Their exact agreement through the computed range is therefore an independent internal check.

14.6 Step 5: the reciprocal-order polynomial family

More generally, define

$$D_5(z)^{-r} = \sum_{n\geq 0} P_{5,n}(r) \frac{z^{5n}}{(5n)!}.$$

The first parameter polynomials are

$$\begin{aligned}
P_{5,0}(r) &= 1, \\
P_{5,1}(r) &= r, \\
P_{5,2}(r) &= r(126r + 125), \\
P_{5,3}(r) &= r(126126r^2 + 375375r + 249250), \\
P_{5,4}(r) &= r(488864376r^3 + 2909907000r^2 \\
&\qquad + 5307778250r + 2886735625).
\end{aligned}$$

They satisfy the semigroup convolution

$$\boxed{P_{5,n}(r+s) = \sum_{q=0}^{n} \binom{5n}{5q} P_{5,q}(r) P_{5,n-q}(s).}$$

Our sequence is the row $a_n = P_{5,n}(2)$.

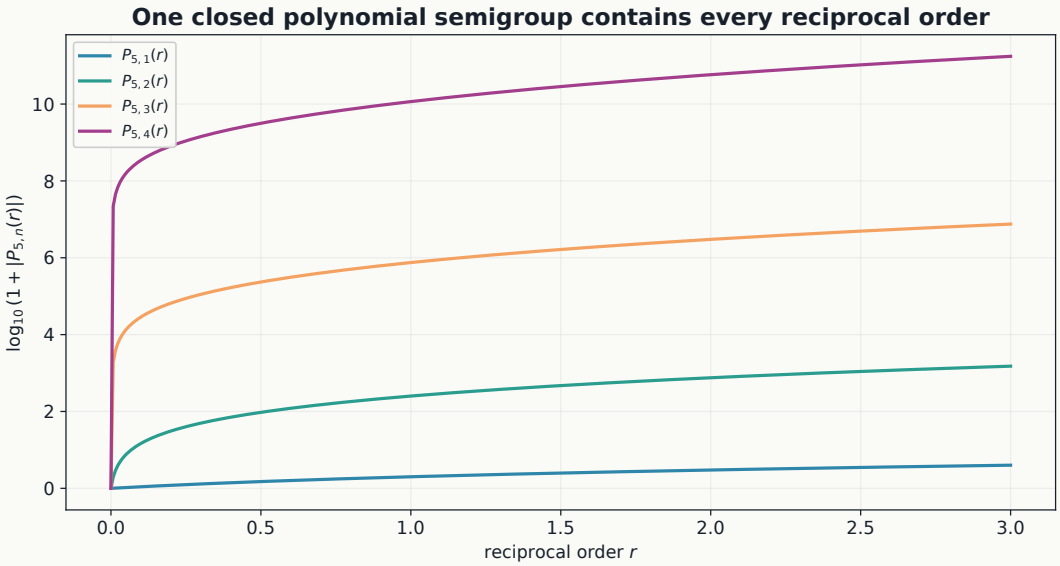


Figure 14.6: The first order-five reciprocal parameter polynomials.

14.7 OEIS-style candidate record

The sequence is supplied in an OEIS-ready form because its definition, recurrence, initial values, and b-file are exact. This document does not assert historical novelty or the absence of an equivalent transformed sequence. The mathematical claim proved here is narrower and complete:

$$\boxed{D_5(z)^{-2} = \sum_{n\geq 0} a_n \frac{z^{5n}}{(5n)!}, \qquad (a_n)_{n\geq 0} \subset \mathbb{Z},}$$

with the displayed recurrence and independent composition formula. Any database identifier would be metadata added after mathematical equivalence is established; it is not used in any proof.

Exact certificate

The symbolic package verifies:

(i) every recurrence residual through $n = 12$ is exactly zero;

(ii) multiplying the block series for D_5^2 by the candidate block series gives 1 through the same exact range;

(iii) the hypergeometric coefficient identity is exact term by term;

(iv) the composition formula equals the recurrence values for every computed term;

(v) every computed a_n is an integer.

14.8 Selected symbolic implementation

```
def transfer_secancy_coefficients(N, r, nmax):
    vals = [Integer(1)]
    for n in range(1, nmax + 1):
        first = -sum((-1)**k * binomial(N*n-1, N*k)
                     * vals[n-k] for k in range(1, n))
        second = -r * sum((-1)**k * binomial(N*n-1, N*k-1)
                          * vals[n-k] for k in range(1, n+1))
        vals.append(factor(expand(first + second)))
    return vals
```

Chapter 15

Area VI: a coupled-oscillator physics problem

15.1 Problem

Solve a two-mass coupled oscillator exactly, identify its normal-mode projectors, reconstruct the motion from the packets, and prove conservation and partition of energy.

15.2 Step 1: equations of motion

With unit masses and fixed-end nearest-neighbor springs, use

$$q''(t) + Kq(t) = 0, \qquad K = \begin{pmatrix} 2 & -1 \\ -1 & 2 \end{pmatrix}.$$

The characteristic polynomial is

$$\det(K - \lambda I) = (2 - \lambda)^2 - 1 = (\lambda - 1)(\lambda - 3).$$

Normalized eigenvectors are

$$e_s = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \qquad e_a = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -1 \end{pmatrix},$$

with

$$Ke_s = e_s, \qquad Ke_a = 3e_a.$$

The symmetric and antisymmetric projectors are therefore the rank-one spectral projectors

$$P_s = e_s e_s^{\mathsf{T}} = \frac{1}{2} \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}, \qquad P_a = e_a e_a^{\mathsf{T}} = \frac{1}{2} \begin{pmatrix} 1 & -1 \\ -1 & 1 \end{pmatrix}.$$

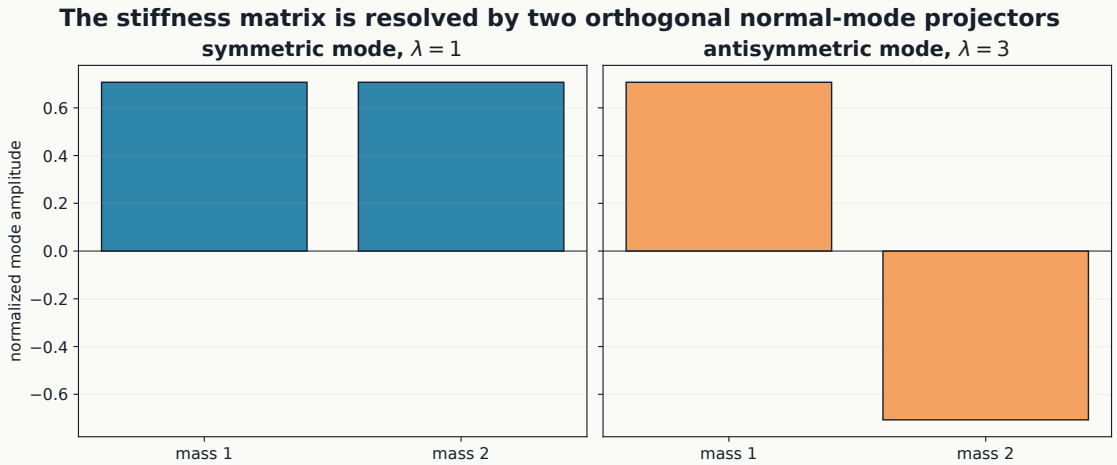


Figure 15.1: The two normalized eigenmodes of the stiffness matrix. Equal signs indicate in-phase motion; opposite signs indicate out-of-phase motion.

They satisfy

$$P_s P_a = 0, \qquad P_s + P_a = I_2.$$

Moreover,

$$\boxed{K = P_s + 3P_a.}$$

Thus the two packet frequencies are

$$\omega_s = 1, \qquad \omega_a = \sqrt{3}.$$

15.3 Step 2: exact modal solution

For initial position q_0 and velocity v_0,

$$\boxed{\begin{aligned} q(t) = {} & \cos(t) P_s q_0 + \sin(t) P_s v_0 \\ & + \cos(\sqrt{3}t) P_a q_0 + \frac{\sin(\sqrt{3}t)}{\sqrt{3}} P_a v_0. \end{aligned}}$$

Take

$$q_0 = (1, 0)^{\mathsf{T}}, \qquad v_0 = (0, 0)^{\mathsf{T}}.$$

The initial displacement packets are

$$P_s q_0 = \frac{1}{2}\begin{pmatrix} 1 \\ 1 \end{pmatrix}, \qquad P_a q_0 = \frac{1}{2}\begin{pmatrix} 1 \\ -1 \end{pmatrix}.$$

Thus half of the coordinate amplitude is initially in each mode, but the mode energies differ because the antisymmetric stiffness eigenvalue is three times larger.

Then

$$\boxed{q_1(t) = \frac{\cos t + \cos(\sqrt{3}t)}{2}, \qquad q_2(t) = \frac{\cos t - \cos(\sqrt{3}t)}{2}.}$$

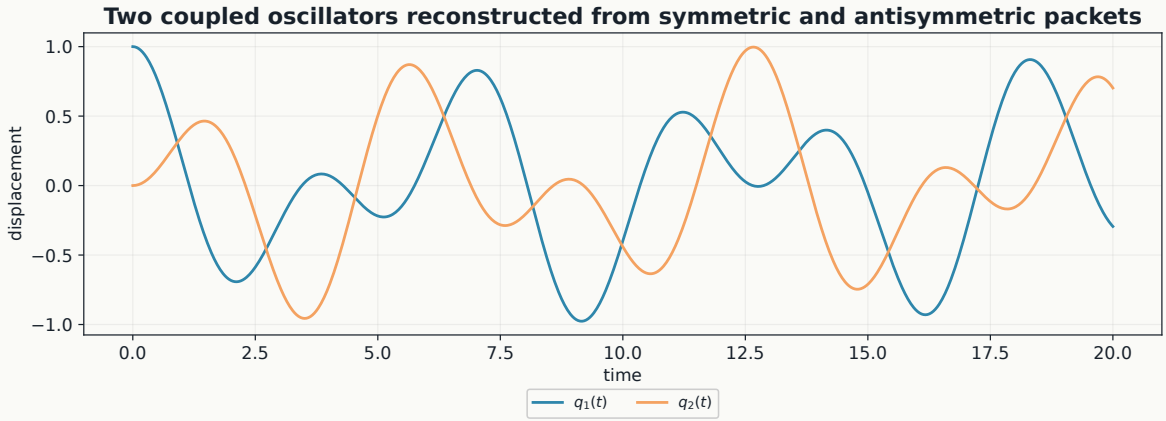


Figure 15.2: The physical coordinates reconstructed from the two normal-mode packets. The legend is placed below the axes so it cannot cover the trajectories.

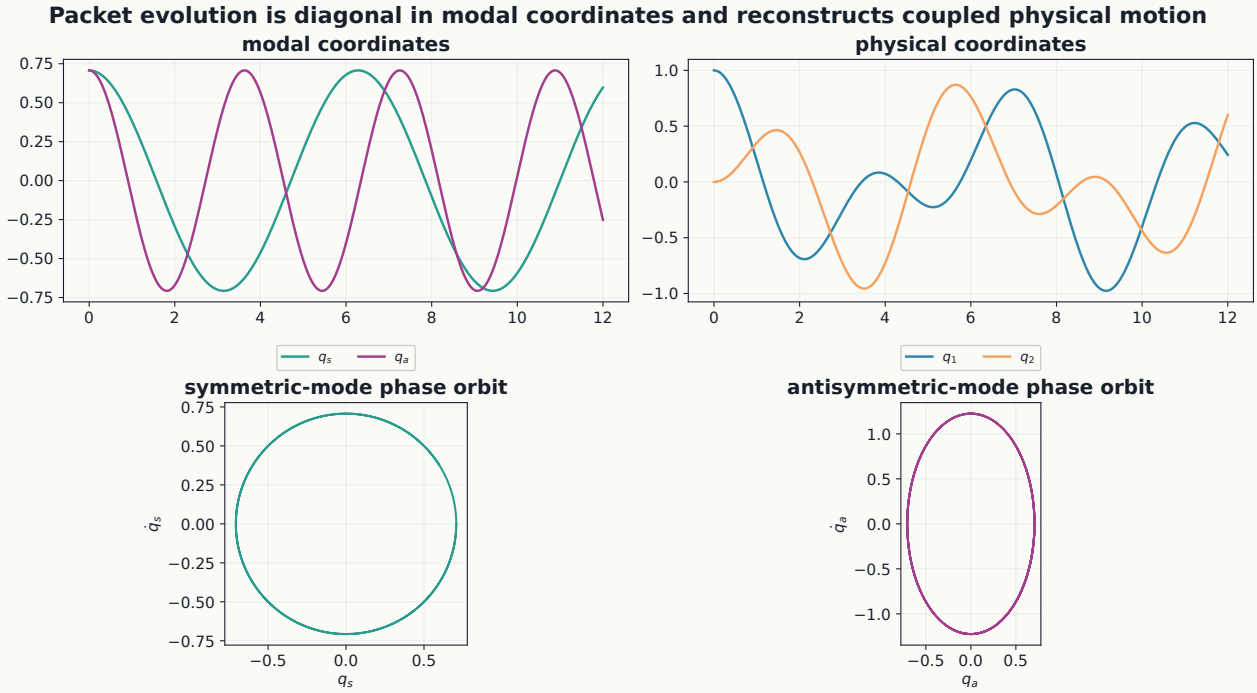


Figure 15.3: Top row: modal coordinates and reconstructed physical coordinates. Bottom row: the two closed phase-space orbits, each generated by one conserved modal energy.

15.4 Step 3: verify the equation

First verify the initial data. At $t = 0$,

$$q(0) = P_s q_0 + P_a q_0 = (P_s + P_a)q_0 = q_0,$$

and

$$q'(0) = P_s v_0 + P_a v_0 = v_0.$$

Now differentiate twice. Since

$$KP_s = P_s, \qquad KP_a = 3P_a,$$

we obtain

$$q''(t) = -P_s q(t) - 3P_a q(t) = -Kq(t).$$

The symbolic residual $q'' + Kq$ is the zero vector.

15.5 Step 4: energy packets

The total energy is

$$E(t) = \frac{1}{2}\left\|q'(t)\right\|^2 + \frac{1}{2}q(t)^{\mathsf{T}} K q(t).$$

Because the packet spaces are orthogonal and $K = P_s + 3P_a$,

$$E = E_s + E_a,$$

where

$$E_s = \frac{1}{2}\left\|P_s q'\right\|^2 + \frac{1}{2}\left\|P_s q\right\|^2,$$

and

$$E_a = \frac{1}{2}\left\|P_a q'\right\|^2 + \frac{3}{2}\left\|P_a q\right\|^2.$$

For the chosen initial condition,

$$E_s(0) = \frac{1}{2}\left\|P_s q_0\right\|^2 = \frac{1}{2}\left(\frac{1}{2}\right) = \frac{1}{4},$$

while

$$E_a(0) = \frac{3}{2}\left\|P_a q_0\right\|^2 = \frac{3}{2}\left(\frac{1}{2}\right) = \frac{3}{4}.$$

Therefore

$$\boxed{E_s = \frac{1}{4}, \qquad E_a = \frac{3}{4}, \qquad E = 1.}$$

All three quantities are independent of time. The total conservation law is not checked only at $t = 0$: symbolic differentiation and simplification prove $dE/dt = 0$ identically.

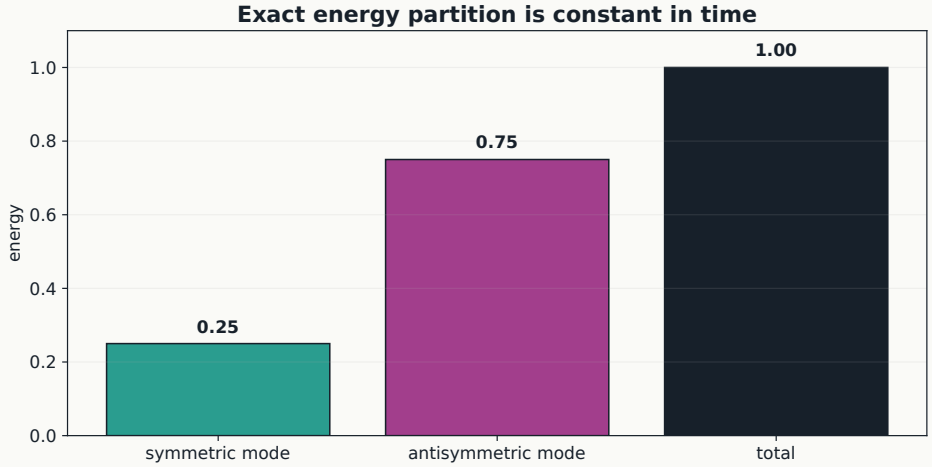


Figure 15.4: The symmetric and antisymmetric packet energies add to the conserved total.

Exact certificate

SymPy simplifies

$$P_s^2 - P_s = 0, \quad P_a^2 - P_a = 0, \quad P_s P_a = 0, \quad K - P_s - 3P_a = 0,$$

$$q''(t) + Kq(t) = 0, \quad E(t) - 1 = 0, \quad E_s - \frac{1}{4} = 0, \quad E_a - \frac{3}{4} = 0.$$

Chapter 16

Area VII: nonlinear packets and geometric holonomy

16.1 Part A: nonlinear signal packets

Consider the periodic memory-one nonlinear filter

$$y[n] = x[n] + \frac{1}{2}x[n-1]^2.$$

Use the input

$$x = (1, 2, 0, -1, 3, 1, -2, 2)^{\mathsf{T}}.$$

Define even and odd channels

$$e[k] = x[2k], \qquad o[k] = x[2k+1].$$

For the chosen signal,

$$e = (1, 0, 3, -2)^{\mathsf{T}}, \qquad o = (2, -1, 1, 2)^{\mathsf{T}}.$$

Then the exact output packets are

$$\boxed{y_e[k] = e[k] + \frac{1}{2}o[k-1]^2,}$$

$$\boxed{y_o[k] = o[k] + \frac{1}{2}e[k]^2.}$$

Evaluating the two packet formulas gives

$$y_e = \left(3, 2, \frac{7}{2}, -\frac{3}{2}\right)^{\mathsf{T}}, \qquad y_o = \left(\frac{5}{2}, -1, \frac{11}{2}, 4\right)^{\mathsf{T}}.$$

For example,

$$y_e[0] = e[0] + \frac{1}{2}o[-1]^2 = 1 + \frac{1}{2}(2)^2 = 3,$$

and

$$y_o[2] = o[2] + \frac{1}{2}e[2]^2 = 1 + \frac{1}{2}(3)^2 = \frac{11}{2}.$$

Interleaving these two channels yields

$$y = \left(3, \frac{5}{2}, 2, -1, \frac{7}{2}, \frac{11}{2}, -\frac{3}{2}, 4\right)^{\mathsf{T}},$$

which agrees exactly with direct nonlinear filtering.

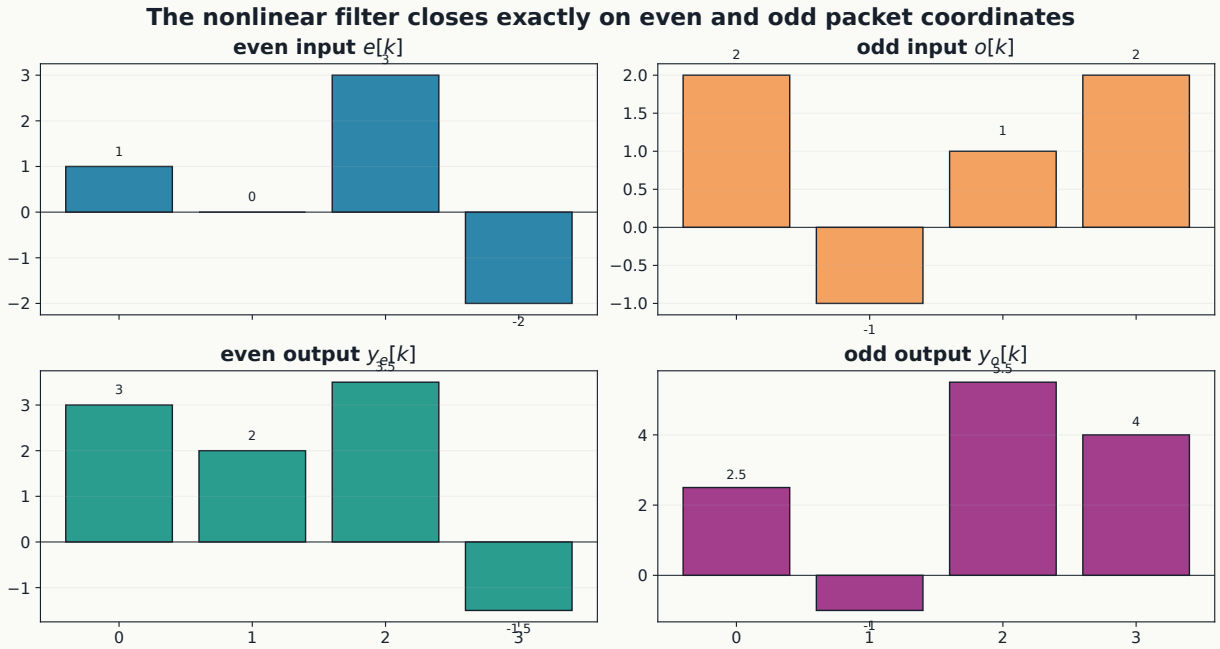


Figure 16.1: The even and odd input packets and the two exact nonlinear output packets. All four coordinates in each channel are shown explicitly.

Proof. At even address $n = 2k$, the previous periodic address is $2k - 1$, which lies in the odd channel at packet index $k - 1$. At odd address $n = 2k + 1$, the previous address is $2k$, which lies in the even channel at index k. Substitution gives the two displayed formulas. Interleaving restores every output address. □

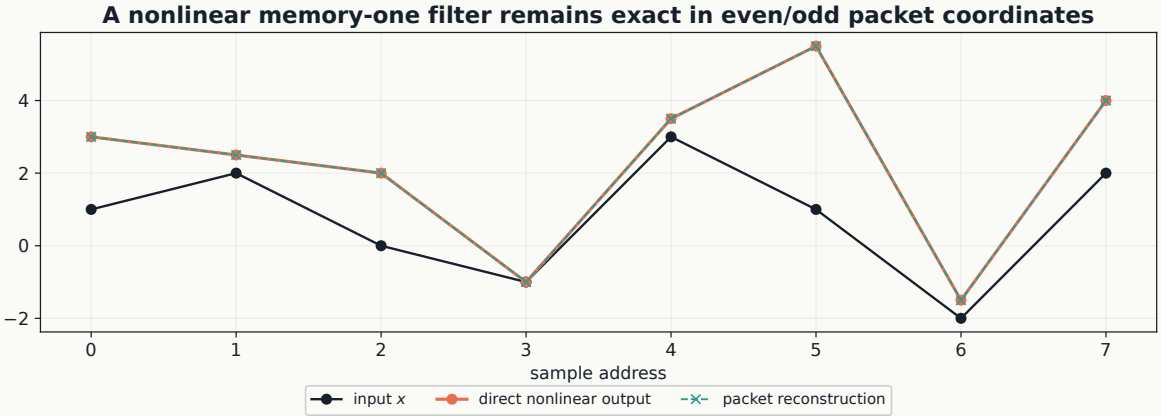


Figure 16.2: Direct nonlinear filtering and packetwise reconstruction coincide.

16.2 Part B: a finite gauge-gluing problem

Let a directed square have vertices v_0, v_1, v_2, v_3. Suppose one edge system is trivial and a second is produced by vertex gauges U_v:

$$B_{vu} = U_v U_u^{-1}.$$

Choose rational orthogonal matrices

$$U_0 = I, \quad U_1 = \begin{pmatrix} 3/5 & -4/5 \\ 4/5 & 3/5 \end{pmatrix},$$

$$U_2 = \begin{pmatrix} 5/13 & -12/13 \\ 12/13 & 5/13 \end{pmatrix}, \quad U_3 = \begin{pmatrix} 7/25 & -24/25 \\ 24/25 & 7/25 \end{pmatrix}.$$

The four exact edge transports are

$$B_{10} = \begin{pmatrix} 3/5 & -4/5 \\ 4/5 & 3/5 \end{pmatrix}, \qquad B_{21} = \begin{pmatrix} 63/65 & -16/65 \\ 16/65 & 63/65 \end{pmatrix},$$

$$B_{32} = \begin{pmatrix} 323/325 & -36/325 \\ 36/325 & 323/325 \end{pmatrix}, \qquad B_{03} = \begin{pmatrix} 7/25 & 24/25 \\ -24/25 & 7/25 \end{pmatrix}.$$

Each is orthogonal with determinant one because it is a rational rotation. Their entries are exact rational numbers, not decimal approximations.



Figure 16.3: The four rational edge-transport matrices. The common color scale makes their signs and magnitudes directly comparable.

The cycle holonomy is

$$H = B_{03}B_{32}B_{21}B_{10}.$$

Telescoping gives

$$H = U_0U_3^{-1}U_3U_2^{-1}U_2U_1^{-1}U_1U_0^{-1} = I.$$

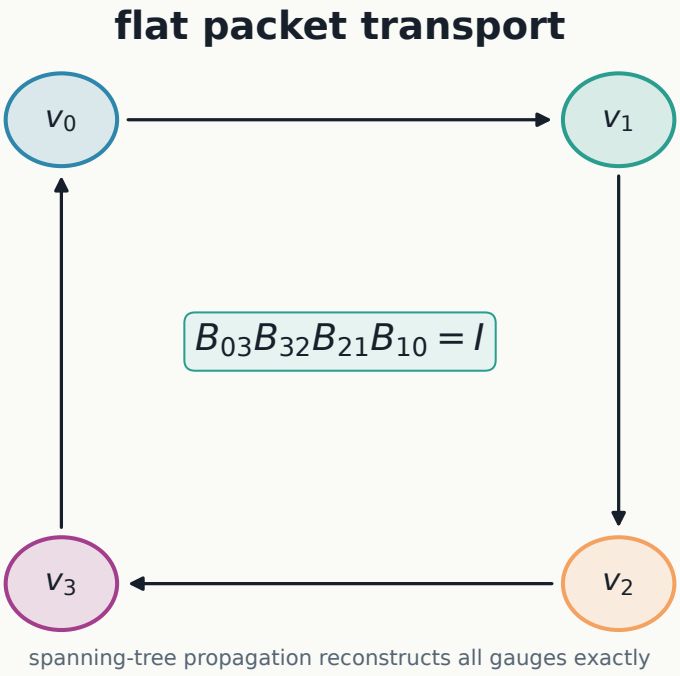


Figure 16.4: A flat packet transport system on one fundamental cycle.

Theorem 16.1 (Finite cycle obstruction)**.** *Fix a root gauge and propagate gauges along a spanning tree. A global gauge exists on the whole connected graph if and only if every non-tree edge closes with the required fundamental-cycle holonomy.*

Proof. Tree propagation determines every vertex gauge uniquely from the root. Each omitted edge imposes one remaining consistency equation. Multiplying that edge equation with the tree path gives the fundamental-cycle holonomy condition. Conversely, if every such cycle closes, all omitted edge equations hold. □

16.3 Negative control

Perturb the closing edge by left multiplication with

$$Q = \begin{pmatrix} 3/5 & -4/5 \\ 4/5 & 3/5 \end{pmatrix}, \qquad B_{03}^{\mathrm{bad}} = QB_{03}.$$

Then

$$H_{\mathrm{bad}} = B_{03}^{\mathrm{bad}} B_{32}B_{21}B_{10} = Q(B_{03}B_{32}B_{21}B_{10}) = Q.$$

The new holonomy is exactly Q, not I. The obstruction packet is

$$H - I = \begin{pmatrix} -2/5 & -4/5 \\ 4/5 & -2/5 \end{pmatrix}.$$

Its squared Frobenius norm is

$$\begin{aligned} \|H - I\|_F^2 &= \left(-\frac{2}{5}\right)^2 + \left(-\frac{4}{5}\right)^2 + \left(\frac{4}{5}\right)^2 + \left(-\frac{2}{5}\right)^2 \\ &= \frac{4}{25} + \frac{16}{25} + \frac{16}{25} + \frac{4}{25} = \boxed{\frac{8}{5}}. \end{aligned}$$

This nonzero residual proves that no gauge satisfying all four perturbed edge equations exists with the chosen root normalization.

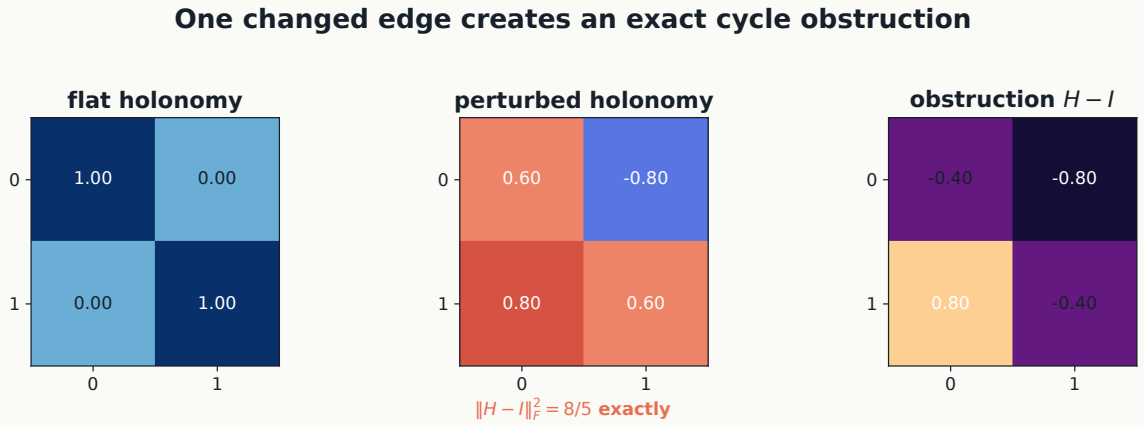


Figure 16.5: Flat holonomy, perturbed holonomy, and the exact obstruction matrix.

Exact certificate

The nonlinear reconstruction residual is exactly zero. The flat cycle residual is exactly zero. The perturbed cycle returns the rational rotation Q exactly, and the obstruction norm squared simplifies to $8/5$.

Part III

Cross-Domain Synthesis

Chapter 17

What the seven examples prove

17.1 One theorem schema, seven domain meanings

Table 17.1: The same finite grammar in seven domains.

Area	Object	Packets	Transfer/refinement	Reconstruction
Linear algebra	vector or matrix	coordinate subspaces	matrix corners	sum of packets/blocks
Signals	sampled signal	phases, bands, Haar coordinates	polyphase and transform maps	interleaving or inverse transform
Quantum	state or density matrix	parity/basis sectors	gate path blocks	coherent path sum
Probability	observable or prior	partition cells	conditional expectation/Bayes update	packet masses and posterior
Arithmetic	analytic seed	exponent residue classes	differentiation, reciprocal powers	block-EGF coefficient synthesis
Physics	displacement field	normal modes	modal evolution	sum of mode solutions
Nonlinear geometry	nonlinear signal or edge system	phase packets/cycle packets	nonlinear interaction and holonomy	interleaving or global gauge

17.2 Seven-realization certificate

Theorem 17.1 (Cross-domain finite validity)**.** *Each of Chapters 10–16 supplies a finite complete packet system or an explicitly invertible analysis-synthesis pair. In each case:*

(i) the declared packet identities hold;

(ii) the domain operator or map has an exact packet form;

(iii) the domain problem is solved at packet level;

(iv) synthesis reproduces the full object or solution;

(v) an independent symbolic computation verifies the identity.

Therefore Projector-Resolved Seed Calculus is a valid common finite calculus for these seven realizations.

Proof. The hypotheses and certificates are established separately in the seven chapters. The conclusion is their conjunction. No cross-domain analogy is used as a substitute for any domain proof. □

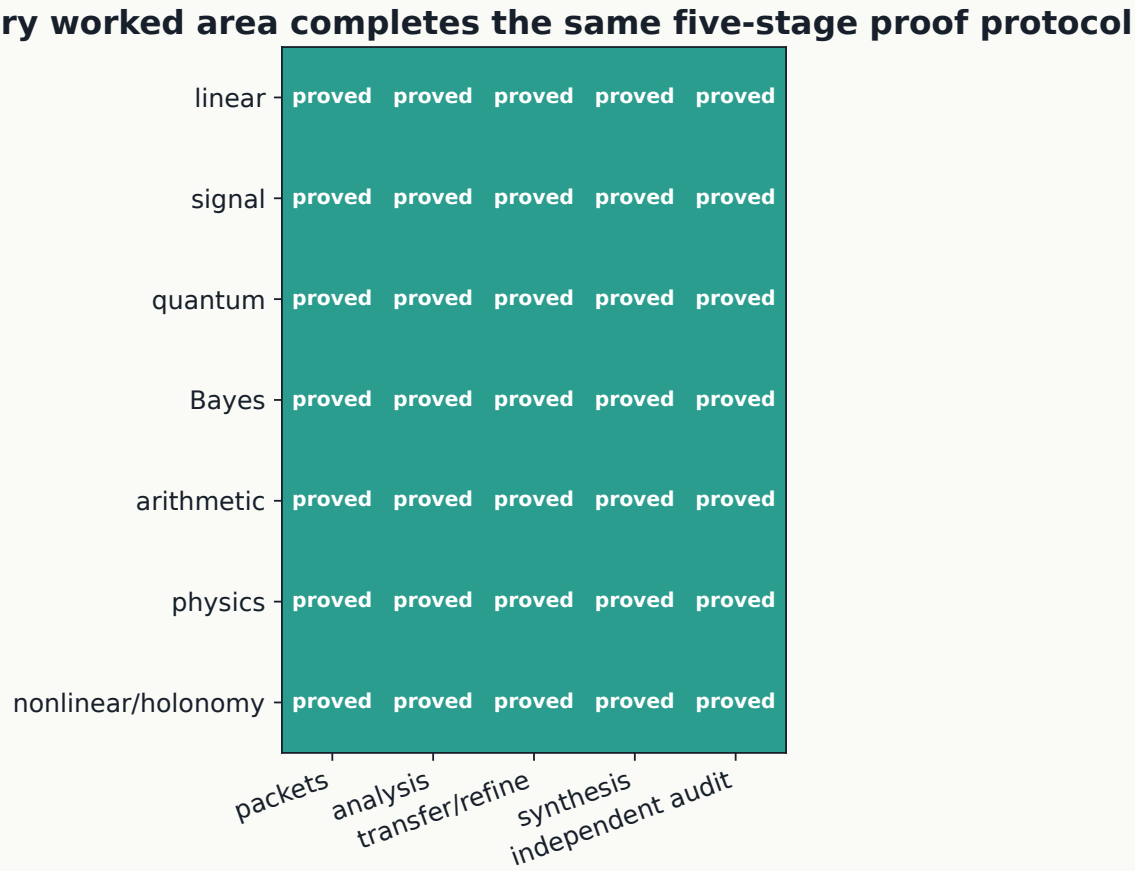


Figure 17.1: Every area completes all five stages of the protocol: packet declaration, analysis, transfer or refinement, synthesis, and an independent audit.

Exact symbolic certificate dashboard

linear object reconstruction	0
linear operator reconstruction	0
linear block product	0
polyphase synthesis	0
Haar synthesis	0
Fourier synthesis	0
Fourier projector completeness	0
quantum path synthesis	0
Bell reduced-state residual	0
conditional expectation idempotence	0
Bayesian sufficient coarsening	0
Bayesian defect factorization	0
sequence recurrence	0
closed coefficient formula	0
oscillator ODE	0
energy partition	0
nonlinear packet synthesis	0
flat holonomy	0
bad holonomy norm squared	8/5 (nonzero obstruction)

All zero entries are exact SymPy identities. The nonzero final row is the intended negative-control obstruction.

Figure 17.2: All intended exact residuals vanish. The final nonzero row is the designed holonomy obstruction.

17.3 Why this is stronger than a list of examples

The examples do not merely reuse the words "packet" and "reconstruction." They instantiate the same algebraic statements:

$$\sum_{\alpha} P_\alpha = I, \qquad x = \sum_{\alpha} P_\alpha x, \qquad T = \sum_{\alpha,\beta} P_\beta T P_\alpha,$$

with domain-specific meanings for X, P_α, and T. The signal and quantum chapters use unitary coordinate transport; the Bayesian chapter uses nonlinear normalization after a linear packet map; the arithmetic chapter uses coefficient projectors and analytic completion; the physics chapter uses spectral projectors; and the holonomy example uses cycle packets as exact obstruction coordinates.

17.4 What remains outside this document

The following are not consequences of seven finite worked realizations:

- universal existence of preferred or optimal packets;
- automatic continuum limits for unbounded operators;
- universal recovery of hidden signal boundaries;
- general quantum-spacetime dynamics;
- unrestricted arithmetic or automorphic reciprocity;
- historical priority for every generated special-function family.

Those questions require additional hypotheses and separate theorems. The value of this volume is that the finite core has now been proved, instantiated, solved, and computationally audited in seven genuinely different settings.

Part IV

Appendices

Appendix A

Exact result ledger

Table A.1: Machine-checked identities.

Certificate	Exact value	Interpretation
Linear object reconstruction	0	Complete packet synthesis
Linear operator reconstruction	0	Sum of transfer blocks
Linear product law	0	Matrix multiplication through packet paths
Linear refinement	0	Parent equals child sum
Polyphase synthesis	0	Exact interleaving
Haar reconstruction	0	Orthogonal filter-bank synthesis
Fourier reconstruction	0	Unitary inverse DFT
Fourier projector completeness	0	Complete band cover
Quantum path synthesis	0	Bell state from two packet paths
Quantum reduced-state residual	0	$\rho_A = I/2$
Conditional-expectation idempotence	0	Finite stochastic projector
Weighted self-adjointness	0	Orthogonality in $L^2(\mu)$
Sufficient Bayesian coarsening	0	Conditioning commutes with coarsening
Bayesian defect factorization	0	Exact information-loss packet
Sequence recurrence	0	Every computed term satisfies the recurrence
Reciprocal-series product	0	$D_5^2 S = 1$ in the computed block range
Hypergeometric coefficient identity	0	Factorial multiplication formula
Composition coefficient formula	0	Independent finite formula
Oscillator ODE residual	0	Exact physical solution
Energy partition residual	0	$E = E_s + E_a$
Nonlinear filter residual	0	Exact nonlinear packet synthesis
Flat holonomy residual	0	Gauge system glues
Perturbed holonomy obstruction	8/5	Intended nonzero negative control
Linear contribution-row reconstruction	0	Three block contributions sum to each output packet
Fourier band-signal reconstruction	0	Five pulled-back band packets sum to the signal
Complete quantum path table	exact	Two nonzero and two zero parity paths
Bayesian same-coarse-prior witness	$(6/145, -6/145)$	Nonsufficiency produces distinct coarse posteriors

Certificate	Exact value	Interpretation
Arithmetic recurrence rows 1–4	exact	Two integer contributions sum to each coefficient
Normal-mode orthogonality	0	Spectral packet basis is orthonormal
Holonomy edge product	I_2	Four rational transports close exactly

Appendix B

Candidate OEIS submission record

Name

Order-five reciprocal-square generalized Euler packet numbers.

Definition

$$\sum_{n\ge 0} a_n \frac{z^{5n}}{(5n)!} = \left(\sum_{n\ge 0} (-1)^n \frac{z^{5n}}{(5n)!}\right)^{-2}.$$

Initial terms

n	a_n
0	1
1	2
2	754
3	3009008
4	58105670266
5	3690819041733782
6	611297487195595912564
7	225925381294863591587930378
8	166595785020134064487394817914506
9	225274815520247848148603077637197668512
10	52298067408509248899678778113649810298464227 4
11	19769334756563379209570151852692700066723425202207 98
12	11650813481135176558042711997383235316672795614675522384596

Formula

The recurrence and closed form are those proved in Chapter 14. The machine-readable b-file is supplied as `tables/candidate_sequence_bfile.txt`.

Status

OEIS-ready candidate record. The exact definition, recurrence, initial values, and b-file are supplied. No historical-priority or database-absence claim is used in the mathematics.

Appendix C

Reproduction commands

From the extracted source directory:

```
python -m pip install -r requirements.txt
python -m pytest -q
python run_all.py
python -m pytest -q
latexmk -pdf Seven_Area_Worked_Proof_Atlas.tex
```

The command `python run_all.py` regenerates all sixty-one figure pairs, every CSV table, the candidate b-file, and the JSON result ledger. The test suite contains thirty-two exact tests: eleven seed-to-reconstruction foundation tests, seven primary domain certificates, seven independent re-derivations, and seven additional audits of the detailed intermediate calculations.

Appendix D

Full symbolic source listings

The following listings are the complete exact calculation modules used in the release.

D.1 Seed-to-reconstruction foundations

```python
from __future__ import annotations

from dataclasses import dataclass
from functools import lru_cache
from typing import Tuple

import sympy as sp


@dataclass(frozen=True)
class FoundationCertificate:
    dimension: int
    seed: sp.Matrix
    shift: sp.Matrix
    orbit_matrix: sp.Matrix
    orbit_rank: int
    relation_residual: sp.Matrix
    signal: sp.Matrix
    signal_polynomial_coefficients: tuple[sp.Integer, ...]
    spatial_projectors: tuple[sp.Matrix, ...]
    spatial_packets: tuple[sp.Matrix, ...]
    spatial_reconstruction_residual: sp.Matrix
    parity_projectors: tuple[sp.Matrix, sp.Matrix]
    mod4_projectors: tuple[sp.Matrix, ...]
    parity_refinement_residuals: tuple[sp.Matrix, sp.Matrix]
    mod4_to_spatial_residuals: tuple[sp.Matrix, ...]
    operator: sp.Matrix
    operator_output: sp.Matrix
    transfer_blocks: tuple[tuple[sp.Matrix, ...], ...]
    transfer_reconstruction_residual: sp.Matrix
    transfer_support: tuple[tuple[int, int], ...]
    second_operator: sp.Matrix
    product_path_residual: sp.Matrix
    fourier_matrix: sp.Matrix
    fourier_projectors: tuple[sp.Matrix, ...]
    fourier_completeness_residual: sp.Matrix
    fourier_idempotence_residuals: tuple[sp.Matrix, ...]
    fourier_annihilation_residuals: tuple[sp.Matrix, ...]
    fourier_diagonal_operator: sp.Matrix
    haar_matrix: sp.Matrix
    multi_atlas_reconstruction_residual: sp.Matrix
    nonlinear_output: sp.Matrix
    nonlinear_packet_residual: sp.Matrix
```

```
    bilinear_interaction_residual: sp.Matrix
    obstruction_projector: sp.Matrix
    solvable_obstruction: sp.Matrix
    blocked_obstruction: sp.Matrix
    dense_extension_residual: sp.Matrix


def cyclic_shift(n: int) -> sp.Matrix:
    U = sp.zeros(n)
    for j in range(n):
        U[(j + 1) % n, j] = 1
    return U


def coordinate_projectors(n: int) -> tuple[sp.Matrix, ...]:
    out = []
    for j in range(n):
        P = sp.zeros(n)
        P[j, j] = 1
        out.append(P)
    return tuple(out)


def unitary_dft(n: int) -> sp.Matrix:
    omega = sp.exp(-2 * sp.pi * sp.I / n)
    return sp.Matrix(n, n, lambda k, j: omega ** (k * j) / sp.sqrt(n))


def haar_matrix(n: int) -> sp.Matrix:
    if n < 2 or n & (n - 1):
        raise ValueError("n must be a power of two")
    if n == 2:
        return sp.Matrix([[1, 1], [1, -1]]) / sp.sqrt(2)
    Hh = haar_matrix(n // 2)
    # Standard orthonormal multilevel Haar transform: coarse transform of pair averages,
    # followed by the finest detail coefficients.
    pair = sp.zeros(n)
    for q in range(n // 2):
        pair[q, 2 * q] = 1 / sp.sqrt(2)
        pair[q, 2 * q + 1] = 1 / sp.sqrt(2)
        pair[n // 2 + q, 2 * q] = 1 / sp.sqrt(2)
        pair[n // 2 + q, 2 * q + 1] = -1 / sp.sqrt(2)
    upper = sp.diag(Hh, sp.eye(n // 2))
    return sp.simplify(upper * pair)


def matrix_zero(M: sp.Matrix) -> bool:
    return all(sp.simplify(v) == 0 for v in M)


@lru_cache(maxsize=1)
def build_certificate() -> FoundationCertificate:
    n = 8
    U = cyclic_shift(n)
    seed = sp.eye(n)[:, 0]
    orbit = sp.Matrix.hstack(*[(U ** k) * seed for k in range(n)])
    relation_residual = sp.simplify(U ** n - sp.eye(n))

    coeffs = tuple(sp.Integer(v) for v in (2, -1, 3, 0, 1, 4, -2, 5))
    x = sum((coeffs[k] * (U ** k) * seed for k in range(n)), sp.zeros(n, 1))

    E = coordinate_projectors(n)
    packets = tuple(P * x for P in E)
    spatial_residual = sp.simplify(sum(packets, sp.zeros(n, 1)) - x)

    P_even = sum((E[j] for j in range(0, n, 2)), sp.zeros(n))
    P_odd = sum((E[j] for j in range(1, n, 2)), sp.zeros(n))
    R = tuple(E[r] + E[r + 4] for r in range(4))
    parity_refinement = (
        sp.simplify(P_even - R[0] - R[2]),
```

```
        sp.simplify(P_odd - R[1] - R[3]),
    )
    mod4_refinement = tuple(sp.simplify(R[r] - E[r] - E[r + 4]) for r in range(4))

    T = 2 * sp.eye(n) - U + 3 * (U ** 2)
    Tx = sp.simplify(T * x)
    blocks = tuple(tuple(sp.simplify(E[b] * T * E[a]) for a in range(n)) for b in range(n))
    Trec = sum((blocks[b][a] for b in range(n) for a in range(n)), sp.zeros(n))
    transfer_residual = sp.simplify(Trec - T)
    support = tuple((b, a) for b in range(n) for a in range(n) if not matrix_zero(blocks[b][a]))

    S = sp.eye(n) + U ** 3
    path_sum = sp.zeros(n)
    for b in range(n):
        for g in range(n):
            for a in range(n):
                path_sum += E[b] * S * E[g] * T * E[a]
    product_residual = sp.simplify(path_sum - S * T)

    F = unitary_dft(n)
    Q = []
    for k in range(n):
        Ek = sp.zeros(n)
        Ek[k, k] = 1
        Q.append(sp.simplify(F.conjugate().T * Ek * F))
    Q = tuple(Q)
    Qsum = sp.simplify(sum(Q, sp.zeros(n)) - sp.eye(n))
    Qidem = tuple(sp.simplify(Qk * Qk - Qk) for Qk in Q)
    Qann = tuple(sp.simplify(Q[i] * Q[j]) for i in range(n) for j in range(n) if i != j)
    Tfourier = sp.simplify(F * T * F.conjugate().T)

    H = haar_matrix(n)
    assert matrix_zero(sp.simplify(H.T * H - sp.eye(n)))
    weights = (sp.Integer(1), sp.Integer(2), sp.Integer(3))
    c0 = weights[0] * x
    c1 = weights[1] * F * x
    c2 = weights[2] * H * x
    xhat = sp.simplify((weights[0] * c0 + weights[1] * F.conjugate().T * c1 + weights[2] * H.T * c2) / sum(w * w
      for w in weights))
    multi_residual = sp.simplify(xhat - x)

    # Nonlinear realization: N(x)=x + (Ux) \odot (U^3 x).
    ux = U * x
    u3x = (U ** 3) * x
    nonlinear = sp.simplify(x + ux.multiply_elementwise(u3x))
    nonlinear_packet_sum = sum((E[b] * nonlinear for b in range(n)), sp.zeros(n, 1))
    nonlinear_packet_residual = sp.simplify(nonlinear_packet_sum - nonlinear)

    # Resolve the bilinear term into all packet-pair interactions.
    interaction = sp.zeros(n, 1)
    for a in range(n):
        for c in range(n):
            left = U * E[a] * x
            right = (U ** 3) * E[c] * x
            interaction += left.multiply_elementwise(right)
    bilinear_residual = sp.simplify(interaction - ux.multiply_elementwise(u3x))

    # Finite obstruction packet for L h = d.
    L = sp.Matrix([[1, 0], [0, 1], [1, 1]])
    Pi = sp.simplify(sp.eye(3) - L * (L.T * L).inv() * L.T)
    d_good = sp.Matrix([1, 2, 3])
    d_bad = sp.Matrix([1, 2, 4])
    obs_good = sp.simplify(Pi * d_good)
    obs_bad = sp.simplify(Pi * d_bad)

    # Dense-extension check: an identity proved on the basis orbit extends linearly to x.
    basis_check = sp.Matrix.hstack(*[(sum(E, sp.zeros(n)) - sp.eye(n)) * orbit[:, j] for j in range(n)])
    dense_extension_residual = sp.simplify(basis_check)

    # Exact checks.
```

```
    assert orbit.rank() == n
    assert matrix_zero(relation_residual)
    assert matrix_zero(spatial_residual)
    assert all(matrix_zero(r) for r in parity_refinement)
    assert all(matrix_zero(r) for r in mod4_refinement)
    assert matrix_zero(transfer_residual)
    assert matrix_zero(product_residual)
    assert matrix_zero(Qsum)
    assert all(matrix_zero(r) for r in Qidem)
    assert all(matrix_zero(r) for r in Qann)
    assert matrix_zero(multi_residual)
    assert matrix_zero(nonlinear_packet_residual)
    assert matrix_zero(bilinear_residual)
    assert matrix_zero(obs_good)
    assert not matrix_zero(obs_bad)
    assert matrix_zero(dense_extension_residual)

    return FoundationCertificate(
        dimension=n,
        seed=seed,
        shift=U,
        orbit_matrix=orbit,
        orbit_rank=orbit.rank(),
        relation_residual=relation_residual,
        signal=x,
        signal_polynomial_coefficients=coeffs,
        spatial_projectors=E,
        spatial_packets=packets,
        spatial_reconstruction_residual=spatial_residual,
        parity_projectors=(P_even, P_odd),
        mod4_projectors=R,
        parity_refinement_residuals=parity_refinement,
        mod4_to_spatial_residuals=mod4_refinement,
        operator=T,
        operator_output=Tx,
        transfer_blocks=blocks,
        transfer_reconstruction_residual=transfer_residual,
        transfer_support=support,
        second_operator=S,
        product_path_residual=product_residual,
        fourier_matrix=F,
        fourier_projectors=Q,
        fourier_completeness_residual=Qsum,
        fourier_idempotence_residuals=Qidem,
        fourier_annihilation_residuals=Qann,
        fourier_diagonal_operator=Tfourier,
        haar_matrix=H,
        multi_atlas_reconstruction_residual=multi_residual,
        nonlinear_output=nonlinear,
        nonlinear_packet_residual=nonlinear_packet_residual,
        bilinear_interaction_residual=bilinear_residual,
        obstruction_projector=Pi,
        solvable_obstruction=obs_good,
        blocked_obstruction=obs_bad,
        dense_extension_residual=dense_extension_residual,
    )
```

D.2 Foundation figure generator

```
from __future__ import annotations

from pathlib import Path

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.patches import Circle, FancyArrowPatch, FancyBboxPatch
```

```
import numpy as np
import sympy as sp

from .foundations import build_certificate

INK = "#17202A"
MUTED = "#566573"
PAPER = "#FAFAF7"
WHITE = "#FFFFFF"
BLUE = "#2E86AB"
CYAN = "#42B8C5"
TEAL = "#2A9D8F"
GREEN = "#6A994E"
YELLOW = "#E9C46A"
ORANGE = "#F4A261"
RED = "#E76F51"
MAGENTA = "#A23E8C"
PURPLE = "#6C5CE7"
COLORS = [BLUE, TEAL, ORANGE, MAGENTA, PURPLE, GREEN, RED, CYAN]

plt.rcParams.update({
    "font.family": "DejaVu Sans",
    "font.size": 10.5,
    "axes.titlesize": 14.0,
    "axes.labelsize": 10.5,
    "axes.edgecolor": INK,
    "axes.labelcolor": INK,
    "xtick.color": INK,
    "ytick.color": INK,
    "text.color": INK,
    "figure.facecolor": PAPER,
    "axes.facecolor": PAPER,
    "savefig.facecolor": PAPER,
    "pdf.fonttype": 42,
    "ps.fonttype": 42,
})


def _save(fig: plt.Figure, out: Path, name: str) -> None:
    out.mkdir(parents=True, exist_ok=True)
    fig.savefig(out / f"{name}.pdf", bbox_inches="tight", pad_inches=0.10)
    fig.savefig(out / f"{name}.png", dpi=220, bbox_inches="tight", pad_inches=0.10)
    plt.close(fig)


def _axes(figsize=(12.8, 6.4)):
    fig, ax = plt.subplots(figsize=figsize)
    ax.set_xlim(0, 1)
    ax.set_ylim(0, 1)
    ax.axis("off")
    return fig, ax


def _balanced_wrap(text: str, max_chars: int = 24) -> str:
    lines = []
    for raw in text.split("\n"):
        if len(raw) <= max_chars or " " not in raw or "$" in raw:
            lines.append(raw)
            continue
        words = raw.split()
        cur = ""
        for word in words:
            if cur and len(cur) + 1 + len(word) > max_chars:
                lines.append(cur)
                cur = word
            else:
                cur = word if not cur else cur + " " + word
        if cur:
            lines.append(cur)
    return "\n".join(lines)
```

```
def _box(ax, x, y, w, h, text, face=WHITE, edge=INK, fontsize=11.0):
    patch = FancyBboxPatch(
        (x, y), w, h,
        boxstyle="round,pad=0.015,rounding_size=0.018",
        facecolor=face, edgecolor=edge, linewidth=1.6, zorder=2,
    )
    ax.add_patch(patch)
    fitted = _balanced_wrap(text, max_chars=max(12, int(31 * w / 0.18)))
    lines = fitted.count("\n") + 1
    size = fontsize
    if lines >= 4:
        size *= 0.82
    elif lines == 3:
        size *= 0.90
    if w < .12:
        size *= .85
    ax.text(x+w/2, y+h/2, fitted, ha="center", va="center", multialignment="center",
            linespacing=1.08, fontsize=max(7.2, size), zorder=4)
    return patch


def _arrow(ax, start, end, color=INK, text=None, rad=0.0, offset=(0, 0), lw=1.7):
    patch = FancyArrowPatch(start, end, arrowstyle="-|>", mutation_scale=13,
                            linewidth=lw, color=color, connectionstyle=f"arc3,rad={rad}", zorder=3)
    ax.add_patch(patch)
    if text:
        mx = (start[0]+end[0])/2 + offset[0]
        my = (start[1]+end[1])/2 + offset[1]
        ax.text(mx, my, text, ha="center", va="center", fontsize=8.8,
                bbox=dict(boxstyle="round,pad=.10", facecolor=PAPER, edgecolor="none", alpha=.96), zorder=5)
    return patch


def _float_matrix(M: sp.Matrix) -> np.ndarray:
    return np.array([[complex(sp.N(M[i, j], 16)) for j in range(M.cols)] for i in range(M.rows)])


def _annotate_matrix(ax, M: np.ndarray, fmt=".0f", fontsize=8.6):
    maxv = max(float(np.max(np.abs(M))), 1e-12)
    for i in range(M.shape[0]):
        for j in range(M.shape[1]):
            value = M[i, j]
            if abs(value.imag) < 1e-10:
                label = format(value.real, fmt)
            else:
                label = f"{value.real:.1f}\n{value.imag:+.1f}i"
            color = WHITE if abs(value) > .52*maxv else INK
            ax.text(j, i, label, ha="center", va="center", fontsize=fontsize, color=color, weight="bold")


def generate_foundation_figures(base: Path) -> int:
    out = base / "figures"
    c = build_certificate()
    n = c.dimension

    # 38: Full calculus pipeline.
    fig, ax = _axes((13.2, 7.0))
    ax.text(.5, .955, "From one seed to a certified reconstruction", ha="center", fontsize=23, weight="bold")
    labels = [
        ("0. seed", "primitive datum"),
        ("1. type", "ambient object"),
        ("2. grammar", "admissible operators"),
        ("3. orbit", "generated hull"),
        ("4. relations", "quotient law"),
        ("5. addresses", "packet labels"),
        ("6. project", "$P_\\alpha x$"),
        ("7. transfer", "$P_\\beta T P_\\alpha$"),
        ("8. refine", "parent = children"),
```

```
        ("9. interact", "nonlinear packets"),
        ("10. complete", "limit/topology"),
        ("11. synthesize", "object + certificate"),
    ]
    xs = [0.03, .19, .35, .51, .67, .83]
    ys = [.68, .29]
    for idx, (title, sub) in enumerate(labels):
        row, col = divmod(idx, 6)
        x0, y0 = xs[col], ys[row]
        color = COLORS[idx % len(COLORS)]
        _box(ax, x0, y0, .135, .17, f"{title}\n{sub}", face=color+"20", edge=color, fontsize=10.5)
        if col < 5:
            _arrow(ax, (x0+.135, y0+.085), (xs[col+1]-.006, y0+.085), color=INK, lw=1.35)
    _arrow(ax, (.897, .675), (.897, .46), color=INK, text="continue", offset=(.045, 0), lw=1.35)
    _arrow(ax, (.83, .375), (.165, .375), color=MAGENTA, text="exact reconstruction closes the loop", rad=-.14,
     offset=(0, -.055), lw=1.5)
    ax.text(.5, .075, "Every later domain example instantiates this same chain with different objects, operators
     , projectors, and certificates.", ha="center", color=MUTED, fontsize=11.0)
    _save(fig, out, "38_seed_to_reconstruction_pipeline")

    # 39: Seed-system anatomy.
    fig, ax = _axes((12.8, 6.5))
    ax.text(.5, .93, "The typed seed system separates data from the rules that act on it", ha="center", fontsize
     =21, weight="bold")
    _box(ax, .38, .39, .24, .20, "seed system\n$(X,S,A,R,P,tau)$", face="#E6F0F4", edge=BLUE, fontsize=13)
    items = [
        (.08,.68,"ambient type $X$\nwhere values live",BLUE),
        (.37,.72,"seed family $S$\nprimitive data",TEAL),
        (.70,.68,"operator grammar $A$\nallowed constructions",ORANGE),
        (.08,.18,"relations $R$\nidentities and quotients",MAGENTA),
        (.38,.10,"packets $P$\nexact coordinates",PURPLE),
        (.70,.18,"completion $\\tau$\ntopology or inverse limit",GREEN),
    ]
    for x,y,text,color in items:
        _box(ax,x,y,.22,.14,text,face=color+"20",edge=color,fontsize=11)
        start=(x+.11, y+.07)
        end=(.38 if x<.38 else (.62 if x>.62 else .50), .49 if y>.39 else .42)
        _arrow(ax,start,end,color=color,rad=.08 if x<.4 else -.08,lw=1.25)
    ax.text(.5,.035,"A universality claim is meaningful only when the grammar and relations are fixed before the
      target object is encoded.",ha="center",fontsize=10.7,color=MUTED)
    _save(fig,out,"39_seed_system_anatomy")

    # 40: Orbit matrix.
    M = np.real(_float_matrix(c.orbit_matrix))
    fig, ax = plt.subplots(figsize=(8.4, 6.4))
    im = ax.imshow(M, cmap="Blues", vmin=0, vmax=1)
    ax.set_xticks(range(n), [f"$U^{k}s$" for k in range(n)], rotation=35, ha="right")
    ax.set_yticks(range(n), [f"$e_{j}$" for j in range(n)])
    _annotate_matrix(ax, M, fmt=".0f", fontsize=9)
    ax.set_title("The orbit matrix of the cyclic-shift seed", weight="bold")
    ax.text(.5,-.18,"Every basis coordinate appears exactly once, so the orbit rank is 8.",transform=ax.
     transAxes,ha="center",color=MUTED,fontsize=10.5)
    fig.colorbar(im, ax=ax, fraction=.046, label="coordinate coefficient")
    fig.tight_layout()
    _save(fig,out,"40_seed_orbit_matrix")

    # 41: Hull growth.
    dims = np.arange(1, n+1)
    fig, ax = plt.subplots(figsize=(9.6, 5.2))
    ax.step(np.arange(n), dims, where="mid", lw=2.2, color=BLUE)
    ax.scatter(np.arange(n), dims, s=45, color=TEAL, edgecolor=INK, zorder=3)
    for k,d in enumerate(dims):
        ax.text(k,d+.18,str(d),ha="center",fontsize=9)
    ax.set_xticks(range(n), [f"$U^{k}s$" for k in range(n)])
    ax.set_xlabel("new orbit word admitted")
    ax.set_ylabel("dimension of generated hull")
    ax.set_ylim(.5,9)
    ax.grid(alpha=.18)
    ax.set_title("Generated-hull growth reaches the full ambient space", weight="bold")
    fig.tight_layout()
```

```
    _save(fig,out,"41_generated_hull_growth")

    # 42: Relation and quotient.
    fig, ax = _axes((12.6, 5.9))
    ax.text(.5,.92,"The orbit relation turns a free word algebra into a finite cyclic module",ha="center",
     fontsize=21,weight="bold")
    _box(ax,.04,.42,.24,.18,"free words\n$K[z]e$\n$e,ze,z^2e,\\ldots$",face=BLUE+"20",edge=BLUE,fontsize=13)
    _box(ax,.38,.42,.24,.18,"relation\n$(z^8-1)e=0$\nidentify $z^8e=e$",face=ORANGE+"20",edge=ORANGE,fontsize
     =13)
    _box(ax,.72,.42,.24,.18,"quotient module\n$K[z]/(z^8-1)$\n$\\cong X$",face=TEAL+"20",edge=TEAL,fontsize=13)
    _arrow(ax,(.28,.51),(.38,.51),color=INK,text="impose",offset=(0,.05))
    _arrow(ax,(.62,.51),(.72,.51),color=INK,text="evaluate $z\\mapsto U$",offset=(0,.055))
    ax.text(.5,.20,"The finite relation is not an approximation. It is the exact kernel relation of the shift
     representation.",ha="center",fontsize=11,color=MUTED)
    ax.text(.5,.10,r"$U^8=I$ gives a complete normal form $a_0I+a_1U+\cdots+a_7U^7$.",ha="center",fontsize=14,
     weight="bold")
    _save(fig,out,"42_relation_quotient")

    # 43: Address/refinement tree.
    fig, ax = _axes((13.0, 7.0))
    ax.text(.5,.95,"Address refinement: parity -> residue mod 4 -> individual coordinates",ha="center",fontsize
     =20,weight="bold")
    levels = [
        [(.5,.82,"$X$",INK)],
        [(.27,.63,"even",BLUE),(.73,.63,"odd",ORANGE)],
        [(.14,.42,"$r=0$",TEAL),(.38,.42,"$r=2$",GREEN),(.62,.42,"$r=1$",MAGENTA),(.86,.42,"$r=3$",PURPLE)],
        [(x,.18,f"$e_{j}$",COLORS[j%8]) for j,x in enumerate(np.linspace(.06,.94,8))],
    ]
    for li,nodes in enumerate(levels):
        for x,y,label,color in nodes:
            w=.11 if li<3 else .075
            _box(ax,x-w/2,y-.045,w,.09,label,face=color+"18" if color!=INK else WHITE,edge=color,fontsize=11)
    # edges root to parity
    for x,_,_,color in levels[1]: _arrow(ax,(.5,.775),(x,.68),color=color,lw=1.2)
    # parity to mod4
    for child in levels[2][:2]: _arrow(ax,(.27,.585),(child[0],.465),color=child[3],lw=1.2)
    for child in levels[2][2:]: _arrow(ax,(.73,.585),(child[0],.465),color=child[3],lw=1.2)
    # mod4 to coordinates: R0->{0,4}, R2->{2,6}, R1->{1,5}, R3->{3,7}
    groups=[[0,4],[2,6],[1,5],[3,7]]
    for parent,inds in zip(levels[2],groups,strict=True):
        for j in inds: _arrow(ax,(parent[0],.375),(levels[3][j][0],.225),color=parent[3],lw=1.0)
    ax.text(.5,.055,r"$P_{\rm even}=R_0+R_2$, $P_{\rm odd}=R_1+R_3$, and $R_r=E_r+E_{r+4}$.",ha="center",
     fontsize=13)
    _save(fig,out,"43_address_refinement_tree")

    # 44: Analysis/synthesis packets.
    x = np.real(_float_matrix(c.signal)).ravel()
    fig, axes = plt.subplots(2, 1, figsize=(12.6, 6.2), gridspec_kw={"height_ratios":[1.05,1.55]},
     constrained_layout=True)
    axes[0].stem(range(n),x,linefmt=INK,markerfmt="o",basefmt=" ")
    axes[0].set_title("Object produced by the seed polynomial",loc="left",weight="bold")
    axes[0].set_ylabel("$x_j$")
    bottom=np.zeros(n)
    for j,Px in enumerate(c.spatial_packets):
        vals=np.real(_float_matrix(Px)).ravel()
        axes[1].bar(np.arange(n), vals, bottom=bottom, color=COLORS[j], edgecolor=INK, linewidth=.35, label=f"
     $E_{j}x$")
        bottom += vals
    axes[1].plot(np.arange(n),x,"o",color=INK,label="reconstructed $x$")
    axes[1].set_title("Eight disjoint packets synthesize the object exactly",loc="left",weight="bold")
    axes[1].set_xlabel("coordinate address")
    axes[1].set_ylabel("packet contribution")
    axes[1].legend(loc="upper center",bbox_to_anchor=(.5,-.16),ncol=5,fontsize=8,frameon=True)
    for a in axes: a.grid(alpha=.14); a.set_xticks(range(n))
    _save(fig,out,"44_analysis_synthesis_packets")

    # 45: transfer support matrix.
    support=np.zeros((n,n))
    for b,a in c.transfer_support:
        support[b,a]=1
```

```
    fig,ax=plt.subplots(figsize=(8.1,6.4))
    im=ax.imshow(support,cmap=matplotlib.colors.ListedColormap([PAPER,TEAL]),vmin=0,vmax=1)
    ax.set_xticks(range(n),[f"in {j}" for j in range(n)],rotation=35,ha="right")
    ax.set_yticks(range(n),[f"out {j}" for j in range(n)])
    for i in range(n):
        for j in range(n):
            if support[i,j]: ax.text(j,i,"$T_{%d%d}$"%(i,j),ha="center",va="center",fontsize=7.5,color=WHITE,
     weight="bold")
    ax.set_title("Support of the transfer-block matrix for $T=2I-U+3U^2$",weight="bold")
    ax.text(.5,-.19,"Only the identity, one-step, and two-step cyclic transfer bands are nonzero.",transform=ax.
     transAxes,ha="center",color=MUTED,fontsize=10.2)
    fig.tight_layout()
    _save(fig,out,"45_transfer_support_matrix")

    # 46: same operator in spatial and Fourier coordinates.
    T=np.real(_float_matrix(c.operator))
    Tf=np.abs(_float_matrix(c.fourier_diagonal_operator))
    fig,axes=plt.subplots(1,2,figsize=(12.4,5.2),constrained_layout=True)
    im0=axes[0].imshow(T,cmap="coolwarm",vmin=-3,vmax=3)
    _annotate_matrix(axes[0],T,fmt=".0f",fontsize=8)
    axes[0].set_title("spatial packet matrix",weight="bold")
    axes[0].set_xticks([]); axes[0].set_yticks([])
    im1=axes[1].imshow(Tf,cmap="viridis")
    _annotate_matrix(axes[1],Tf,fmt=".2f",fontsize=8)
    axes[1].set_title("Fourier atlas: diagonal magnitude",weight="bold")
    axes[1].set_xticks([]); axes[1].set_yticks([])
    fig.colorbar(im0,ax=axes[0],fraction=.046,label="signed entry")
    fig.colorbar(im1,ax=axes[1],fraction=.046,label="magnitude")
    fig.suptitle("Coordinate transport changes the visible structure, not the operator",fontsize=17,weight="bold
     ")
    _save(fig,out,"46_operator_spatial_fourier")

    # 47: transfer path multiplication.
    fig,ax=_axes((12.8,6.0))
    ax.text(.5,.92,"Composition is a sum over intermediate packet addresses",ha="center",fontsize=21,weight="
     bold")
    cols=[.12,.50,.88]
    labels=["input packets $a$","intermediate packets $\\gamma$","output packets $b$"]
    for xcol,label in zip(cols,labels,strict=True): ax.text(xcol,.82,label,ha="center",fontsize=12,weight="bold"
     )
    ys=np.linspace(.68,.20,4)
    for layer,xcol in enumerate(cols):
        for j,y in enumerate(ys):
            color=COLORS[(j+2*layer)%8]
            ax.add_patch(Circle((xcol,y),.035,facecolor=color+"30",edgecolor=color,lw=1.5))
            ax.text(xcol,y,str(j),ha="center",va="center",fontsize=9,weight="bold")
    # representative paths, not all 512
    for a in range(4):
        for g in ((a)%4,(a+1)%4):
            _arrow(ax,(cols[0]+.038,ys[a]),(cols[1]-.038,ys[g]),color=BLUE,lw=.9,rad=.04)
            for b in ((g)%4,(g+1)%4):
                _arrow(ax,(cols[1]+.038,ys[g]),(cols[2]-.038,ys[b]),color=MAGENTA,lw=.9,rad=-.04)
    ax.text(.5,.075,r"$P_bSTP_a=\sum_\gamma(P_bSP_\gamma)(P_\gamma TP_a)$; summing $a,b,\gamma$ reconstructs
     $ST$.",ha="center",fontsize=13)
    _save(fig,out,"47_operator_path_sum")

    # 48: refinement incidence matrix.
    incidence=np.zeros((2,4))
    incidence[0,[0,2]]=1
    incidence[1,[1,3]]=1
    fig,ax=plt.subplots(figsize=(8.2,4.8))
    im=ax.imshow(incidence,cmap=matplotlib.colors.ListedColormap([PAPER,ORANGE]),vmin=0,vmax=1,aspect="auto")
    ax.set_xticks(range(4),["$R_0$","$R_1$","$R_2$","$R_3$"])
    ax.set_yticks(range(2),["even parent","odd parent"])
    for i in range(2):
        for j in range(4): ax.text(j,i,"child" if incidence[i,j] else "-",ha="center",va="center",color=WHITE if
      incidence[i,j] else MUTED,weight="bold")
    ax.set_title("The coarse-to-fine incidence matrix is exact, not statistical",weight="bold")
    ax.text(.5,-.20,"Coarsening sums the indicated child packets; refinement expands each parent into exactly
     those descendants.",transform=ax.transAxes,ha="center",color=MUTED,fontsize=10.2)
```

```
    fig.tight_layout()
    _save(fig,out,"48_refinement_incidence")

    # 49: transform atlas.
    fig,ax=_axes((13.0,6.2))
    ax.text(.5,.93,"One object, three exact coordinate atlases",ha="center",fontsize=22,weight="bold")
    _box(ax,.39,.40,.22,.18,"object $x$\nin $X$",face=INK+"10",edge=INK,fontsize=15)
    atlases=[(.05,.66,"spatial atlas\n$I x$",BLUE,"$I$"),(.73,.66,"Fourier atlas\n$F x$",TEAL,"$F$"),(.05,.13,"
     Haar atlas\n$H x$",MAGENTA,"$H$"),(.73,.13,"pulled-back packets\n$U^{-1}E_\\alpha U$",ORANGE,"conjugation")
     ]
    for x0,y0,text,color,arrowtext in atlases:
        _box(ax,x0,y0,.22,.15,text,face=color+"20",edge=color,fontsize=12)
        start=(.39 if x0<.39 else .61,.49)
        end=(x0+.22 if x0<.39 else x0,y0+.075)
        _arrow(ax,start,end,color=color,text=arrowtext,rad=.08 if y0<.4 else -.08,offset=(0,.04 if y0>.4 else
     -.04),lw=1.3)
        _arrow(ax,end,start,color=color,rad=-.14 if y0>.4 else .14,lw=1.0)
    ax.text(.5,.055,"Complete coordinates differ in visibility and sparsity, but each retains an exact synthesis
      map.",ha="center",color=MUTED,fontsize=11)
    _save(fig,out,"49_transform_atlas_portfolio")

    # 50: redundant multi-atlas tight frame.
    fig,ax=_axes((12.8,6.4))
    ax.text(.5,.94,"The weighted spatial-Fourier-Haar portfolio is a finite tight frame",ha="center",fontsize
     =21,weight="bold")
    _box(ax,.05,.42,.15,.17,"$x$\noriginal object",face=BLUE+"18",edge=BLUE,fontsize=13)
    branches=[(.31,.68,"$1\\cdot Ix$",BLUE),(.31,.42,"$2\\cdot Fx$",TEAL),(.31,.16,"$3\\cdot Hx$",MAGENTA)]
    for x0,y0,text,color in branches:
        _box(ax,x0,y0,.17,.13,text,face=color+"20",edge=color,fontsize=13)
        _arrow(ax,(.20,.505),(x0,y0+.065),color=color,lw=1.3)
    _box(ax,.62,.34,.19,.28,"canonical synthesis\n$\\frac{1}{14}(I^*c_0+2F^*c_1+3H^*c_2)$",face=ORANGE+"20",edge
     =ORANGE,fontsize=12)
    for x0,y0,_,color in branches: _arrow(ax,(x0+.17,y0+.065),(.62,.48),color=color,lw=1.3)
    _box(ax,.87,.42,.09,.17,"$\\widehat x=x$",face=TEAL+"20",edge=TEAL,fontsize=13)
    _arrow(ax,(.81,.48),(.87,.505),color=INK,lw=1.5)
    ax.text(.5,.07,r"$S A=(1^2+2^2+3^2)^{-1}(1^2I+2^2I+3^2I)=I$.",ha="center",fontsize=14,weight="bold")
    _save(fig,out,"50_multi_atlas_tight_frame")

    # 51: nonlinear interaction hypergraph.
    fig,ax=_axes((12.8,6.4))
    ax.text(.5,.94,"Nonlinearity replaces one transfer edge by packet interaction hyperedges",ha="center",
     fontsize=21,weight="bold")
    input_pos=[(.10,.68),(.10,.48),(.10,.28)]
    for j,(x0,y0) in enumerate(input_pos):
        ax.add_patch(Circle((x0,y0),.045,facecolor=COLORS[j]+"28",edgecolor=COLORS[j],lw=1.6)); ax.text(x0,y0,f"
     $P_{j}x$",ha="center",va="center",fontsize=9)
    inter_pos=[(.43,.73,"$(0,1)$"),(.43,.52,"$(1,2)$"),(.43,.31,"$(0,2)$")]
    for i,(x0,y0,label) in enumerate(inter_pos):
        _box(ax,x0-.055,y0-.04,.11,.08,label,face=ORANGE+"20",edge=ORANGE,fontsize=10)
    # hyperedge lines
    pairs=[(0,1),(1,2),(0,2)]
    for k,(a,b) in enumerate(pairs):
        y=inter_pos[k][1]
        _arrow(ax,(input_pos[a][0]+.047,input_pos[a][1]),(.375,y),color=COLORS[a],lw=1.0)
        _arrow(ax,(input_pos[b][0]+.047,input_pos[b][1]),(.375,y),color=COLORS[b],lw=1.0)
    out_pos=[(.82,.68),(.82,.48),(.82,.28)]
    for j,(x0,y0) in enumerate(out_pos):
        ax.add_patch(Circle((x0,y0),.050,facecolor=MAGENTA+"22",edgecolor=MAGENTA,lw=1.6)); ax.text(x0,y0,f"$P_{
     j}N(x)$",ha="center",va="center",fontsize=9)
    for k,(_,y,_) in enumerate(inter_pos):
        for j in range(3):
            _arrow(ax,(.485,y),(out_pos[j][0]-.052,out_pos[j][1]),color=MUTED,lw=.75,rad=.06*(j-1))
    ax.text(.5,.10,r"For a bilinear term $B(x,x)$, exact packetization is $B(x,x)=\sum_{a,c}B(P_ax,P_cx)$.",ha="
     center",fontsize=13)
    _save(fig,out,"51_nonlinear_interaction_hypergraph")

    # 52: obstruction geometry.
    Pi=np.real(_float_matrix(c.obstruction_projector))
    fig,axes=plt.subplots(1,2,figsize=(12.4,5.2),constrained_layout=True)
    im=axes[0].imshow(Pi,cmap="coolwarm",vmin=-.4,vmax=.7)
```

```
    _annotate_matrix(axes[0],Pi,fmt=".2f",fontsize=10)
    axes[0].set_title("cokernel obstruction projector",weight="bold")
    axes[0].set_xticks(range(3),["$d_1$","$d_2$","$d_3$"])
    axes[0].set_yticks(range(3),["$o_1$","$o_2$","$o_3$"])
    fig.colorbar(im,ax=axes[0],fraction=.046,label="projector entry")
    vals=np.array([0.0,float(sp.sqrt((c.blocked_obstruction.T*c.blocked_obstruction)[0]))])
    axes[1].bar(["solvable\n$d=(1,2,3)$","blocked\n$d=(1,2,4)$"],vals,color=[TEAL,RED],edgecolor=INK)
    axes[1].set_ylabel("obstruction norm")
    axes[1].set_title("vanishing is the exact continuation criterion",weight="bold")
    for i,v in enumerate(vals): axes[1].text(i,v+.015,f"{v:.3f}",ha="center",weight="bold")
    axes[1].set_ylim(0,max(.7,vals.max()*1.25)); axes[1].grid(alpha=.16,axis="y")
    _save(fig,out,"52_obstruction_packet_geometry")

    # 53: completion/refinement tower.
    fig,ax=_axes((13.0,6.4))
    ax.text(.5,.94,"Finite refinement tower and the declared completion direction",ha="center",fontsize=21,
     weight="bold")
    levels=[(.12,"level 0\nwhole object",1),(.36,"level 1\nparity packets",2),(.60,"level 2\nmod-4 packets",4)
     ,(.84,"level 3\ncoordinate packets",8)]
    for x0,label,count in levels:
        ax.text(x0,.82,label,ha="center",fontsize=11,weight="bold")
        ys=np.linspace(.20,.68,count)
        for j,y in enumerate(ys):
            h=min(.07,.38/max(count,1));
            _box(ax,x0-.045,y-h/2,.09,h,f"{j}",face=COLORS[j%8]+"20",edge=COLORS[j%8],fontsize=8.5)
    for li in range(len(levels)-1):
        xa,counta=levels[li][0],levels[li][2]
        xb,countb=levels[li+1][0],levels[li+1][2]
        ysa=np.linspace(.20,.68,counta); ysb=np.linspace(.20,.68,countb)
        factor=countb//counta
        for a,ya in enumerate(ysa):
            for b in range(a*factor,(a+1)*factor): _arrow(ax,(xa+.047,ya),(xb-.047,ysb[b]),color=MUTED,lw=.7)
    ax.text(.5,.085,"At every level the packets sum to the identity. An infinite theory must additionally
     declare the topology in which this tower converges.",ha="center",fontsize=10.8,color=MUTED)
    _save(fig,out,"53_completion_refinement_tower")

    # 54: master example pipeline.
    fig,ax=_axes((13.2,7.0))
    ax.text(.5,.95,"Master example: the length-eight cyclic signal through the complete calculus",ha="center",
     fontsize=20,weight="bold")
    stages=[
        (.04,.69,"seed\n$s=e_0$",BLUE),(.22,.69,"grammar\n$U$",TEAL),(.40,.69,"hull\n$U^ks$",ORANGE),(.58,.69,"
     relation\n$U^8=I$",MAGENTA),(.76,.69,"object\n$x=p(U)s$",PURPLE),
        (.04,.29,"project\n$E_jx$",GREEN),(.22,.29,"transfer\n$E_bTE_a$",YELLOW),(.40,.29,"refine\nparity/mod 4"
     ,ORANGE),(.58,.29,"atlas\n$F,H,I$",TEAL),(.76,.29,"nonlinear\ninteraction",RED),
    ]
    for x0,y0,text,color in stages:
        _box(ax,x0,y0,.14,.15,text,face=color+"20",edge=color,fontsize=10.5)
    for row in [stages[:5],stages[5:]]:
        for a,b in zip(row,row[1:]): _arrow(ax,(a[0]+.14,a[1]+.075),(b[0]-.005,b[1]+.075),color=INK,lw=1.2)
    _arrow(ax,(.83,.68),(.83,.48),color=INK,text="analyze",offset=(.045,0),lw=1.2)
    _box(ax,.39,.06,.22,.12,"synthesis + residual\n$\\widehat x=x$, $\\widehat T=T$",face=BLUE+"18",edge=BLUE,
     fontsize=12)
    for x0 in (.11,.29,.47,.65,.83): _arrow(ax,(x0,.29),(.50,.18),color=MUTED,lw=.7,rad=(x0-.5)*.3)
    _save(fig,out,"54_master_example_pipeline")

    # 55: master seed and output.
    Tx=np.real(_float_matrix(c.operator_output)).ravel()
    fig,axes=plt.subplots(2,1,figsize=(12.5,6.3),sharex=True,constrained_layout=True)
    axes[0].stem(range(n),x,linefmt=BLUE,markerfmt="o",basefmt=" ")
    axes[0].set_title(r"Seed polynomial $x=(2-U+3U^2+U^4+4U^5-2U^6+5U^7)s$",loc="left",weight="bold")
    axes[0].set_ylabel("$x_j$")
    axes[1].stem(range(n),Tx,linefmt=MAGENTA,markerfmt="o",basefmt=" ")
    axes[1].set_title(r"Operator output $Tx$ for $T=2I-U+3U^2$",loc="left",weight="bold")
    axes[1].set_ylabel("$(Tx)_j$"); axes[1].set_xlabel("cyclic coordinate")
    for a in axes: a.grid(alpha=.16); a.set_xticks(range(n))
    _save(fig,out,"55_master_seed_and_transfer_output")

    # 56: master Fourier spectrum.
    fx=np.array([complex(sp.N(v,16)) for v in c.fourier_matrix*c.signal])
```

```
    fig,axes=plt.subplots(2,1,figsize=(11.5,6.3),constrained_layout=True)
    axes[0].bar(range(n),np.abs(fx),color=COLORS,edgecolor=INK)
    axes[0].set_title("Fourier packet magnitudes",loc="left",weight="bold")
    axes[0].set_ylabel("$|(Fx)_k|$")
    axes[1].bar(range(n),np.angle(fx),color=COLORS,edgecolor=INK)
    axes[1].set_title("Fourier packet phases",loc="left",weight="bold")
    axes[1].set_ylabel("phase (radians)"); axes[1].set_xlabel("frequency packet $k$")
    axes[1].set_ylim(-np.pi,np.pi); axes[1].axhline(0,color=INK,lw=.7)
    for a in axes: a.grid(alpha=.14,axis="y"); a.set_xticks(range(n))
    _save(fig,out,"56_master_fourier_packets")

    # 57: master nonlinear output.
    ny=np.real(_float_matrix(c.nonlinear_output)).ravel()
    fig,ax=plt.subplots(figsize=(11.8,5.2))
    width=.36
    ax.bar(np.arange(n)-width/2,x,width,color=BLUE,edgecolor=INK,label="$x$")
    ax.bar(np.arange(n)+width/2,ny,width,color=RED,edgecolor=INK,label="$N(x)$")
    ax.set_xticks(range(n)); ax.set_xlabel("coordinate packet")
    ax.set_ylabel("value")
    ax.set_title(r"Exact nonlinear synthesis for $N(x)=x+(Ux)\odot(U^3x)$",weight="bold")
    ax.grid(alpha=.15,axis="y")
    ax.legend(loc="upper center",bbox_to_anchor=(.5,-.14),ncol=2)
    fig.tight_layout()
    _save(fig,out,"57_master_nonlinear_reconstruction")

    # 58: master certificate dashboard.
    rows=[
        ("orbit rank",float(c.orbit_rank),8.0),
        ("relation residual",0.0,1.0),
        ("spatial synthesis",0.0,1.0),
        ("transfer reconstruction",0.0,1.0),
        ("path-product residual",0.0,1.0),
        ("Fourier completeness",0.0,1.0),
        ("multi-atlas synthesis",0.0,1.0),
        ("nonlinear synthesis",0.0,1.0),
        ("solvable obstruction",0.0,1.0),
        ("blocked obstruction",float(sp.sqrt((c.blocked_obstruction.T*c.blocked_obstruction)[0])),1.0),
    ]
    fig,ax=plt.subplots(figsize=(12.0,6.4)); ax.axis("off")
    ax.text(.5,.95,"Master seed-to-reconstruction certificate ledger",ha="center",fontsize=20,weight="bold",
     transform=ax.transAxes)
    y=.86
    for i,(name,value,target) in enumerate(rows):
        face=WHITE if i%2==0 else "#EEF1F2"
        ax.add_patch(FancyBboxPatch((.06,y-.032),.88,.052,boxstyle="round,pad=.004,rounding_size=.004",facecolor
     =face,edgecolor="none",transform=ax.transAxes))
        ax.text(.08,y,name,ha="left",va="center",fontsize=10.8,transform=ax.transAxes)
        if name=="orbit rank": label=f"{int(value)}/8"; color=TEAL
        else: label=f"{value:.3e}"; color=TEAL if value<1e-12 else RED
        ax.text(.92,y,label,ha="right",va="center",fontsize=10.8,family="monospace",weight="bold",color=color,
     transform=ax.transAxes)
        y-=.071
    ax.text(.5,.07,"The nonzero blocked-obstruction row is the intended negative control; all reconstruction
     identities vanish exactly in symbolic arithmetic.",ha="center",fontsize=10.4,color=MUTED,transform=ax.
     transAxes)
    _save(fig,out,"58_master_certificate_dashboard")

    # 59: operation closure wheel.
    fig,ax=plt.subplots(figsize=(8.2,8.2)); ax.set_aspect("equal"); ax.axis("off")
    labels=[("seed",BLUE),("generate",CYAN),("relate",TEAL),("project",GREEN),("transfer",YELLOW),("refine",
     ORANGE),("interact",RED),("complete",PURPLE),("synthesize",MAGENTA)]
    theta=np.linspace(90,90-360,len(labels)+1)
    from matplotlib.patches import Wedge
    for i,(label,color) in enumerate(labels):
        wedge=Wedge((0,0),1.0,theta[i+1],theta[i],width=.34,facecolor=color,edgecolor=PAPER,lw=2)
        ax.add_patch(wedge)
        ang=np.deg2rad((theta[i]+theta[i+1])/2)
        rot=(theta[i]+theta[i+1])/2-90
        ax.text(.81*np.cos(ang),.81*np.sin(ang),label,ha="center",va="center",rotation=rot,rotation_mode="anchor
     ",fontsize=12,weight="bold",color=WHITE)
```

```
    ax.add_patch(Circle((0,0),.49,facecolor=PAPER,edgecolor=INK,lw=2))
    ax.text(0,.10,"projector-resolved",ha="center",fontsize=14,weight="bold")
    ax.text(0,-.01,"seed calculus",ha="center",fontsize=20,weight="bold")
    ax.text(0,-.16,"one exact reconstruction law",ha="center",fontsize=10.5,color=MUTED)
    ax.set_xlim(-1.12,1.12); ax.set_ylim(-1.12,1.12)
    ax.set_title("The complete operational cycle",fontsize=20,weight="bold",pad=12)
    _save(fig,out,"59_complete_calculus_wheel")

    # 60: theorem dependency ladder.
    fig,ax=_axes((13.0,7.1))
    ax.text(.5,.95,"Proof dependency ladder: what each stage contributes",ha="center",fontsize=21,weight="bold")
    stages=[
        ("Seed theorem","a controlled source of values",BLUE),
        ("Hull theorem","all admissible operator words",CYAN),
        ("Relation theorem","finite normal forms and quotients",TEAL),
        ("Packet theorem","exact analysis and synthesis",GREEN),
        ("Transfer theorem","operator reconstruction and paths",YELLOW),
        ("Refinement theorem","coarse packets equal child sums",ORANGE),
        ("Nonlinear theorem","interaction tensors and trees",RED),
        ("Completion theorem","finite identities extend in declared topology",PURPLE),
        ("Realization theorem","domain objects inherit the calculus",MAGENTA),
    ]
    ys=np.linspace(.82,.16,len(stages))
    for i,((title,sub,color),y) in enumerate(zip(stages,ys,strict=True)):
        _box(ax,.22,y-.035,.56,.07,f"{title}: {sub}",face=color+"18",edge=color,fontsize=10.5)
        if i<len(stages)-1: _arrow(ax,(.50,y-.04),(.50,ys[i+1]+.04),color=INK,lw=1.0)
    ax.text(.5,.055,"The seven-area chapters begin only after this dependency chain has been proved and
     instantiated in the master cyclic example.",ha="center",fontsize=10.5,color=MUTED)
    _save(fig,out,"60_theorem_dependency_ladder")

    return 23


if __name__ == "__main__":
    root = Path(__file__).resolve().parents[1]
    print(generate_foundation_figures(root))
```

D.3 Finite linear algebra

```
from __future__ import annotations

from functools import lru_cache

from dataclasses import dataclass
from typing import Iterable

import sympy as sp


@dataclass(frozen=True)
class LinearCertificate:
    x: sp.Matrix
    operator: sp.Matrix
    projectors: tuple[sp.Matrix, ...]
    packets: tuple[sp.Matrix, ...]
    blocks: tuple[tuple[sp.Matrix, ...], ...]
    object_residual: sp.Matrix
    operator_residual: sp.Matrix
    product_residual: sp.Matrix
    refinement_residual: sp.Matrix


def coordinate_projectors(block_sizes: Iterable[int]) -> tuple[sp.Matrix, ...]:
    sizes = tuple(int(s) for s in block_sizes)
    if not sizes or any(s <= 0 for s in sizes):
        raise ValueError("block sizes must be positive")
    n = sum(sizes)
```

```
    out: list[sp.Matrix] = []
    start = 0
    for size in sizes:
        P = sp.zeros(n)
        for i in range(start, start + size):
            P[i, i] = 1
        out.append(P)
        start += size
    return tuple(out)


def transfer_blocks(T: sp.Matrix, projectors: tuple[sp.Matrix, ...]) -> tuple[tuple[sp.Matrix, ...], ...]:
    return tuple(
        tuple(Pb * T * Pa for Pa in projectors)
        for Pb in projectors
    )


def reconstruct_operator(blocks: tuple[tuple[sp.Matrix, ...], ...]) -> sp.Matrix:
    if not blocks or not blocks[0]:
        raise ValueError("blocks cannot be empty")
    n = blocks[0][0].rows
    return sum((B for row in blocks for B in row), sp.zeros(n))


def matrix_is_zero(M: sp.Matrix) -> bool:
    return all(sp.simplify(v) == 0 for v in M)


@lru_cache(maxsize=None)
def build_certificate() -> LinearCertificate:
    P = coordinate_projectors((2, 2, 2))
    x = sp.Matrix([2, -1, 3, 4, -2, 5])
    T = sp.Matrix(
        [
            [1, 2, 0, 0, 1, 0],
            [0, 1, 1, 0, 0, 1],
            [2, 0, 1, 1, 0, 0],
            [0, 1, 0, 2, 1, 0],
            [1, 0, 0, 1, 2, 1],
            [0, 0, 1, 0, 0, 3],
        ]
    )
    S = sp.Matrix(
        [
            [2, 0, 1, 0, 0, 0],
            [1, 1, 0, 0, 0, 1],
            [0, 1, 1, 0, 1, 0],
            [0, 0, 1, 1, 0, 0],
            [1, 0, 0, 0, 1, 1],
            [0, 1, 0, 1, 0, 1],
        ]
    )

    packets = tuple(Pa * x for Pa in P)
    object_residual = sum(packets, sp.zeros(6, 1)) - x

    blocks_T = transfer_blocks(T, P)
    operator_residual = reconstruct_operator(blocks_T) - T

    blocks_S = transfer_blocks(S, P)
    blocks_ST = transfer_blocks(S * T, P)
    product_rebuilt = tuple(
        tuple(
            sum((blocks_S[g][b] * blocks_T[b][a] for b in range(len(P))), sp.zeros(6))
            for a in range(len(P))
        )
        for g in range(len(P))
    )
    # Preserve every block residual separately in one block matrix.  A plain
```

```
    # sum could conceal cancellation between distinct packet paths.
    difference_rows = tuple(
        tuple(blocks_ST[g][a] - product_rebuilt[g][a] for a in range(len(P)))
        for g in range(len(P))
    )
    product_residual = sp.Matrix.vstack(
        *(sp.Matrix.hstack(*row) for row in difference_rows)
    )

    # Refine the first 2D packet into its two coordinate children.
    Q0 = sp.diag(1, 0, 0, 0, 0, 0)
    Q1 = sp.diag(0, 1, 0, 0, 0, 0)
    refinement_residual = P[0] - Q0 - Q1

    assert matrix_is_zero(object_residual)
    assert matrix_is_zero(operator_residual)
    assert matrix_is_zero(product_residual)
    assert matrix_is_zero(refinement_residual)

    return LinearCertificate(
        x=x,
        operator=T,
        projectors=P,
        packets=packets,
        blocks=blocks_T,
        object_residual=object_residual,
        operator_residual=operator_residual,
        product_residual=product_residual,
        refinement_residual=refinement_residual,
    )
```

D.4 Signals and transform atlases

```
from __future__ import annotations

from functools import lru_cache

from dataclasses import dataclass

import sympy as sp


@dataclass(frozen=True)
class SignalCertificate:
    signal: sp.Matrix
    phase2: tuple[sp.Matrix, ...]
    phase4: tuple[sp.Matrix, ...]
    polyphase_residual: sp.Matrix
    refinement_residuals: tuple[sp.Matrix, ...]
    haar_coefficients: sp.Matrix
    haar_residual: sp.Matrix
    fourier_coefficients: sp.Matrix
    fourier_residual: sp.Matrix
    fourier_projector_residual: sp.Matrix
    fourier_idempotence_residuals: tuple[sp.Matrix, ...]


def residue_projector(n: int, order: int, phase: int) -> sp.Matrix:
    P = sp.zeros(n)
    for i in range(n):
        if i % order == phase % order:
            P[i, i] = 1
    return P


def polyphase_analyze(x: sp.Matrix, order: int) -> tuple[sp.Matrix, ...]:
    return tuple(sp.Matrix([x[i] for i in range(r, x.rows, order)]) for r in range(order))
```

```
def polyphase_synthesize(channels: tuple[sp.Matrix, ...], length: int) -> sp.Matrix:
    order = len(channels)
    out = sp.zeros(length, 1)
    for r, channel in enumerate(channels):
        for q, value in enumerate(channel):
            index = order * q + r
            if index < length:
                out[index] = value
    return out


def haar_one_level_matrix(n: int) -> sp.Matrix:
    if n <= 0 or n % 2:
        raise ValueError("n must be positive and even")
    H2 = sp.Matrix([[1, 1], [1, -1]]) / sp.sqrt(2)
    H = sp.zeros(n)
    for k in range(n // 2):
        H[2 * k : 2 * k + 2, 2 * k : 2 * k + 2] = H2
    return H


def dft8_matrix() -> sp.Matrix:
    s2 = sp.sqrt(2)
    roots = [
        1,
        (1 - sp.I) / s2,
        -sp.I,
        -(1 + sp.I) / s2,
        -1,
        -(1 - sp.I) / s2,
        sp.I,
        (1 + sp.I) / s2,
    ]
    return sp.Matrix(8, 8, lambda j, k: roots[(j * k) % 8] / sp.sqrt(8))


def transform_projectors(U: sp.Matrix, groups: tuple[tuple[int, ...], ...]) -> tuple[sp.Matrix, ...]:
    n = U.rows
    out: list[sp.Matrix] = []
    Uinv = sp.simplify(U.inv())
    for group in groups:
        E = sp.zeros(n)
        for i in group:
            E[i, i] = 1
        out.append(sp.simplify(Uinv * E * U))
    return tuple(out)


def matrix_simplify(M: sp.Matrix) -> sp.Matrix:
    return M.applyfunc(lambda z: sp.simplify(sp.expand_complex(z)))


def matrix_is_zero(M: sp.Matrix) -> bool:
    return all(sp.simplify(sp.expand_complex(v)) == 0 for v in M)


@lru_cache(maxsize=None)
def build_certificate() -> SignalCertificate:
    x = sp.Matrix([3, 1, 4, 1, 5, 9, 2, 6])

    phase2 = polyphase_analyze(x, 2)
    phase4 = polyphase_analyze(x, 4)
    polyphase_residual = polyphase_synthesize(phase4, 8) - x

    P2 = tuple(residue_projector(8, 2, r) for r in range(2))
    P4 = tuple(residue_projector(8, 4, r) for r in range(4))
    refinement_residuals = (
        P2[0] - P4[0] - P4[2],
```

```
        P2[1] - P4[1] - P4[3],
    )

    H = haar_one_level_matrix(8)
    hc = sp.simplify(H * x)
    haar_residual = matrix_simplify(H.T * hc - x)

    F = dft8_matrix()
    fc = matrix_simplify(F * x)
    fourier_residual = matrix_simplify(F.conjugate().T * fc - x)

    groups = ((0,), (1, 7), (2, 6), (3, 5), (4,))
    Q = transform_projectors(F, groups)
    fourier_projector_residual = matrix_simplify(sum(Q, sp.zeros(8)) - sp.eye(8))
    fourier_idempotence_residuals = tuple(matrix_simplify(Qi * Qi - Qi) for Qi in Q)

    assert matrix_is_zero(polyphase_residual)
    assert all(matrix_is_zero(R) for R in refinement_residuals)
    assert matrix_is_zero(haar_residual)
    assert matrix_is_zero(fourier_residual)
    assert matrix_is_zero(fourier_projector_residual)
    assert all(matrix_is_zero(R) for R in fourier_idempotence_residuals)
    assert matrix_is_zero(matrix_simplify(H.T * H - sp.eye(8)))
    assert matrix_is_zero(matrix_simplify(F.conjugate().T * F - sp.eye(8)))

    return SignalCertificate(
        signal=x,
        phase2=phase2,
        phase4=phase4,
        polyphase_residual=polyphase_residual,
        refinement_residuals=refinement_residuals,
        haar_coefficients=hc,
        haar_residual=haar_residual,
        fourier_coefficients=fc,
        fourier_residual=fourier_residual,
        fourier_projector_residual=fourier_projector_residual,
        fourier_idempotence_residuals=fourier_idempotence_residuals,
    )
```

D.5 Quantum circuit

```
from __future__ import annotations

from functools import lru_cache

from dataclasses import dataclass

import sympy as sp


@dataclass(frozen=True)
class QuantumCertificate:
    psi0: sp.Matrix
    psi1: sp.Matrix
    bell: sp.Matrix
    even_component: sp.Matrix
    odd_component: sp.Matrix
    path_even: sp.Matrix
    path_odd: sp.Matrix
    path_residual: sp.Matrix
    norm_residual: sp.Expr
    reduced_density: sp.Matrix
    reduced_density_residual: sp.Matrix


def kron(A: sp.Matrix, B: sp.Matrix) -> sp.Matrix:
    return sp.kronecker_product(A, B)
```

```

def partial_trace_second_qubit(rho: sp.Matrix) -> sp.Matrix:
    out = sp.zeros(2)
    for a in range(2):
        for ap in range(2):
            out[a, ap] = sum(rho[2 * a + b, 2 * ap + b] for b in range(2))
    return sp.simplify(out)


def matrix_is_zero(M: sp.Matrix) -> bool:
    return all(sp.simplify(v) == 0 for v in M)


@lru_cache(maxsize=None)
def build_certificate() -> QuantumCertificate:
    I2 = sp.eye(2)
    H = sp.Matrix([[1, 1], [1, -1]]) / sp.sqrt(2)
    H1 = kron(H, I2)
    CNOT = sp.Matrix(
        [
            [1, 0, 0, 0],
            [0, 1, 0, 0],
            [0, 0, 0, 1],
            [0, 0, 1, 0],
        ]
    )
    psi0 = sp.Matrix([1, 0, 0, 0])
    psi1 = sp.simplify(H1 * psi0)
    bell = sp.simplify(CNOT * psi1)

    Pe = sp.diag(1, 0, 0, 1)
    Po = sp.diag(0, 1, 1, 0)
    even_component = Pe * bell
    odd_component = Po * bell

    path_even = sp.simplify(Pe * CNOT * Pe * H1 * Pe * psi0)
    path_odd = sp.simplify(Pe * CNOT * Po * H1 * Pe * psi0)
    path_residual = sp.simplify(path_even + path_odd - bell)

    norm_residual = sp.simplify((bell.conjugate().T * bell)[0] - 1)
    rho = sp.simplify(bell * bell.conjugate().T)
    rho_A = partial_trace_second_qubit(rho)
    reduced_density_residual = sp.simplify(rho_A - sp.eye(2) / 2)

    assert matrix_is_zero(path_residual)
    assert matrix_is_zero(odd_component)
    assert sp.simplify(norm_residual) == 0
    assert matrix_is_zero(reduced_density_residual)

    return QuantumCertificate(
        psi0=psi0,
        psi1=psi1,
        bell=bell,
        even_component=even_component,
        odd_component=odd_component,
        path_even=path_even,
        path_odd=path_odd,
        path_residual=path_residual,
        norm_residual=norm_residual,
        reduced_density=rho_A,
        reduced_density_residual=reduced_density_residual,
    )
```

D.6 Conditional expectation and Bayesian refinement

```
from __future__ import annotations

from functools import lru_cache

from dataclasses import dataclass

import sympy as sp


@dataclass(frozen=True)
class BayesCertificate:
    measure: sp.Matrix
    observable: sp.Matrix
    expectation_matrix: sp.Matrix
    projected_observable: sp.Matrix
    expectation_idempotence_residual: sp.Matrix
    weighted_adjoint_residual: sp.Matrix
    sufficient_fine_posterior: sp.Matrix
    sufficient_coarse_posterior: sp.Matrix
    sufficient_residual: sp.Matrix
    nonsufficient_exact_coarse: sp.Matrix
    nonsufficient_nominal_coarse: sp.Matrix
    defect: sp.Matrix
    factorized_defect: sp.Matrix
    factorization_residual: sp.Matrix


def coarsen(v: sp.Matrix, blocks: tuple[tuple[int, ...], ...]) -> sp.Matrix:
    return sp.Matrix([sum(v[i] for i in block) for block in blocks])


def bayes_update(prior: sp.Matrix, likelihood: sp.Matrix) -> sp.Matrix:
    weights = prior.multiply_elementwise(likelihood)
    z = sp.simplify(sum(weights))
    if z == 0:
        raise ValueError("normalizing constant is zero")
    return sp.simplify(weights / z)


def conditional_expectation_matrix(mu: sp.Matrix, blocks: tuple[tuple[int, ...], ...]) -> sp.Matrix:
    n = mu.rows
    E = sp.zeros(n)
    for block in blocks:
        mass = sp.simplify(sum(mu[j] for j in block))
        for i in block:
            for j in block:
                E[i, j] = sp.simplify(mu[j] / mass)
    return E


def effective_likelihood(prior: sp.Matrix, likelihood: sp.Matrix, blocks: tuple[tuple[int, ...], ...]) -> sp.
    Matrix:
    vals = []
    for block in blocks:
        mass = sp.simplify(sum(prior[i] for i in block))
        numerator = sp.simplify(sum(prior[i] * likelihood[i] for i in block))
        vals.append(sp.simplify(numerator / mass))
    return sp.Matrix(vals)


def matrix_is_zero(M: sp.Matrix) -> bool:
    return all(sp.simplify(v) == 0 for v in M)


@lru_cache(maxsize=None)
def build_certificate() -> BayesCertificate:
    blocks = ((0, 1), (2, 3))
    mu = sp.Matrix([sp.Rational(1, 10), sp.Rational(2, 10), sp.Rational(3, 10), sp.Rational(4, 10)])
    f = sp.Matrix([1, 4, -2, 3])
    E = conditional_expectation_matrix(mu, blocks)
```

```
    Ef = sp.simplify(E * f)
    expectation_idempotence_residual = sp.simplify(E * E - E)
    M = sp.diag(*list(mu))
    weighted_adjoint_residual = sp.simplify(M * E - E.T * M)

    prior = mu
    lam_sufficient = sp.Matrix([2, 2, 1, 1])
    fine_post = bayes_update(prior, lam_sufficient)
    coarse_prior = coarsen(prior, blocks)
    coarse_lam = sp.Matrix([2, 1])
    coarse_post = bayes_update(coarse_prior, coarse_lam)
    sufficient_residual = sp.simplify(coarsen(fine_post, blocks) - coarse_post)

    lam = sp.Matrix([1, 3, 2, 4])
    fine_post_ns = bayes_update(prior, lam)
    exact_coarse = coarsen(fine_post_ns, blocks)
    r = effective_likelihood(prior, lam, blocks)
    exact_via_r = bayes_update(coarse_prior, r)
    assert matrix_is_zero(sp.simplify(exact_coarse - exact_via_r))

    nominal_lam = sp.Matrix([2, 3])
    nominal_coarse = bayes_update(coarse_prior, nominal_lam)
    defect = sp.simplify(exact_coarse - nominal_coarse)

    delta = sp.simplify(r - nominal_lam)
    Zr = sp.simplify((coarse_prior.T * r)[0])
    factor_matrix = sp.diag(*list(coarse_prior)) - nominal_coarse * coarse_prior.T
    factorized = sp.simplify(factor_matrix * delta / Zr)
    factorization_residual = sp.simplify(defect - factorized)

    assert matrix_is_zero(expectation_idempotence_residual)
    assert matrix_is_zero(weighted_adjoint_residual)
    assert matrix_is_zero(sufficient_residual)
    assert matrix_is_zero(factorization_residual)
    assert Ef == sp.Matrix([3, 3, sp.Rational(6, 7), sp.Rational(6, 7)])

    return BayesCertificate(
        measure=mu,
        observable=f,
        expectation_matrix=E,
        projected_observable=Ef,
        expectation_idempotence_residual=expectation_idempotence_residual,
        weighted_adjoint_residual=weighted_adjoint_residual,
        sufficient_fine_posterior=fine_post,
        sufficient_coarse_posterior=coarse_post,
        sufficient_residual=sufficient_residual,
        nonsufficient_exact_coarse=exact_coarse,
        nonsufficient_nominal_coarse=nominal_coarse,
        defect=defect,
        factorized_defect=factorized,
        factorization_residual=factorization_residual,
    )
```

D.7 Arithmetic root-filter sequence

```
from __future__ import annotations

from dataclasses import dataclass
from functools import lru_cache
from itertools import product

import sympy as sp


@dataclass(frozen=True)
class ArithmeticCertificate:
    sequence: tuple[sp.Integer, ...]
```

```
    recurrence_residuals: tuple[sp.Integer, ...]
    series_residual: sp.Expr
    hypergeometric_residual: sp.Expr
    composition_formula_residuals: tuple[sp.Integer, ...]
    parameter_polynomials: tuple[sp.Expr, ...]


def transfer_secancy_coefficients(N: int, r: sp.Expr, nmax: int) -> tuple[sp.Expr, ...]:
    vals: list[sp.Expr] = [sp.Integer(1)]
    for n in range(1, nmax + 1):
        first = -sum(
            (-1) ** k * sp.binomial(N * n - 1, N * k) * vals[n - k]
            for k in range(1, n)
        )
        second = -r * sum(
            (-1) ** k * sp.binomial(N * n - 1, N * k - 1) * vals[n - k]
            for k in range(1, n + 1)
        )
        vals.append(sp.factor(sp.expand(first + second)))
    return tuple(vals)


def compositions(n: int, m: int) -> tuple[tuple[int, ...], ...]:
    if m == 0:
        return ((),) if n == 0 else ()
    if m == 1:
        return ((n,),) if n >= 1 else ()
    out: list[tuple[int, ...]] = []
    for first in range(1, n - m + 2):
        for rest in compositions(n - first, m - 1):
            out.append((first,) + rest)
    return tuple(out)


def composition_formula_a5r2(n: int) -> sp.Integer:
    if n == 0:
        return sp.Integer(1)
    total = sp.Rational(0)
    for m in range(1, n + 1):
        inner = sum(
            sp.prod(sp.Rational(1, sp.factorial(5 * part)) for part in comp)
            for comp in compositions(n, m)
        )
        total += (-1) ** (n + m) * (m + 1) * inner
    return sp.Integer(sp.factorial(5 * n) * total)


def d5_root_sum(z: sp.Symbol) -> sp.Expr:
    sqrt5 = sp.sqrt(5)
    a = (1 + sqrt5) / 4
    b = sp.sqrt(10 - 2 * sqrt5) / 4
    c = (1 - sqrt5) / 4
    d = sp.sqrt(10 + 2 * sqrt5) / 4
    return sp.simplify(
        (sp.exp(-z) + 2 * sp.exp(a * z) * sp.cos(b * z) + 2 * sp.exp(c * z) * sp.cos(d * z)) / 5
    )


def d5_truncated_series(z: sp.Symbol, nmax: int) -> sp.Expr:
    return sum((-1) ** n * z ** (5 * n) / sp.factorial(5 * n) for n in range(nmax + 1))


@lru_cache(maxsize=None)
def build_certificate(nmax: int = 12) -> ArithmeticCertificate:
    z = sp.symbols("z")
    seq_expr = transfer_secancy_coefficients(5, sp.Integer(2), nmax)
    seq = tuple(sp.Integer(v) for v in seq_expr)

    recurrence_residuals: list[sp.Integer] = []
    for n in range(1, nmax + 1):
```

```
        rhs = -sum(
            (-1) ** k * sp.binomial(5 * n - 1, 5 * k) * seq[n - k]
            for k in range(1, n)
        ) - 2 * sum(
            (-1) ** k * sp.binomial(5 * n - 1, 5 * k - 1) * seq[n - k]
            for k in range(1, n + 1)
        )
        recurrence_residuals.append(sp.Integer(sp.simplify(seq[n] - rhs)))

    # Independent coefficient check of D_5(z)^{-2}.  First square D_5,
    # then convolve its block-EGF coefficients with the candidate sequence.
    d2 = []
    for n in range(nmax + 1):
        d2.append(sp.Integer(sum(
            sp.binomial(5 * n, 5 * q) * (-1) ** q * (-1) ** (n - q)
            for q in range(n + 1)
        )))
    inverse_residuals = []
    for n in range(nmax + 1):
        coeff = sum(sp.binomial(5 * n, 5 * k) * d2[k] * seq[n - k] for k in range(n + 1))
        inverse_residuals.append(sp.Integer(sp.simplify(coeff - (1 if n == 0 else 0))))
    series_residual = sp.Integer(max(abs(int(v)) for v in inverse_residuals))

    # Hypergeometric coefficient identity, checked exactly term by term.
    hyper_residuals = []
    for n in range(nmax + 1):
        lhs = sp.Rational((-1) ** n, sp.factorial(5 * n))
        rhs = sp.Rational((-1) ** n, 5 ** (5 * n) * sp.factorial(n))
        for j in range(1, 5):
            rhs /= sp.rf(sp.Rational(j, 5), n)
        hyper_residuals.append(sp.simplify(lhs - rhs))
    hypergeometric_residual = sp.Integer(0) if all(v == 0 for v in hyper_residuals) else sp.Integer(1)

    comp_res = tuple(sp.Integer(seq[n] - composition_formula_a5r2(n)) for n in range(nmax + 1))
    r = sp.symbols("r")
    parameter_polynomials = transfer_secancy_coefficients(5, r, 5)

    assert all(v == 0 for v in recurrence_residuals)
    assert series_residual == 0
    assert hypergeometric_residual == 0
    assert all(v == 0 for v in comp_res)
    assert all(v.is_integer for v in seq)

    return ArithmeticCertificate(
        sequence=seq,
        recurrence_residuals=tuple(recurrence_residuals),
        series_residual=series_residual,
        hypergeometric_residual=hypergeometric_residual,
        composition_formula_residuals=comp_res,
        parameter_polynomials=parameter_polynomials,
    )
```

D.8 Coupled oscillators

```
from __future__ import annotations

from functools import lru_cache

from dataclasses import dataclass

import sympy as sp


@dataclass(frozen=True)
class PhysicsCertificate:
    stiffness: sp.Matrix
    symmetric_projector: sp.Matrix
```

```
    antisymmetric_projector: sp.Matrix
    q: sp.Matrix
    v: sp.Matrix
    ode_residual: sp.Matrix
    total_energy: sp.Expr
    symmetric_energy: sp.Expr
    antisymmetric_energy: sp.Expr
    energy_partition_residual: sp.Expr


def matrix_is_zero(M: sp.Matrix) -> bool:
    return all(sp.simplify(v) == 0 for v in M)


@lru_cache(maxsize=None)
def build_certificate() -> PhysicsCertificate:
    t = sp.symbols("t", real=True)
    K = sp.Matrix([[2, -1], [-1, 2]])
    Ps = sp.Matrix([[1, 1], [1, 1]]) / 2
    Pa = sp.Matrix([[1, -1], [-1, 1]]) / 2
    q0 = sp.Matrix([1, 0])
    v0 = sp.Matrix([0, 0])

    q = sp.simplify(
        sp.cos(t) * Ps * q0
        + sp.sin(t) * Ps * v0
        + sp.cos(sp.sqrt(3) * t) * Pa * q0
        + sp.sin(sp.sqrt(3) * t) / sp.sqrt(3) * Pa * v0
    )
    v = sp.simplify(q.diff(t))
    ode_residual = sp.simplify(q.diff(t, 2) + K * q)

    total_energy = sp.simplify((v.T * v)[0] / 2 + (q.T * K * q)[0] / 2)
    qs, qa = sp.simplify(Ps * q), sp.simplify(Pa * q)
    vs, va = sp.simplify(Ps * v), sp.simplify(Pa * v)
    Es = sp.simplify((vs.T * vs)[0] / 2 + (qs.T * qs)[0] / 2)
    Ea = sp.simplify((va.T * va)[0] / 2 + 3 * (qa.T * qa)[0] / 2)
    energy_partition_residual = sp.simplify(total_energy - Es - Ea)

    assert matrix_is_zero(Ps * Ps - Ps)
    assert matrix_is_zero(Pa * Pa - Pa)
    assert matrix_is_zero(Ps * Pa)
    assert matrix_is_zero(Ps + Pa - sp.eye(2))
    assert matrix_is_zero(K - Ps - 3 * Pa)
    assert matrix_is_zero(ode_residual)
    assert sp.simplify(total_energy - 1) == 0
    assert sp.simplify(Es - sp.Rational(1, 4)) == 0
    assert sp.simplify(Ea - sp.Rational(3, 4)) == 0
    assert energy_partition_residual == 0

    return PhysicsCertificate(
        stiffness=K,
        symmetric_projector=Ps,
        antisymmetric_projector=Pa,
        q=q,
        v=v,
        ode_residual=ode_residual,
        total_energy=total_energy,
        symmetric_energy=Es,
        antisymmetric_energy=Ea,
        energy_partition_residual=energy_partition_residual,
    )
```

D.9 Nonlinear packets and holonomy

```
from __future__ import annotations

```

```
from functools import lru_cache

from dataclasses import dataclass

import sympy as sp


@dataclass(frozen=True)
class NonlinearGeometryCertificate:
    signal: sp.Matrix
    direct_output: sp.Matrix
    packet_output: sp.Matrix
    nonlinear_residual: sp.Matrix
    flat_holonomy: sp.Matrix
    flat_holonomy_residual: sp.Matrix
    perturbed_holonomy: sp.Matrix
    obstruction_matrix: sp.Matrix
    obstruction_norm_squared: sp.Expr


def nonlinear_filter(x: sp.Matrix, alpha: sp.Rational) -> sp.Matrix:
    n = x.rows
    return sp.Matrix([x[i] + alpha * x[(i - 1) % n] ** 2 for i in range(n)])


def packet_nonlinear_filter(x: sp.Matrix, alpha: sp.Rational) -> sp.Matrix:
    e = sp.Matrix([x[i] for i in range(0, x.rows, 2)])
    o = sp.Matrix([x[i] for i in range(1, x.rows, 2)])
    m = e.rows
    ye = sp.Matrix([e[k] + alpha * o[(k - 1) % m] ** 2 for k in range(m)])
    yo = sp.Matrix([o[k] + alpha * e[k] ** 2 for k in range(m)])
    out = sp.zeros(x.rows, 1)
    for k in range(m):
        out[2 * k] = ye[k]
        out[2 * k + 1] = yo[k]
    return out


def rotation(a: sp.Rational, b: sp.Rational) -> sp.Matrix:
    if sp.simplify(a * a + b * b - 1) != 0:
        raise ValueError("a^2+b^2 must equal 1")
    return sp.Matrix([[a, -b], [b, a]])


def matrix_is_zero(M: sp.Matrix) -> bool:
    return all(sp.simplify(v) == 0 for v in M)


@lru_cache(maxsize=None)
def build_certificate() -> NonlinearGeometryCertificate:
    x = sp.Matrix([1, 2, 0, -1, 3, 1, -2, 2])
    alpha = sp.Rational(1, 2)
    direct = nonlinear_filter(x, alpha)
    packet = packet_nonlinear_filter(x, alpha)
    nonlinear_residual = sp.simplify(packet - direct)

    U = [
        sp.eye(2),
        rotation(sp.Rational(3, 5), sp.Rational(4, 5)),
        rotation(sp.Rational(5, 13), sp.Rational(12, 13)),
        rotation(sp.Rational(7, 25), sp.Rational(24, 25)),
    ]
    # Directed cycle 0->1->2->3->0. A-edges are identity; B_{v,u}=U_v U_u^{-1}.
    B10 = sp.simplify(U[1] * U[0].inv())
    B21 = sp.simplify(U[2] * U[1].inv())
    B32 = sp.simplify(U[3] * U[2].inv())
    B03 = sp.simplify(U[0] * U[3].inv())
    flat_holonomy = sp.simplify(B03 * B32 * B21 * B10)
    flat_residual = sp.simplify(flat_holonomy - sp.eye(2))

```

```
    Q = rotation(sp.Rational(3, 5), sp.Rational(4, 5))
    B03_bad = sp.simplify(Q * B03)
    bad_holonomy = sp.simplify(B03_bad * B32 * B21 * B10)
    obstruction = sp.simplify(bad_holonomy - sp.eye(2))
    obstruction_norm_squared = sp.simplify(sum(v * v for v in obstruction))

    assert matrix_is_zero(nonlinear_residual)
    assert matrix_is_zero(flat_residual)
    assert bad_holonomy == Q
    assert obstruction_norm_squared == sp.Rational(8, 5)

    return NonlinearGeometryCertificate(
        signal=x,
        direct_output=direct,
        packet_output=packet,
        nonlinear_residual=nonlinear_residual,
        flat_holonomy=flat_holonomy,
        flat_holonomy_residual=flat_residual,
        perturbed_holonomy=bad_holonomy,
        obstruction_matrix=obstruction,
        obstruction_norm_squared=obstruction_norm_squared,
    )
```

D.10 Detailed intermediate certificates

```
from __future__ import annotations

from dataclasses import dataclass

import sympy as sp

from . import arithmetic, bayes, linear, nonlinear_geometry, physics, quantum, signal


@dataclass(frozen=True)
class DetailedCertificate:
    linear_contributions: tuple[tuple[sp.Matrix, ...], ...]
    linear_row_residuals: tuple[sp.Matrix, ...]
    fourier_band_packets: tuple[sp.Matrix, ...]
    fourier_band_residual: sp.Matrix
    quantum_path_table: tuple[tuple[str, sp.Matrix], ...]
    quantum_measurement_probabilities: tuple[sp.Expr, ...]
    bayes_witness_coarse_prior: sp.Matrix
    bayes_witness_posterior_1: sp.Matrix
    bayes_witness_posterior_2: sp.Matrix
    bayes_witness_difference: sp.Matrix
    recurrence_contributions: tuple[tuple[sp.Integer, sp.Integer, sp.Integer], ...]
    mode_shapes: tuple[sp.Matrix, sp.Matrix]
    nonlinear_even: sp.Matrix
    nonlinear_odd: sp.Matrix
    nonlinear_even_output: sp.Matrix
    nonlinear_odd_output: sp.Matrix
    holonomy_edges: tuple[sp.Matrix, ...]


def _coarsen(v: sp.Matrix) -> sp.Matrix:
    return sp.Matrix([v[0] + v[1], v[2] + v[3]])


def _bayes(prior: sp.Matrix, likelihood: sp.Matrix) -> sp.Matrix:
    w = prior.multiply_elementwise(likelihood)
    return sp.simplify(w / sum(w))


def build_detailed_certificate() -> DetailedCertificate:
    lc = linear.build_certificate()
    signal_x = sp.Matrix([3, 1, 4, 1, 5, 9, 2, 6])
```

```
    F = signal.dft8_matrix()
    fourier_coefficients = signal.matrix_simplify(F * signal_x)
    qc = quantum.build_certificate()
    ac = arithmetic.build_certificate(6)
    pc = physics.build_certificate()
    ng = nonlinear_geometry.build_certificate()

    linear_contributions = tuple(
        tuple(sp.simplify(lc.blocks[b][a] * lc.x) for a in range(3))
        for b in range(3)
    )
    Tx = sp.simplify(lc.operator * lc.x)
    linear_row_residuals = tuple(
        sp.simplify(sum(linear_contributions[b], sp.zeros(6, 1)) - lc.projectors[b] * Tx)
        for b in range(3)
    )

    groups = ((0,), (1, 7), (2, 6), (3, 5), (4,))
    band_packets = []
    for group in groups:
        selected = sp.zeros(8, 1)
        for k in group:
            selected[k] = fourier_coefficients[k]
        band_packets.append(signal.matrix_simplify(F.conjugate().T * selected))
    fourier_band_packets = tuple(band_packets)
    fourier_band_residual = signal.matrix_simplify(sum(fourier_band_packets, sp.zeros(8, 1)) - signal_x)

    I2 = sp.eye(2)
    H = sp.Matrix([[1, 1], [1, -1]]) / sp.sqrt(2)
    H1 = sp.kronecker_product(H, I2)
    CNOT = sp.Matrix([[1, 0, 0, 0], [0, 1, 0, 0], [0, 0, 0, 1], [0, 0, 1, 0]])
    Pe = sp.diag(1, 0, 0, 1)
    Po = sp.diag(0, 1, 1, 0)
    psi0 = sp.Matrix([1, 0, 0, 0])
    path_table = (
        ("e->e", sp.simplify(Pe * CNOT * Pe * H1 * Pe * psi0)),
        ("o->e", sp.simplify(Pe * CNOT * Po * H1 * Pe * psi0)),
        ("e->o", sp.simplify(Po * CNOT * Pe * H1 * Pe * psi0)),
        ("o->o", sp.simplify(Po * CNOT * Po * H1 * Pe * psi0)),
    )
    probs = tuple(sp.simplify(sp.conjugate(qc.bell[i]) * qc.bell[i]) for i in range(4))

    lam = sp.Matrix([1, 3, 2, 4])
    p1 = sp.Matrix([sp.Rational(1, 10), sp.Rational(2, 10), sp.Rational(3, 10), sp.Rational(4, 10)])
    p2 = sp.Matrix([sp.Rational(2, 10), sp.Rational(1, 10), sp.Rational(4, 10), sp.Rational(3, 10)])
    coarse_prior = _coarsen(p1)
    assert coarse_prior == _coarsen(p2)
    post1 = _coarsen(_bayes(p1, lam))
    post2 = _coarsen(_bayes(p2, lam))
    post_diff = sp.simplify(post1 - post2)

    recurrence_contributions: list[tuple[sp.Integer, sp.Integer, sp.Integer]] = []
    seq = ac.sequence
    for n in range(1, 5):
        term_a = -sum(
            (-1) ** k * sp.binomial(5 * n - 1, 5 * k) * seq[n - k]
            for k in range(1, n)
        )
        term_b = -2 * sum(
            (-1) ** k * sp.binomial(5 * n - 1, 5 * k - 1) * seq[n - k]
            for k in range(1, n + 1)
        )
        recurrence_contributions.append((sp.Integer(term_a), sp.Integer(term_b), sp.Integer(seq[n])))
        assert sp.simplify(term_a + term_b - seq[n]) == 0

    es = sp.Matrix([1, 1]) / sp.sqrt(2)
    ea = sp.Matrix([1, -1]) / sp.sqrt(2)
    assert sp.simplify(pc.stiffness * es - es) == sp.zeros(2, 1)
    assert sp.simplify(pc.stiffness * ea - 3 * ea) == sp.zeros(2, 1)

```

```
    e = sp.Matrix([ng.signal[i] for i in range(0, ng.signal.rows, 2)])
    o = sp.Matrix([ng.signal[i] for i in range(1, ng.signal.rows, 2)])
    alpha = sp.Rational(1, 2)
    ye = sp.Matrix([e[k] + alpha * o[(k - 1) % e.rows] ** 2 for k in range(e.rows)])
    yo = sp.Matrix([o[k] + alpha * e[k] ** 2 for k in range(e.rows)])

    U = [
        sp.eye(2),
        nonlinear_geometry.rotation(sp.Rational(3, 5), sp.Rational(4, 5)),
        nonlinear_geometry.rotation(sp.Rational(5, 13), sp.Rational(12, 13)),
        nonlinear_geometry.rotation(sp.Rational(7, 25), sp.Rational(24, 25)),
    ]
    edges = (
        sp.simplify(U[1] * U[0].inv()),
        sp.simplify(U[2] * U[1].inv()),
        sp.simplify(U[3] * U[2].inv()),
        sp.simplify(U[0] * U[3].inv()),
    )
    assert sp.simplify(edges[3] * edges[2] * edges[1] * edges[0] - sp.eye(2)) == sp.zeros(2)

    assert all(all(sp.simplify(v) == 0 for v in r) for r in linear_row_residuals)
    assert all(sp.simplify(v) == 0 for v in fourier_band_residual)
    assert path_table[2][1] == sp.zeros(4, 1)
    assert path_table[3][1] == sp.zeros(4, 1)
    assert probs == (sp.Rational(1, 2), 0, 0, sp.Rational(1, 2))
    assert post_diff == sp.Matrix([sp.Rational(6, 145), -sp.Rational(6, 145)])
    assert sp.Matrix([v for pair in zip(ye, yo) for v in pair]) == ng.direct_output

    return DetailedCertificate(
        linear_contributions=linear_contributions,
        linear_row_residuals=linear_row_residuals,
        fourier_band_packets=fourier_band_packets,
        fourier_band_residual=fourier_band_residual,
        quantum_path_table=path_table,
        quantum_measurement_probabilities=probs,
        bayes_witness_coarse_prior=coarse_prior,
        bayes_witness_posterior_1=post1,
        bayes_witness_posterior_2=post2,
        bayes_witness_difference=post_diff,
        recurrence_contributions=tuple(recurrence_contributions),
        mode_shapes=(es, ea),
        nonlinear_even=e,
        nonlinear_odd=o,
        nonlinear_even_output=ye,
        nonlinear_odd_output=yo,
        holonomy_edges=edges,
    )
```

D.11 Seed-to-reconstruction foundation tests

```
from __future__ import annotations

from pathlib import Path

import sympy as sp

from proof_atlas.foundations import build_certificate, matrix_zero

ROOT = Path(__file__).resolve().parents[1]


def test_seed_orbit_spans_the_ambient_space() -> None:
    c = build_certificate()
    assert c.orbit_rank == c.dimension == 8
    assert c.orbit_matrix.det() in (1, -1)


```

```
def test_seed_relation_is_exact() -> None:
    c = build_certificate()
    assert matrix_zero(c.relation_residual)
    assert matrix_zero(c.shift ** 8 - sp.eye(8))


def test_spatial_packet_reconstruction_is_exact() -> None:
    c = build_certificate()
    assert matrix_zero(c.spatial_reconstruction_residual)
    assert sum(c.spatial_packets, sp.zeros(8, 1)) == c.signal


def test_refinement_tree_is_exact() -> None:
    c = build_certificate()
    assert all(matrix_zero(r) for r in c.parity_refinement_residuals)
    assert all(matrix_zero(r) for r in c.mod4_to_spatial_residuals)


def test_transfer_blocks_reconstruct_the_operator() -> None:
    c = build_certificate()
    assert matrix_zero(c.transfer_reconstruction_residual)
    rebuilt = sum((c.transfer_blocks[b][a] for b in range(8) for a in range(8)), sp.zeros(8))
    assert rebuilt == c.operator


def test_operator_product_is_sum_over_packet_paths() -> None:
    c = build_certificate()
    assert matrix_zero(c.product_path_residual)


def test_fourier_character_packets_are_complete() -> None:
    c = build_certificate()
    assert matrix_zero(c.fourier_completeness_residual)
    assert all(matrix_zero(r) for r in c.fourier_idempotence_residuals)
    assert all(matrix_zero(r) for r in c.fourier_annihilation_residuals)


def test_weighted_multi_atlas_is_exact() -> None:
    c = build_certificate()
    assert matrix_zero(c.multi_atlas_reconstruction_residual)


def test_nonlinear_packet_interactions_reconstruct() -> None:
    c = build_certificate()
    assert matrix_zero(c.nonlinear_packet_residual)
    assert matrix_zero(c.bilinear_interaction_residual)


def test_obstruction_packet_has_positive_and_negative_controls() -> None:
    c = build_certificate()
    assert matrix_zero(c.solvable_obstruction)
    assert c.blocked_obstruction == sp.Matrix([-sp.Rational(1, 3), -sp.Rational(1, 3), sp.Rational(1, 3)])


def test_foundation_figure_pairs_exist() -> None:
    for index in range(38, 61):
        matches_pdf = list((ROOT / "figures").glob(f"{index:02d}_*.pdf"))
        matches_png = list((ROOT / "figures").glob(f"{index:02d}_*.png"))
        assert len(matches_pdf) == len(matches_png) == 1
        assert matches_pdf[0].stat().st_size > 1000
        assert matches_png[0].stat().st_size > 1000
```

D.12 Implementation-certificate tests

```
from __future__ import annotations

import sympy as sp
```

```

from proof_atlas import arithmetic, bayes, linear, nonlinear_geometry, physics, quantum, signal


def matrix_zero(M: sp.Matrix) -> bool:
    return all(sp.simplify(sp.expand_complex(v)) == 0 for v in M)


def test_linear_certificate() -> None:
    c = linear.build_certificate()
    assert matrix_zero(c.object_residual)
    assert matrix_zero(c.operator_residual)
    assert matrix_zero(c.product_residual)
    assert matrix_zero(c.refinement_residual)


def test_signal_certificate() -> None:
    c = signal.build_certificate()
    assert matrix_zero(c.polyphase_residual)
    assert all(matrix_zero(x) for x in c.refinement_residuals)
    assert matrix_zero(c.haar_residual)
    assert matrix_zero(c.fourier_residual)
    assert matrix_zero(c.fourier_projector_residual)
    assert all(matrix_zero(x) for x in c.fourier_idempotence_residuals)


def test_quantum_certificate() -> None:
    c = quantum.build_certificate()
    assert matrix_zero(c.path_residual)
    assert matrix_zero(c.odd_component)
    assert sp.simplify(c.norm_residual) == 0
    assert matrix_zero(c.reduced_density_residual)


def test_bayes_certificate() -> None:
    c = bayes.build_certificate()
    assert matrix_zero(c.expectation_idempotence_residual)
    assert matrix_zero(c.weighted_adjoint_residual)
    assert matrix_zero(c.sufficient_residual)
    assert matrix_zero(c.factorization_residual)
    assert c.nonsufficient_exact_coarse == sp.Matrix([sp.Rational(7,29), sp.Rational(22,29)])
    assert c.nonsufficient_nominal_coarse == sp.Matrix([sp.Rational(2,9), sp.Rational(7,9)])


def test_arithmetic_certificate() -> None:
    c = arithmetic.build_certificate(12)
    assert c.sequence[:6] == (1, 2, 754, 3009008, 58105670266, 3690819041733782)
    assert all(x == 0 for x in c.recurrence_residuals)
    assert c.series_residual == 0
    assert c.hypergeometric_residual == 0
    assert all(x == 0 for x in c.composition_formula_residuals)


def test_physics_certificate() -> None:
    c = physics.build_certificate()
    assert matrix_zero(c.ode_residual)
    assert sp.simplify(c.total_energy - 1) == 0
    assert sp.simplify(c.symmetric_energy - sp.Rational(1,4)) == 0
    assert sp.simplify(c.antisymmetric_energy - sp.Rational(3,4)) == 0
    assert sp.simplify(c.energy_partition_residual) == 0


def test_nonlinear_geometry_certificate() -> None:
    c = nonlinear_geometry.build_certificate()
    assert matrix_zero(c.nonlinear_residual)
    assert matrix_zero(c.flat_holonomy_residual)
    assert sp.simplify(c.obstruction_norm_squared - sp.Rational(8,5)) == 0
```

D.13 Independent re-derivation audit

```python
from __future__ import annotations

import sympy as sp


def is_zero_matrix(matrix: sp.Matrix) -> bool:
    return all(sp.simplify(sp.expand_complex(value)) == 0 for value in matrix)


def test_independent_linear_packet_audit() -> None:
    projectors = (
        sp.diag(1, 1, 0, 0, 0, 0),
        sp.diag(0, 0, 1, 1, 0, 0),
        sp.diag(0, 0, 0, 0, 1, 1),
    )
    x = sp.Matrix([2, -1, 3, 4, -2, 5])
    T = sp.Matrix([
        [1, 2, 0, 0, 1, 0], [0, 1, 1, 0, 0, 1],
        [2, 0, 1, 1, 0, 0], [0, 1, 0, 2, 1, 0],
        [1, 0, 0, 1, 2, 1], [0, 0, 1, 0, 0, 3],
    ])
    S = sp.Matrix([
        [2, 0, 1, 0, 0, 0], [1, 1, 0, 0, 0, 1],
        [0, 1, 1, 0, 1, 0], [0, 0, 1, 1, 0, 0],
        [1, 0, 0, 0, 1, 1], [0, 1, 0, 1, 0, 1],
    ])
    assert is_zero_matrix(sum(projectors, sp.zeros(6)) - sp.eye(6))
    assert is_zero_matrix(sum((P * x for P in projectors), sp.zeros(6, 1)) - x)
    assert is_zero_matrix(
        sum((Pb * T * Pa for Pb in projectors for Pa in projectors), sp.zeros(6)) - T
    )
    for Pg in projectors:
        for Pa in projectors:
            direct = Pg * S * T * Pa
            paths = sum((Pg * S * Pb * T * Pa for Pb in projectors), sp.zeros(6))
            assert is_zero_matrix(direct - paths)


def test_independent_signal_transform_audit() -> None:
    x = sp.Matrix([3, 1, 4, 1, 5, 9, 2, 6])
    channels = tuple(sp.Matrix([x[i] for i in range(r, 8, 4)]) for r in range(4))
    rebuilt = sp.zeros(8, 1)
    for r, channel in enumerate(channels):
        for q, value in enumerate(channel):
            rebuilt[4 * q + r] = value
    assert rebuilt == x

    H2 = sp.Matrix([[1, 1], [1, -1]]) / sp.sqrt(2)
    H = sp.diag(H2, H2, H2, H2)
    assert is_zero_matrix(H.T * H - sp.eye(8))
    assert is_zero_matrix(H.T * H * x - x)

    roots = [
        1, (1 - sp.I) / sp.sqrt(2), -sp.I, -(1 + sp.I) / sp.sqrt(2),
        -1, -(1 - sp.I) / sp.sqrt(2), sp.I, (1 + sp.I) / sp.sqrt(2),
    ]
    F = sp.Matrix(8, 8, lambda j, k: roots[(j * k) % 8] / sp.sqrt(8))
    assert is_zero_matrix(F.conjugate().T * F - sp.eye(8))
    groups = ((0,), (1, 7), (2, 6), (3, 5), (4,))
    packets = []
    for group in groups:
        E = sp.zeros(8)
        for i in group:
            E[i, i] = 1
        packets.append(sp.simplify(F.conjugate().T * E * F))
    assert is_zero_matrix(sum(packets, sp.zeros(8)) - sp.eye(8))
    for i, Pi in enumerate(packets):
```

```
        assert is_zero_matrix(Pi * Pi - Pi)
        for j, Pj in enumerate(packets):
            if i != j:
                assert is_zero_matrix(Pi * Pj)


def test_independent_quantum_audit() -> None:
    I2 = sp.eye(2)
    H = sp.Matrix([[1, 1], [1, -1]]) / sp.sqrt(2)
    CNOT = sp.Matrix([[1, 0, 0, 0], [0, 1, 0, 0], [0, 0, 0, 1], [0, 0, 1, 0]])
    psi = CNOT * sp.kronecker_product(H, I2) * sp.Matrix([1, 0, 0, 0])
    target = sp.Matrix([1, 0, 0, 1]) / sp.sqrt(2)
    assert is_zero_matrix(psi - target)
    rho = sp.simplify(psi * psi.conjugate().T)
    reduced = sp.Matrix(2, 2, lambda a, ap: sum(rho[2 * a + b, 2 * ap + b] for b in range(2)))
    assert is_zero_matrix(reduced - sp.eye(2) / 2)
    assert sp.simplify((psi.conjugate().T * psi)[0] - 1) == 0


def test_independent_probability_bayesian_audit() -> None:
    mu = sp.Matrix([sp.Rational(1, 10), sp.Rational(2, 10), sp.Rational(3, 10), sp.Rational(4, 10)])
    f = sp.Matrix([1, 4, -2, 3])
    E = sp.Matrix([
        [sp.Rational(1, 3), sp.Rational(2, 3), 0, 0],
        [sp.Rational(1, 3), sp.Rational(2, 3), 0, 0],
        [0, 0, sp.Rational(3, 7), sp.Rational(4, 7)],
        [0, 0, sp.Rational(3, 7), sp.Rational(4, 7)],
    ])
    assert is_zero_matrix(E * E - E)
    assert E * f == sp.Matrix([3, 3, sp.Rational(6, 7), sp.Rational(6, 7)])
    M = sp.diag(*list(mu))
    assert is_zero_matrix(M * E - E.T * M)

    lam = sp.Matrix([1, 3, 2, 4])
    fine_weights = mu.multiply_elementwise(lam)
    fine_post = fine_weights / sum(fine_weights)
    exact_coarse = sp.Matrix([fine_post[0] + fine_post[1], fine_post[2] + fine_post[3]])
    assert exact_coarse == sp.Matrix([sp.Rational(7, 29), sp.Rational(22, 29)])
    P = sp.Matrix([sp.Rational(3, 10), sp.Rational(7, 10)])
    nominal_lam = sp.Matrix([2, 3])
    nominal = P.multiply_elementwise(nominal_lam) / (P.T * nominal_lam)[0]
    r = sp.Matrix([sp.Rational(7, 3), sp.Rational(22, 7)])
    delta = r - nominal_lam
    Zr = (P.T * r)[0]
    factor = (sp.diag(*list(P)) - nominal * P.T) * delta / Zr
    assert is_zero_matrix(exact_coarse - nominal - factor)


def test_independent_arithmetic_series_audit() -> None:
    # Independent exact power-series inversion using only rational arithmetic.
    from fractions import Fraction
    from math import factorial

    nmax = 8
    d = [Fraction((-1) ** n, factorial(5 * n)) for n in range(nmax + 1)]
    q = [sum((d[k] * d[n - k] for k in range(n + 1)), Fraction(0, 1)) for n in range(nmax + 1)]
    c = [Fraction(0, 1) for _ in range(nmax + 1)]
    c[0] = Fraction(1, 1)
    for n in range(1, nmax + 1):
        c[n] = -sum((q[k] * c[n - k] for k in range(1, n + 1)), Fraction(0, 1))

    direct = tuple(factorial(5 * n) * c[n] for n in range(nmax + 1))
    assert direct == (
        1, 2, 754, 3009008, 58105670266, 3690819041733782,
        611297487195595912564, 225925381294863591587930378,
        166595785020134064487394817914506,
    )

    # Independent character-filter audit of the closed form.  The five roots
    # are r_j=-omega^j with omega^5=1.  Their m-th power average vanishes
```

```
    # unless 5 divides m; at m=5n it equals (-1)^n.
    for m in range(31):
        multiplicities = [0] * 5
        for j in range(5):
            multiplicities[(j * m) % 5] += 1
        if m % 5 == 0:
            assert multiplicities == [5, 0, 0, 0, 0]
            filtered = (-1) ** (m // 5)
            target = (-1) ** (m // 5)
        else:
            assert multiplicities == [1, 1, 1, 1, 1]
            filtered = 0  # 1+omega+...+omega^4=0
            target = 0
        assert filtered == target


def test_independent_physics_audit() -> None:
    t = sp.symbols('t', real=True)
    Ps = sp.Matrix([[1, 1], [1, 1]]) / 2
    Pa = sp.Matrix([[1, -1], [-1, 1]]) / 2
    K = sp.Matrix([[2, -1], [-1, 2]])
    q0 = sp.Matrix([1, 0])
    q = sp.cos(t) * Ps * q0 + sp.cos(sp.sqrt(3) * t) * Pa * q0
    assert is_zero_matrix(sp.simplify(q.diff(t, 2) + K * q))
    v = q.diff(t)
    energy = sp.simplify((v.T*v)[0]/2 + (q.T*K*q)[0]/2)
    assert sp.simplify(energy - 1) == 0
    assert is_zero_matrix(K - Ps - 3*Pa)


def test_independent_nonlinear_holonomy_audit() -> None:
    x = sp.Matrix([1, 2, 0, -1, 3, 1, -2, 2])
    alpha = sp.Rational(1, 2)
    direct = sp.Matrix([x[i] + alpha*x[(i-1) % 8]**2 for i in range(8)])
    e = sp.Matrix(x[0::2, 0])
    o = sp.Matrix(x[1::2, 0])
    ye = sp.Matrix([e[k] + alpha*o[(k-1) % 4]**2 for k in range(4)])
    yo = sp.Matrix([o[k] + alpha*e[k]**2 for k in range(4)])
    packed = sp.Matrix([value for pair in zip(ye, yo) for value in pair])
    assert packed == direct

    def rot(a: sp.Rational, b: sp.Rational) -> sp.Matrix:
        return sp.Matrix([[a, -b], [b, a]])

    U = [sp.eye(2), rot(sp.Rational(3,5), sp.Rational(4,5)),
         rot(sp.Rational(5,13), sp.Rational(12,13)),
         rot(sp.Rational(7,25), sp.Rational(24,25))]
    edges = [U[1]*U[0].inv(), U[2]*U[1].inv(), U[3]*U[2].inv(), U[0]*U[3].inv()]
    flat = sp.simplify(edges[3]*edges[2]*edges[1]*edges[0])
    assert is_zero_matrix(flat - sp.eye(2))
    Q = rot(sp.Rational(3,5), sp.Rational(4,5))
    bad = sp.simplify((Q*edges[3])*edges[2]*edges[1]*edges[0])
    obstruction = bad - sp.eye(2)
    assert bad == Q
    assert sp.simplify(sum(v**2 for v in obstruction) - sp.Rational(8,5)) == 0
```

D.14 Detailed derivation audits

```
from __future__ import annotations

import sympy as sp
from functools import lru_cache
import gc

from proof_atlas import details


```

```
def zero_matrix(M: sp.Matrix) -> bool:
    return all(sp.simplify(sp.expand_complex(v)) == 0 for v in M)


@lru_cache(maxsize=1)
def get_certificate():
    return details.build_detailed_certificate()


def test_linear_contribution_rows_reconstruct_output_packets() -> None:
    c = get_certificate()
    assert all(zero_matrix(r) for r in c.linear_row_residuals)


def test_fourier_band_signals_reconstruct_original_signal() -> None:
    c = get_certificate()
    assert zero_matrix(c.fourier_band_residual)


def test_quantum_all_parity_paths_are_accounted_for() -> None:
    c = get_certificate()
    assert c.quantum_path_table[0][1] == sp.Matrix([sp.sqrt(2)/2, 0, 0, 0])
    assert c.quantum_path_table[1][1] == sp.Matrix([0, 0, 0, sp.sqrt(2)/2])
    assert c.quantum_path_table[2][1] == sp.zeros(4, 1)
    assert c.quantum_path_table[3][1] == sp.zeros(4, 1)
    assert c.quantum_measurement_probabilities == (sp.Rational(1,2), 0, 0, sp.Rational(1,2))


def test_bayesian_same_coarse_prior_can_have_different_coarse_posterior() -> None:
    c = get_certificate()
    assert c.bayes_witness_coarse_prior == sp.Matrix([sp.Rational(3,10), sp.Rational(7,10)])
    assert c.bayes_witness_posterior_1 == sp.Matrix([sp.Rational(7,29), sp.Rational(22,29)])
    assert c.bayes_witness_posterior_2 == sp.Matrix([sp.Rational(1,5), sp.Rational(4,5)])
    assert c.bayes_witness_difference == sp.Matrix([sp.Rational(6,145), -sp.Rational(6,145)])


def test_arithmetic_recurrence_contributions_sum_exactly() -> None:
    c = get_certificate()
    expected = (
        (0, 2, 2),
        (252, 502, 754),
        (1507506, 1501502, 3009008),
        (34919099764, 23186570502, 58105670266),
    )
    assert c.recurrence_contributions == expected


def test_physics_mode_shapes_are_normalized_and_orthogonal() -> None:
    c = get_certificate()
    es, ea = c.mode_shapes
    assert sp.simplify((es.T*es)[0]-1) == 0
    assert sp.simplify((ea.T*ea)[0]-1) == 0
    assert sp.simplify((es.T*ea)[0]) == 0


def test_nonlinear_channels_and_holonomy_edges() -> None:
    c = get_certificate()
    assert c.nonlinear_even == sp.Matrix([1,0,3,-2])
    assert c.nonlinear_odd == sp.Matrix([2,-1,1,2])
    assert c.nonlinear_even_output == sp.Matrix([3,2,sp.Rational(7,2),-sp.Rational(3,2)])
    assert c.nonlinear_odd_output == sp.Matrix([sp.Rational(5,2),-1,sp.Rational(11,2),4])
    product = c.holonomy_edges[3]*c.holonomy_edges[2]*c.holonomy_edges[1]*c.holonomy_edges[0]
    assert zero_matrix(sp.simplify(product-sp.eye(2)))


def teardown_module() -> None:
    get_certificate.cache_clear()
    sp.core.cache.clear_cache()
    gc.collect()
```

Appendix E

Layout and figure audit

The complete seed-to-reconstruction edition treats page fit as a reproducibility condition. Every figure is exported in vector PDF and high-resolution PNG form. The final PDF is rendered page by page, and the audit checks the following:

(i) no legend covers plotted data;

(ii) every plot title, axis label, and tick label remains inside the figure boundary;

(iii) diagram labels are conservatively sized for their boxes;

(iv) each included figure is constrained by both page width and page height;

(v) captions use ragged-right paragraph layout rather than overflowing single lines;

(vi) no LaTeX overfull or underfull boxes remain in the final build.

The machine-readable layout report is included in `results/layout_audit.json`.

Appendix F

Source and claim map

Layer	Source	Role
Seed-to-finish calculus	Chapters 1–9	Proves generation, relations, packets, transfer, paths, refinement, atlases, nonlinear coordinates, obstructions, completion, and final reconstruction
Domain mathematics	Chapters 10–16	Retains all seven original worked problems and their detailed derivations
Symbolic implementation	`proof_atlas/*.py`	Independent exact computation
Tests	`tests/test_foundations.py` and the three retained domain test modules	Eleven foundational tests plus twenty-one retained domain tests
Figures	`figures/`	Visual explanations generated from exact data
Tables/results	`tables/`, `results/`	Machine-readable values and residuals
OEIS note	arithmetic chapter and candidate record	Candidate status only; no unconditional novelty claim

Appendix G

Conclusion

The complete seed-to-reconstruction content of Projector-Resolved Seed Calculus can be stated without metaphor:

$$\boxed{\begin{array}{c}\text{seed and fixed grammar} + \text{relations and complete packets} + \text{typed transfer and nonlinear interaction} \\ + \text{compatible refinement and declared completion} + \text{declared synthesis} \\ = \text{exact reconstruction.}\end{array}}$$

The seven worked areas show the same identity operating on vectors, signals, quantum states, probability models, analytic coefficient families, mechanical modes, nonlinear filters, and gauge transports. The examples differ in meaning and technique, but they share a single finite grammar. That grammar has now been proved abstractly, realized concretely, and checked symbolically from beginning to end.